

Mail-Delivery Robot in Carleton Tunnels using Large Language Model Agents

Final Report Draft

Tommy Phang - 101211316

Owen McKibbon - 101269794

Connor Yelle - 101260641

Jonas Andaya - 101224271

Umniah Mohammed - 101158792

Supervisor: Dr. Babak Esfandiari

A report submitted in partial fulfillment of the requirements
of SYSC 4907: Engineering Project

Department of Systems and Computer Engineering
Faculty of Engineering
Carleton University

Thursday, April 9, 2026

Table of Contents

List of Figures	5
List of Tables	7
List of Abbreviations	8
Abstract	9
1.0 Introduction.....	10
1.1 Project Background.....	10
1.2 Project Motivation	10
1.3 Problem Statement.....	11
1.4 Proposed Solution	11
1.5 Overview	11
2.0 The Engineering Project	13
2.1 Health and Safety	13
2.2 Engineering Professionalism	14
2.3 Project Management	14
2.3.1 Agile methodology.....	14
2.3.2 Proposed Gantt Chart.....	15
2.3.3 Complete timeline.....	16
2.3.4 Risks and Mitigation Strategies	18
2.4 Justification of Suitability for Degree Program	19
2.5 Individual Contributions	20
2.5.1 Project Contributions	20
2.5.2 Report Contributions.....	22
3.0 Requirements	23
3.1 Requirements Process	23
3.2 Function Requirements	24
3.2 Non-Functional Requirements	25
3.3 Use Case Diagram.....	27
3.4 Use Case Tables	28
4.0 Background	29
4.1 Contributions from Last Year	29

4.1.1 Subsumption architecture.....	29
4.2 Tools and Software	30
4.2.1 Robot Operating System.....	30
4.2.2 iRobot Create	31
4.2.3 LiDAR Sensor.....	31
4.2.4 Gazebo	31
4.2.5 Ollama.....	32
4.2.6 UV	32
4.2.7 BluePy.....	33
5.0 Group Skills	34
6.0 Methods.....	36
7.0 Analysis.....	37
7.1 LLM Feasibility Evaluation.....	37
7.1.1 LLM selection criteria for Raspberry Pi 4	37
7.1.2 Evaluation methodology	38
7.1.3 Results and observations.....	38
7.1.4 Conclusion	40
7.2 Analysis of Current System Architecture	40
7.3 Analysis of the Influence of Large Language Model Capacity on Navigation Decision Accuracy	43
7.3.1 Experiment results	44
7.3.2 Results and analysis	44
7.3.3 Conclusion	45
7.4 Effects of Prompt Variations on Model Performance.....	46
7.4.1 Effects of Prompt Variation on Navigation AI.....	46
7.4.2 Effects of Prompt Variation on Avoidance Layer AI	49
7.4.3 Effects of Prompt Variation on Beacon Sensor AI.....	52
7.4.4 Effects of Prompt Variation on Lidar Sensor AI	55
7.4.5 Comparing the Accuracy of a Larger LLM (Gemma 2 9B) with No Prompt Style Against the Accuracy of a Smaller LLM (Qwen 2.5: 0.5B).....	59
7.4.6 Summary of Findings on Effects of Prompt Variations on Model Performance.....	60

8.0 Metrics	61
8.1 Preface.....	61
8.2 Metrics Tracking.....	61
8.3 Automated Report Generation for Metrics Tracking.....	63
8.4 Dashboard	66
9.0 Design	69
9.1 System Overview and ROS 2 Architecture Diagrams	69
9.2 LLM-Augmented Subsumption Architecture	73
10.0 Implementation	75
10.1 LLM Integration Process	75
10.1.1 Injection Methodology.....	75
10.1.2 LLM fallback	76
10.2 Avoidance Layer LLM Implementation	76
10.3 Docking Layer LLM Implementation.....	78
10.4 Turning Layer LLM Implementation.....	80
10.5 Beacon Sensor LLM Implementation	82
10.6 Navigation Unit LLM Implementation	83
10.7 LiDAR Sensor LLM Implementation	84
10.8 AI node Integration	86
10.9 Connecting to the Physical Robot.....	87
10.10 Docker Usage.....	89
10.11 Implementation Challenges	90
10.11.1 Debugging the previous year's version of the codebase	90
10.11.2 Integrating AI Nodes.....	92
11.0 Verification and Validation.....	93
11.1 Continuous Integration.....	93
11.2 Robot Testing.....	94
11.3 Unit Testing	94
11.4 System-Level Validation	95
12.0 Required Components and Facilities	97
13.0 Conclusion	99

14.0 Recommendations	100
15.0 Project Reflections	100
16.0 References	101
Appendix A: Test Data	102
Appendix B: Use Case Tables	103
Appendix C: Create 3 Hardware Setup.....	118
Appendix D: Create 3 Software Setup.....	118
Appendix E: Gazebo Setup	122
Appendix F: Experiment methodology.....	124
Appendix G: GitHub Repository	130
Appendix H: Original Project Proposal	131

List of Figures

Figure 1: Gantt Chart of the Milestones	16
Figure 2: Requirements Engineering Process	23
Figure 3: Managing Conflicts Process	24
Figure 4: Use Case Diagram of Robot Mail-Delivery System	27
Figure 5: Diagram of the subsumption layers from last year's project	29
Figure 6: The system architecture from the previous year's version of the system	40
Figure 7: Vertical Partitioning of Architecture	41
Figure 8: Horizontal Partitioning of Architecture.....	41
Figure 9: Accuracy of Outputting a Valid Navigation Command.....	44
Figure 10: Bar chart of Navigation AI prompt Accuracy	49
Figure 11: Bar chart of Avoidance Layer AI prompt Accuracy	52
Figure 12: Bar chart of Beacon Sensor AI prompt Accuracy	54
Figure 13: Bar chart of LiDAR sensor AI prompt Accuracy.....	58
Figure 14: Bar chart of Comparison Between Prompt Style accuracy and Model Size	59
Figure 15: Report generated after creating a commit in an existing pull request.	65
Figure 16: Robot Dashboard Home screen displaying metrics for test runs	67
Figure 17: What team members would see if they clicked graphs on the Robot dashboard home screen displaying all metric graphs.....	67
Figure 18: What team members would see if they clicked a run day allowing them to view all metric reports for the day, they clicked	68
Figure 19: Overview of the ROS for the mail delivery robot.	69
Figure 20: Overview of the ROS of the mail delivery robot alongside the AI nodes.....	70
Figure 21: Flow of the /navigation topic and Navigation Unit Node.	71
Figure 22: Comparison Overview of the ROS of the mail delivery robot with the AI nodes in Red.	72
Figure 23: LLM-Augmented Subsumption Architecture	73
Figure 24: LLM-Driven avoidance layer output.....	77
Figure 25: Dock decision making code snippet.....	78
Figure 26: System prompt for Ollama	79
Figure 27: lidar_summary() code snippet	81
Figure 28: System Prompt for the Ollama chat	81
Figure 29: Beacon_sensor_AI console output	86
Figure 30: Navigation_unit_AI console output	87
Figure 31: Docking_layer_AI console output.....	87
Figure 32: Tailscale Client running on a personal computer	88
Figure 34: Faulty Docking code snippet.....	91
Figure 35: Fixed Docking code snippet.....	91
Figure 36: Mock Data creation for the LiDAR sensor unit test.....	95
Figure A.37: Qwen 3.0 response times.....	102

Figure A.38: Qwen 2.5:0.5b	102
----------------------------------	-----

List of Tables

Table 1: Comparison of Planned and Actual Milestone Completion Dates	17
Table 2: Project Risks and Mitigation Strategies.....	18
Table 3: Functional Requirements	25
Table 4: Non-Functional Requirements.....	25
Table 5: Table of LLMs Tested	39
Table 6: Table of Pros and Cons for Each Partitioning Method.....	42
Table 7: Table of Metrics and Justifications	61
Table 8: Comparison of Non-Functional Requirements for Tailscale vs. Firebase.....	89
Table 9: List of required Components/Facilities, Purposes, and Estimated Costs	97
Table 10: Use Case Description Table of Simulating in Gazebo	103
Table 11: Use Case Description Table of Viewing Historical Performance	105
Table 12: Use Case Description Table of Monitoring the System Dashboard	106
Table 13: Use Case Description Table of Executing the Navigation Plan	108
Table 14: Use Case Description Table of Navigating the Tunnel Network	110
Table 15: Use Case Description Table of Docking for Charging.....	112
Table 16: Use Case Description Table of Avoiding Obstacles.....	114
Table 17: Use Case Description Table of Update Dashboard on pull request.....	116
Table 18: Use Case Description Table of Assist with sensor data processing	117

List of Abbreviations

AI: Artificial Intelligence
CD: Continuous Development
CI: Continuous Integration
CSS: Cascading Style Sheets
FR: Functional Requirement
HTML: Hyper Text Markup Language
IP: Internet Protocol
LiDAR: Light Detection and Ranging
LLM: Large Language Model
NFR: Non-Functional Requirement
RFS: Reference Flow Step
ROS: Robot Operating System
RSSI: Received Signal Strength Indicator
SDLC: Software Development Life Cycle
SSH: Secure Shell
UI: User Interface
VPN: Virtual Private Network

Abstract

Robotics and artificial intelligence are two technical fields at the forefront of innovation due to their diverse array of modern-day applications. The combination of these two fields is a new topic of research and development with many unanswered questions. This project focuses on the investigation of the benefits and capabilities of integration of AI-based technologies into a robotic system for delivering mail at the Carleton University Campus.

This project implements local LLMs (large language models) to drive dynamic decision making for various journey scenarios of an existing mail delivery robot's delivery trip. Using data collected from a LiDAR (Light Detection and Ranging), bumper sensor, and beacon sensors, LLMs are queried to make dynamic decisions to send commands actuating robot movement. This gives the models responsibility for navigation, avoidance, docking, and other necessary steps to successfully deliver mail.

The objective is to investigate the performance and capabilities of LLMs for a variety of tasks that make up the operation of a robotic system. The performance is measured through responsiveness, accuracy, and many further mentioned metrics.

1.0 Introduction

1.1 Project Background

By Jonas Andaya and Tommy Phang

This project presents the design and implementation of an autonomous mail delivery robot system for use within the Carleton University tunnel network. The system consists of a mobile robot platform equipped with sensors for environmental perception, onboard computation for decision-making, and a software framework that enables navigation, obstacle avoidance, and task execution.

The robot is designed to travel between predefined locations in the tunnel system while safely navigating corridors, avoiding obstacles, and handling intersections. It operates autonomously without direct human control, using sensor data to understand its surroundings and make movement decisions in real time.

A key feature of this system is the integration of a large language model (LLM) into the robot's architecture. The LLM is used to support high-level reasoning tasks such as navigation planning, decision-making at intersections, and interpretation of sensor inputs. This approach differs from previous implementations of the mail delivery robot which relied on fixed rule-based logic.

The system is implemented using a modular architecture that separates perception, decision-making, and actuation. This allows the LLM to interact with structured data from the robot while maintaining compatibility with existing robotics frameworks. The system is tested in both simulation and real-world environments to evaluate its performance and reliability.

This capstone project is the fifth-year iteration of the Carleton University mail delivery robot system. The objective of this project was to explore the use of LLM agents to program an iRobot for mail delivery in the Carleton tunnels.

In addition to the robot system, a software-based reporting tool is developed to collect and analyze performance data. This tool provides insights into navigation accuracy, obstacle avoidance effectiveness, and the consistency of LLM-driven decisions.

1.2 Project Motivation

By Jonas Andaya, Connor Yelle, Umniyah Mohammed, and Tommy Phang

This year, the objective of the mail-delivery robot project was to introduce reliable mail-delivery inside the tunnels of Carleton University using large language models (LLMs). This would provide Carleton University with an alternative method to deliver mail to other buildings and faculty. It would also give faculty more free time to focus on other important tasks.

The current project explored the use of large language model (LLM) agents to program an iRobot for mail delivery in the Carleton tunnels, with an emphasis on maximizing LLM

involvement within the system. The LLM controlled different aspects of the robot. For example, the LLM was tasked to control how it detects and avoids objects and walls and how it moves in the tunnels. Each part of the system was evaluated using a report that displayed performance metrics, alongside a simulation tool that supported testing and analysis.

Throughout the year, the team followed an agile methodology to support an iterative software development lifecycle. Agile practices helped maintain organization and ensure the team progresses at a consistent pace. The team maintained steady progress, meeting sprint goals, and incorporating the project supervisor's feedback is essential to ensuring the project's success.

1.3 Problem Statement

By Owen McKibbin

The objective is to research and implement and analyze the injection of AI tools and LLMs into a mail delivery robot system. The robot must be able to navigate the Carleton University tunnels, avoid obstacles, and successfully deliver mail. This must be attempted through use of LLM models installed locally on the robot while having enough resources to fit computationally. LLM performance must be measured to analyze its capabilities and overall effectiveness within the scope of this project.

1.4 Proposed Solution

By Owen McKibbin

To achieve this goal, the LLMs were injected into individual components of the robot's software architecture to enable decision-making at multiple levels of the system. LLM performance was measured through metrics in robot performance and responsiveness, to view progress in success through iterations. Metrics were formatted into a report for each run using a CI/CD workflow to be viewed and tabulated. Integration testing was then performed to see how ai nodes operated in conjunction for runs.

1.5 Overview

By Owen McKibbin

The following report explores the requirements for the project and background on the original architecture of the system, as well as a plan for the project structure, analysis, implementation, verification, and conclusion. The report will explore the effectiveness of implementing AI tools and LLMs into robotic systems and their uses.

2.0 The Engineering Project

By Connor Yelle

This section contains general engineering practices that were followed during the capstone project. Topics such as health and safety, engineering professionalism, and project management are covered in this section. Each member's contribution to both the project and the report is discussed, along with an explanation of how the project relates to the degree program.

2.1 Health and Safety

By Jonas Andaya and Umniyah Mohammed

This section outlines the precautions the team took to maintain a safe environment for the capstone project members and the public. Safety considerations were treated as a key component of system design, particularly due to the robot operating in shared indoor environments.

The robot was tested in the hallways of the Canal Building (5th floor) and within the Carleton University tunnel system. These locations were selected due to their relatively low pedestrian traffic, allowing for more controlled testing conditions while still representing a realistic deployment environment. Testing in these areas enabled the team to evaluate system performance while minimizing risk to bystanders.

Several potential hazards were identified, including collisions with pedestrians, obstruction of walkways, and risks associated with exposed mechanical or electrical components during maintenance. These hazards were considered high priority due to their potential impact on both public safety and equipment integrity. To mitigate these risks, testing in the tunnel system was scheduled during periods when classes were in session, avoiding peak traffic immediately after class dismissals. This reduced the likelihood of interaction with large groups of pedestrians and minimized the risk of collisions.

In addition, at least one team member always remained near the robot during the operation. This ensured that nearby individuals were aware of the robot's presence and allowed for immediate intervention if unsafe behavior occurred. This supervisory approach acted as a safeguard against unexpected system failures or incorrect navigation decisions.

During hardware interaction, safety precautions were strictly followed. The robot was powered off before any maintenance to eliminate electrical and mechanical risks. Team members avoided wearing loose clothing or jewelry to prevent entanglement with moving components, reducing the likelihood of injury during hardware handling.

Overall, these safety measures established a controlled testing environment and minimized risks to both team members and the public, while ensuring that system development is aligned with responsible engineering practices.

2.2 Engineering Professionalism

By Owen McKibbin and Jonas Andaya

Throughout this project, the principles the team learned in ECOR 4995: Professional Practice were actively applied to guide both technical and team-based decisions. The course emphasized communication and professional engineering responsibilities, among other topics. These concepts were directly reflected in how the team approached the capstone project.

An important topic in ECOR 4995 was about effective communication and that was a key component of the project. The importance of clear, structured communication was applied in the development of the final report and in the discussions between team members. Writing clearly and concisely was particularly important when explaining technical concepts in an understandable way for readers with varying technical backgrounds. Proper citation practices and careful attention to wording were also used to avoid plagiarism and ensure academic integrity throughout the report. Structuring oral presentations was a topic in ECOR 4995 that the team should have applied more in the capstone project as it is a requirement to do an oral presentation for the capstone project.

Teamwork principles covered in ECOR 4995 were also directly applied throughout the project. An understanding of what constitutes an effective team and effective team member was demonstrated through task delegation and shared ownership of deliverables. Clear role distribution and mutual accountability allowed the team to function efficiently across software and documentation tasks. Regular communication and structured collaboration practices have ensured that all members contribute meaningfully while maintaining alignment with project objectives.

ECOR 4995 provided guidance on how the project was communicated and how engineering decisions were evaluated from a professional perspective. These principles were essential in ensuring that the system was developed in a responsible and collaborative manner consistent with professional engineering practice.

2.3 Project Management

To effectively manage the project, the team applied a range of project management techniques throughout the term. These are outlined in the sections that follow.

2.3.1 Agile methodology

By Jonas Andaya

The Agile methodology is an iterative approach to project management that emphasizes flexibility, incremental development, and continuous feedback.

The team worked in short development cycles, setting weekly goals, and reassessing progress during regular team meetings. GitHub is a version control tool that was used to manage code changes and support collaborative development, allowing multiple team members to contribute to different components simultaneously while maintaining system stability.

Tasks were distributed among team members in a way that encouraged shared understanding across all subsystems, rather than strict specialization. This approach improved collaboration and ensured that multiple team members were familiar with each component of the system. As a result, work could continue smoothly even if a team member was unavailable, reducing project risk.

Progress was reviewed through regular meetings with the capstone supervisor, during which the team presented updates on both technical development and report writing. These meetings sometimes included demonstrations of system functionality in both the simulation environment and on the physical robot, as well as discussions on the structure and content of the final report. This provided consistent feedback across both implementation and documentation, supporting an iterative workflow aligned with Agile principles.

2.3.2 Proposed Gantt Chart

By Jonas Andaya

The proposed Gantt chart outlines the initial project schedule, showing planned milestones and their expected completion timelines over the course of the term. It was developed at the beginning of the project to provide a structured plan for task distribution and to guide the team's workflow.

While the Gantt chart served as a useful planning tool, several milestones were completed later than originally scheduled due to integration challenges and unexpected delays in system setup.

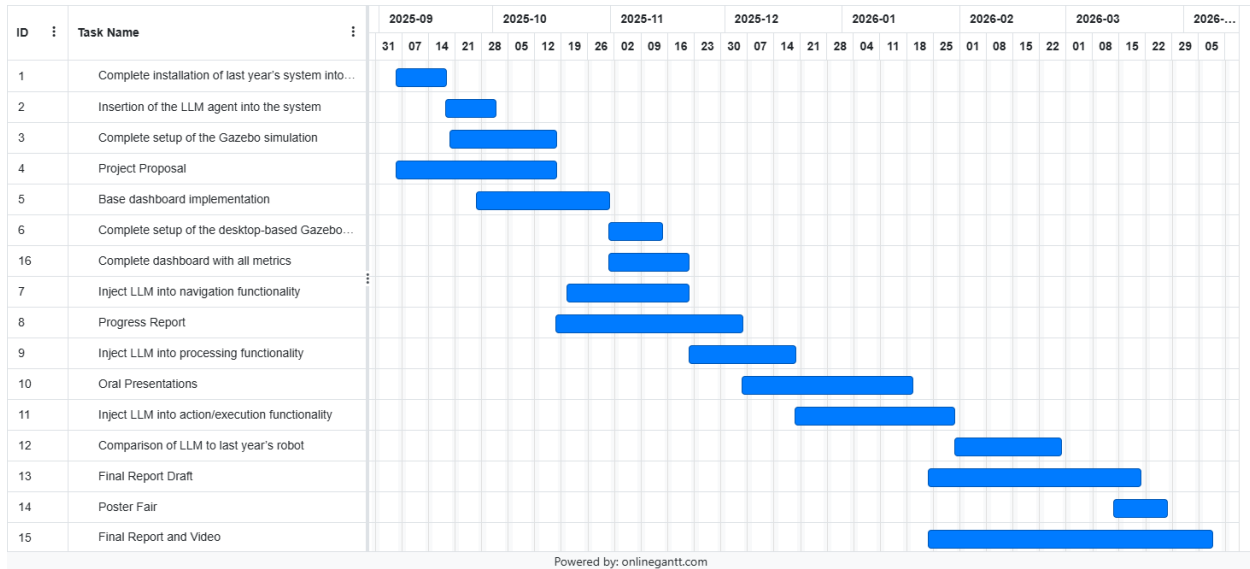


Figure 1: Gantt Chart of the Milestones

2.3.3 Complete timeline

By Jonas Andaya, Owen McKibbin, Umniyah Mohamed, Tommy Phang and Connor Yelle

Several project milestones were completed later than initially planned. The primary source of delay was the integration of the previous year's system into the current robot platform. Significant effort was required to adapt and debug legacy code to function correctly within the hardware and software environment, which impacted progress across multiple milestones.

To mitigate these delays, the team adjusted its development approach by focusing on modular integration of the LLM components. Instead of waiting for the full system to become operational, individual LLM functionalities were developed and tested independently. This approach allowed for faster validation and ensured that once the full system was functional, these components could be integrated with minimal additional delay.

Another challenge encountered was the setup of the simulation environment. Early in the project timeline, insufficient emphasis was placed on establishing a functional simulation, which led to delays later when simulation-based testing became necessary. This highlighted the importance of prioritizing development tools earlier in the project lifecycle.

The table below summarizes the actual completion dates of key project milestones.

Table 1: Comparison of Planned and Actual Milestone Completion Dates

Milestone	Proposed Completion Date	Actual Completion Date	Status
Complete working installation of last year's system into the robot	September 18 th , 2025	November 27 th , 2025	Completed
Insertion of the LLM agent into the system	October 1 st , 2025	October 1 st , 2025	Completed
Project Proposal	October 17 th , 2025	October 17 th , 2025	Completed
Base report generator implementation	October 31 st , 2025	October 31 st , 2025	Completed
Complete setup of the simulation	November 14 th , 2025	November 20 th , 2025	Completed
Inject LLM into navigation functionality	November 21 st , 2025	November 28 th , 2025	Completed
Complete report generator with all metrics	November 28 th , 2025	January 15 th , 2026	Completed
Fall Progress Report	December 5 th , 2025	December 5 th , 2025	Completed
Inject LLM into processing functionality	December 19 th , 2025	February 25 th , 2026	Completed
Oral Presentation	January 19 th -23 rd , 2026	January 20 th , 2026	Completed
Inject LLM into action/execution functionality	January 30 th , 2026	March 18 th , 2026	Completed
Comparison of LLM to last year's robot	February 27 th , 2026		Incomplete
Final Report Draft	March 20 th , 2026	March 20 th , 2026	Completed
Poster Fair	March 27 th , 2026	March 27 th , 2026	Completed
Final Report and Video	April 8 th , 2026	April 8 th , 2026	Completed

2.3.4 Risks and Mitigation Strategies

By Connor Yelle, Owen McKibbin, Tommy Phang

The table below summarizes the main project risks and their mitigation strategies, which were used to guide safe and reliable system development throughout the project.

Table 2: Project Risks and Mitigation Strategies

Project Risk	Mitigation Strategy
LLM latency when querying on Raspberry Pi 4 due to low resources.	The team will analyze alternate locations that the LLM could fit inside the system to lessen the dependency on the AI for real-time robot decision making
Unorganized use of git and conflicting branches and pull requests.	The team will utilize a CI/CD pipeline using GitHub and GitHub actions to ensure all code is tested before being accepted into the codebase. At least one team member will review the pull request to ensure the code conforms to the defined requirements.
The iCreate3 and the Raspberry Pi 4 could be damaged by external or internal forces during operation	The team will ensure that the robot is always in safe operating conditions. This means only testing the robot during non-busy times in the tunnels. The team will also ensure that they do not use software, libraries, or frameworks without prior research on if the Raspberry Pi can run it to ensure the Pi is not damaged during operation.
Electrostatic shock while handling the hardware components in the robot	The team will ensure to handle all robot hardware related material with utmost care and precaution to ensure nobody gets hurt. The Pi will be always held by the edges to ensure no internal circuitry becomes damaged or emits electricity that can shock someone.
Legacy code of previous years failing runs	The team will take advantage of the modularity of ROS to analyze which portions of code are salvageable, and which must be redone to ensure successful trials are completed.

LLM layers and nodes failing to operate in conjunction.	The team will integrate one new AI node or layer at a time and perform integration testing.
---	---

2.4 Justification of Suitability for Degree Program

By Connor Yelle and Umniyah Mohammed

The team, consisting of five members, possesses a complimentary skill set that will contribute to a successful year of working on this capstone project. All five team members are students in the Software Engineering program at Carleton University, each bringing a unique combination of experience, programming proficiency, problem-solving ability, and communication skills. These strengths will support the design and development of a thoughtful and well-designed system for the robot. The team is confident that the knowledge gained through both academic coursework and professional experience will be sufficient to address the technical and organizational challenges presented by this capstone project.

The codebase inherited from previous years is primarily written in Python. Since every team member has prior experience with Python through ECOR 1041 (Computation and Programming), ECOR 1042 (Data Management), and SYSC 2100 (Data Structures and Algorithms), becoming familiar with and extending the existing system should be manageable and efficient. Because the robot operates using a Raspberry Pi, having team members with an understanding of low-level programming languages such as C (developed through SYSC 2006: Foundations of Imperative Programming) will be crucial for ensuring efficient iteration throughout the project.

The robot codebase utilizes middleware called ROS2 (Robot Operating System 2). ROS provides a variety of frameworks, libraries, and tools that assist in developing robot functionality. Since most team members have taken SYSC 4001 (Operating Systems), becoming familiar with ROS should be straightforward. ROS heavily relies on state machines to capture robot states and support decision-making. Therefore, the experience gained from SYSC 3303 (Real-Time Concurrent Systems) will be an asset throughout the project.

To ensure that weekly iterations do not disrupt the overall system, the team is implementing automated testing and automatic report updates through GitHub Actions, a tool introduced in SYSC 4806 (Software Engineering Lab). Testing remains a primary focus throughout the project and will be conducted using techniques learned in SYSC 4101 (Software Validation and Verification).

2.5 Individual Contributions

The sections below outline the contributions of each team member to both the overall project and the final report.

2.5.1 Project Contributions

By Connor Yelle, Jonas Andaya, Owen McKibbon, Tommy Phang and Umniyah Mohammed

Tommy Phang

- Helped debug and getting last year's robot to run
- LLM feasibility and prompt experiments
- Developed Robot Dashboard for tracking metrics over time, displaying graphs
- Developed report generator that tracks the metrics, generates a report after each run on the robot
- Developed a GitHub Actions workflow to automate simulation testing for the mail delivery robot, triggering on pull requests and commits
- Developed a Docker-based environment for the mail delivery robot using ROS 2
- Integrated Docker to run the robot simulation in a consistent, isolated environment
- Automated execution of simulation tests and generation of performance metrics
- Peer review team members codes

Owen McKibbon

- Debugged and tested previous years code on robot and simulation
- Developed methodology for LLM injection
- Proposed partitioning methodology for AI integration
- Wrote, developed, and tested: ai navigation unit, ai avoidance layer, ai lidar sensor
- Contributed to development and debugging of beacon sensor ai node
- Integrated ai navigation, ai docking, and ai beacon sensor nodes
- Ran integration simulation test runs for ai nodes and layers
- Wrote unit tests for bumper sensor nodes
- Developed workflow for running linting and code checking tools on code base
- Developed plan of action for partitioning of LLM injection
- Created system architecture comparison diagram
- Reviewed team members pull requests and provide comments on improvements

Jonas Andaya

- Designed and implemented the beacon sensor AI

- Performed simulation tests for the beacon sensor AI
- Assisted in installing and setting up last year's project codebase, including resolving dependency and configuration issues
- Helped debug and fix issues in legacy code from the previous project iteration
- Contributed to the development of the report generator by assisting with feature implementation and testing output formatting
- Assisted in creating and configuring GitHub Actions workflows
- Reviewed pull requests to identify bugs, suggest improvements, and maintain code quality

Connor Yelle

- Installed and configured all required dependencies to support project setup and development
- Debugged and refactored the previous year's codebase to restore as much robot functionality as possible
- Deployed and configured Tailscale on the robot to enable secure remote access and communication
- Reviewed team members pull requests and provide comments where applicable
- Developed setup and launch files to support consistent initialization, configuration, and coordination of ROS 2 nodes across both AI-enabled and traditional robot system configurations, ensuring seamless switching between operational modes
- Configured Ollama for local deployment and execution of LLMs on the Raspberry Pi
- Integrated Navigation Unit AI with Docking Layer AI to ensure the robot can function with them both running

Umniah Mohammed

- Gathered project requirements through the requirements elicitation process and contributed to defining system functionality
- Created the use case diagram
- Configured a local development environment to run simulations on a personal laptop to enable more efficient testing outside the lab
- Implemented LLM-based Docking Layer (`docking_layer_AI.py`), integrating the `gemma2:2b-instruct-q4_0` model via Ollama with fallback and retry logic
- Implemented LLM-based Turning Layer (`turning_layer_AI.py`), integrating the `qwen2:0.5b` model with decision validation, caching, fallback logic, and launch toggle support

- Developed GitHub Actions CI pipeline (unit-tests.yml) with a custom ROS-free mocking framework (confest.py) to enable unit testing without ROS 2
- Created control layer unit tests (action_translator, report_generator) and automated them with a CI workflow (unit-tests-control.yml)
- Demonstrated CI pipeline functionality and integrated it into the team's development workflow
- Integrated AI LiDAR metrics into the dashboard pipeline, enabling comparison between AI-generated and classical outputs
- Implemented LiDAR source tagging (AI vs fallback) for improved observability and performance analysis
- Co-developed Gazebo launch automation script (runGazebo.sh) to streamline simulation setup and environment reset
- Unified the simulation and physical robot codebases into a single ROS 2 workspace, introducing a use_sim launch argument to switch between Gazebo and the physical Create3
- Created the LLM-augmented subsumption architecture diagram
- Created and managed 22 GitHub issues to support agile development and organize work across testing, LLM integration, metrics, and repository structure
- Led repository organization and cleanup, including PR/issue standardization and introduction of Conventional Commits
- Conducted robot and simulation testing, including recording demos for baseline, LLM-enabled, and component-level validation runs
- Designed and printed the capstone fair poster
- Took weekly meeting notes with the project supervisor
- Reviewed team members' pull requests and provided feedback to improve code quality and consistency

2.5.2 Report Contributions

By Jonas Andaya

The author(s) of each subsection are identified directly beneath the subsection headings throughout this report. Subsections listing multiple contributors indicate collaborative effort among those team members.

3.0 Requirements

3.1 Requirements Process

By Umniyah Mohammed

The final requirements were established through the iterative requirements engineering process; they meet the target qualities of completeness, consistency, and feasibility, as illustrated in Figure 2.

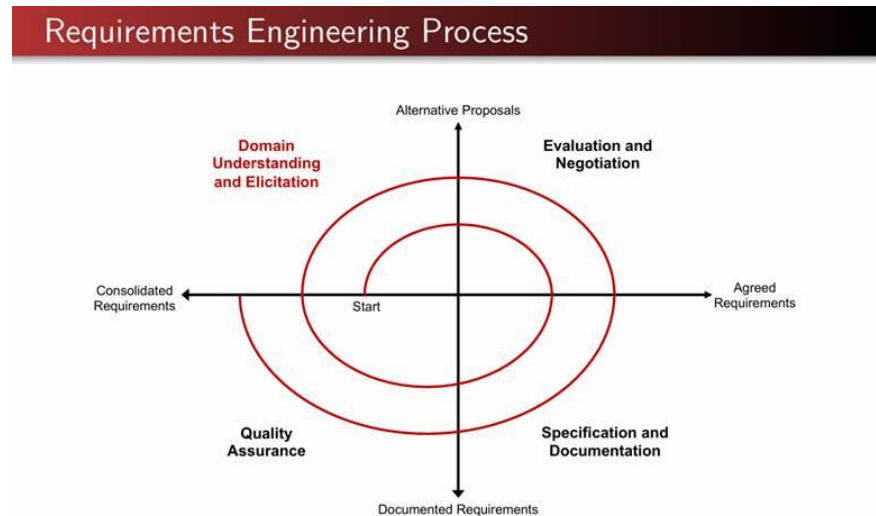


Figure 2: Requirements Engineering Process

Process Phases:

- **Domain Understanding and Elicitation**
The Domain Understanding activity defined the system's organizational and technical context. Stakeholders were identified, including the Robot (the core machine), and the LLM Agent (an external system component). Elicitation relied primarily on Artefact-Driven Techniques:
 - **Background Study:** Analysis of the previous year's codebase and project report defined the existing system capabilities and limitations, informing the functional baseline.
 - **Scenarios:** Detailed scenarios were created to explore new business events. Specifically, situations where standard ROS navigation fails (e.g., unexpected construction) and the LLM Agent must use contextual knowledge to generate a novel plan. These scenarios were critical for defining the new intelligent control requirements.
- **Evaluation and Negotiation**
In this phase of the requirements engineering process, informed decisions about issues raised during the elicitation process were made. The decisions are based on trade-offs and

negotiation is required to reach a consensus. The goal is to develop a set of low-risk, conflict-free, and agreed-upon requirements and assumptions. Conflicts/inconsistencies were handled using systematic techniques for a more effective outcome; the process of managing conflicts is shown in Figure 3. Candidate requirements were analyzed for quality and risk before being finalized.

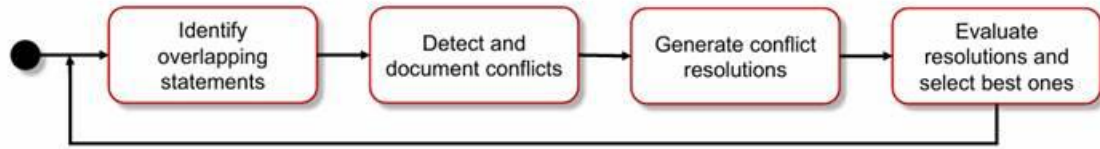


Figure 3: Managing Conflicts Process

- **Inconsistency Management:** A strong conflict was identified between the Non-Functional Requirements for high Performance (low latency/quick decision-making) and high Safety (collision avoidance). This was resolved by prioritizing Safety as a non-negotiable constraint, which may lead to slower, more cautious movement when LLM intervention is required.
- **Risk Analysis:** The primary product-related risk identified is LLM Agent Misinterpretation, where the LLM provides an unsafe, incorrect, or delayed plan. This risk is mitigated through a Defect Detection Prevention (DDP) strategy, which specifies requirements for a Monitoring Dashboard (for real-time validation) and a Safe Fallback State (to revert to proven standard ROS navigation if LLM input is invalid or fails).
- **Prioritization:** Requirements are prioritized using MoSCoW to guide the iterative Agile development sprints.

3.2 Function Requirements

By Umniyah Mohammed

The functional requirements define the core capabilities of the mail delivery robot system. These requirements describe the actions and behaviors the system must perform to achieve autonomous navigation, obstacle avoidance, and task execution. They were derived from the system's intended operation within the Carleton University tunnel environment and reflect both the baseline functionality from previous iterations and the additional capabilities introduced through LLM integration.

Table 3: Functional Requirements

ID	Requirement Statement
FR-1.0	The System shall autonomously navigate the Carleton University tunnel network to deliver mail between predefined locations.
FR-2.0	The LLM Agent shall utilize a locally hosted LLM to assist with sensor data processing throughout as many areas of the robot as possible.
FR-3.0	The Robot shall execute the navigation plan provided by the Bluetooth beacons or the ROS Navigation Stack, maintaining real-time control.
FR-4.0	The Robot shall avoid static and dynamic obstacles using data from its LiDAR and bump sensors.
FR-5.0	The System shall autonomously dock at its charging station upon low battery detection or task completion.
FR-6.0	The Dashboard shall provide a web-based interface for monitoring the robot status, performance metrics, and LLM decision-making rationale.
FR-6.1	The Dashboard shall display historical data and performance comparisons between test runs with and without the LLM enabled.
FR-7.0	The System shall be operable within a Gazebo simulation environment for development and testing purposes.

3.2 Non-Functional Requirements

By Umniyah Mohammed

The non-functional requirements define the constraints and quality attributes that the system must satisfy during operation. These include performance, safety, reliability, and compatibility considerations that ensure the system operates effectively within its hardware limitations and real-world environment. These requirements are particularly important given the use of LLMs, where response time, accuracy, and system fallback behavior directly impact the safety and reliability of the robot.

Table 4: Non-Functional Requirements

ID	Requirement Statement	Category	Measurement/Constraint
NFR-1.0	The Robot shall maintain a minimum safe distance of 1 meter from all detected	Safety	Measured by LiDAR sensor input during motion.

	dynamic obstacles during travel.		
NFR-2.0	The System shall successfully complete a standard mail delivery route with an acceptable success rate to be determined in future testing.	Reliability	Over the test runs on a predefined route, the robot must reach the correct destination without manual intervention in an acceptable number of runs.
NFR-3.0	The LLM Agent shall return a valid plan to the Robot within a system-defined acceptable response time threshold of receiving an intervention request.	Performance	System-wide latency under complex conditions.
NFR-4.0	The System shall revert to a Safe Fallback State if the LLM Agent returns an invalid or null response.	Reliability	Verified by state machine transitions.
NFR-5.0	The System shall run on the existing Raspberry Pi 4 hardware and be compatible with ROS 2.	Compatibility/ Architectural	Verified by successful compilation and operation on the specified hardware/software stack.
NFR-6.0	The selected LLM must be able to run on the existing Raspberry Pi 4.	Development Constraint	Verified by model file size and published parameter count before deployment to Raspberry Pi 4.
NFR-7.0	The Dashboard shall be accessible via a standard web browser (Chrome, Firefox, etc.) on a desktop operating system.	Compliance/ Usability	Verified by testing accessibility on specified browsers

Together, these functional and non-functional requirements define the expected behavior and operational constraints of the system, providing a foundation for the design and implementation described in subsequent sections.

3.3 Use Case Diagram

By Umniyah Mohammed

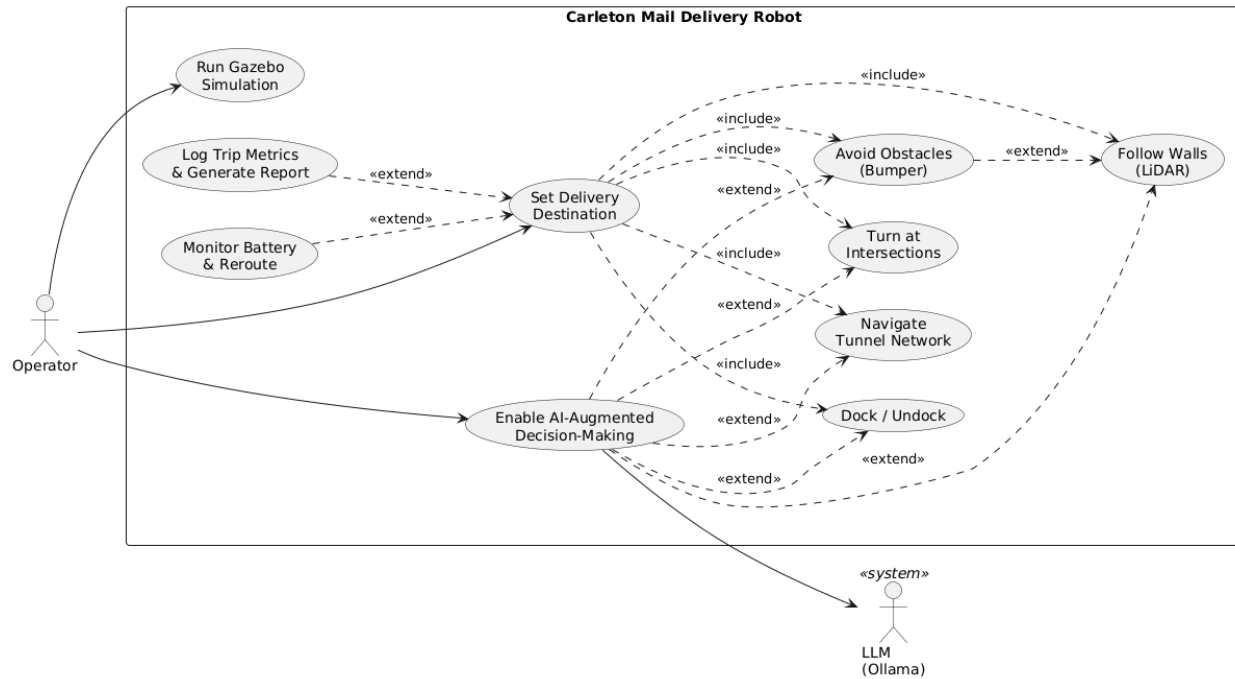


Figure 4: Use Case Diagram of Robot Mail-Delivery System

Figure 4 presents the Use Case Diagram for the Carleton Mail Delivery Robot system. It illustrates the interactions between external actors and the system as well as the relationships between the core functionalities required for autonomous mail delivery. The diagram provides a high-level view of how the system operates, emphasizing how different components collaborate to achieve reliable navigation, obstacle avoidance, and task execution within the tunnel network.

The system includes two primary actors: the Operator and the LLM (Ollama). The Operator represents a human user responsible for initiating and supervising system operations, including running simulations, setting delivery destinations, and monitoring system performance. The LLM is modeled as a system-level actor that provides AI-augmented decision-making capabilities, supporting various subsystems by generating navigation and control decisions based on contextual inputs. This distinction highlights the hybrid nature of the system, where traditional control logic is enhanced by the external AI component.

At the center of the diagram is the “Set Delivery Destination” use case, which serves as the main entry point for executing a delivery task. Once a destination is specified, the system activates a series of essential behaviors required for successful navigation. These include navigating the tunnel network, making decisions at intersections, avoiding obstacles using bumper sensors, following walls using LiDAR data, and managing docking and undocking procedures. These

functionalities are connected through «include» relationships, indicating that they are fundamental and consistently executed as part of the delivery process.

In addition to the core functionalities, the system incorporates several supporting behaviors that enhance reliability and observability. These include monitoring battery levels and rerouting when necessary, logging trip metrics and generating reports, and running simulations in Gazebo for testing and validation. These use cases are connected through «extend» relationships: they reflect that they are conditionally triggered based on system state or operational context. For example, battery monitoring may initiate rerouting only when power levels fall below a threshold, while logging is performed to support performance analysis rather than direct navigation.

A key aspect of the system design is the “Enable AI-Augmented Decision-Making” use case, which integrates the LLM into multiple subsystems. This use case extends several core functionalities, demonstrating how AI is selectively applied to enhance decision-making in navigation, obstacle avoidance, and other control processes. By structuring the system in this way, AI is treated as an augmentative layer rather than a replacement for existing logic. This allows fallback mechanisms to maintain system reliability when AI outputs are invalid or unavailable.

Overall, the diagram reflects a modular and extensible architecture in which complex behaviors are decomposed into well-defined use cases with clear relationships. This design supports incremental development, simplifies debugging and testing, and enables the integration of AI components without tightly coupling them to the core system. As a result, the system can maintain robust baseline functionality while exploring the benefits of LLM-driven decision-making in autonomous robotic navigation.

3.4 Use Case Tables

By Connor Yelle, Owen McKibbin

See Appendix B for the use case tables.

4.0 Background

4.1 Contributions from Last Year

By Jonas Andaya

In the past two years, there have been two significant additions to the robot. The most notable addition to the robot was the implementation of an architecture called the subsumption architecture. The second being the addition of the LiDAR sensor to the robot. The subsumption architecture is explained in the following section. For an explanation of LiDAR sensors, see section 3.2.3.

4.1.1 Subsumption architecture

By Jonas Andaya

A subsumption architecture is a type of architecture in robotics which is made up of different layers. Lower layers control more primitive behaviors while higher layers control more complex behaviors. Higher layers can subsume lower layers and suppress their outputs, while the lower layers can still function [5]. The subsumption layer was developed by Rodney A. Brooks.

Last year, the team shifted the system to a subsumption architecture and provided a simulator to create a more scalable system for future iterations. This approach organized the robot's behavior into multiple independent layers, each responsible for a different task, with a defined priority system to determine which actions should be executed.

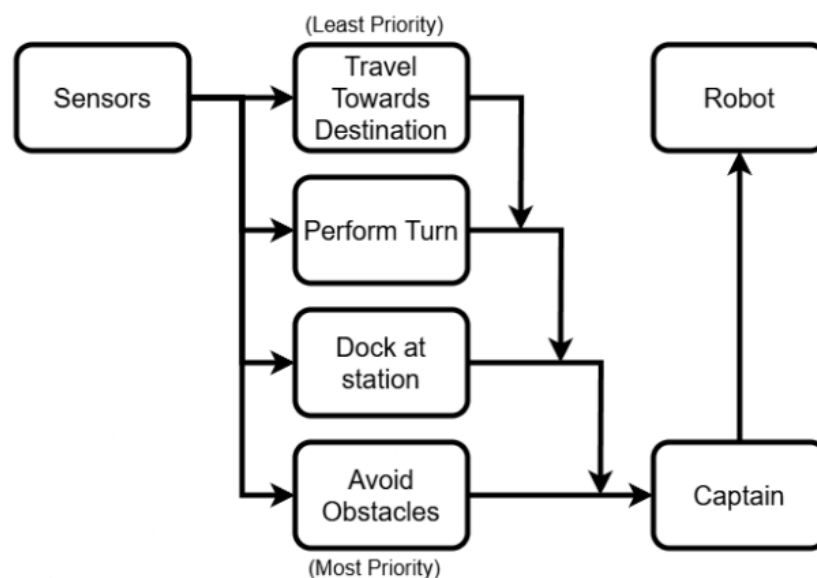


Figure 5: Diagram of the subsumption layers from last year's project

The figure seen above shows a diagram of the subsumption architecture made by the group of last year's mail-delivery robot capstone project [6].

The system was composed of four primary behavioral layers: obstacle avoidance, docking, turning, and travel. The obstacle avoidance layer had the highest priority and handled collision detection and resolution using bumper sensor data. The docking layer managed both docking and undocking procedures based on navigation commands and dock visibility. The turning layer was responsible for executing navigation decisions at intersections, including left turns, right turns, and straight traversal. Finally, the travel layer served as the default behavior, enabling the robot to follow walls and move toward its destination. These layers worked together under a priority-based system, allowing higher-priority behaviors to override lower ones when necessary.

4.2 Tools and Software

By Owen McKibbin

To facilitate the engineering process of implementing an LLM-driven mail-delivery robot, many specific tools and software were integrated into the project. This section outlines and describes such tools which have been useful for the development of the project.

4.2.1 Robot Operating System

By Jonas Andaya

The Robot Operating System (ROS) is a middleware framework for developing robotics software. Despite its name, it is not a traditional operating system, but rather a set of tools, libraries, and communication infrastructure that enables modular robot development. ROS allows different components of a robotic system to communicate through a publish-subscribe messaging architecture.

This project used ROS 2, the second generation of ROS, which improved upon ROS 1 by addressing key limitations such as unreliable data delivery, a centralized single point of failure, and the lack of built-in security features [1]. ROS 2 introduced a distributed architecture, improved reliability, and secure communication between nodes.

The system ran the Humble Hawksbill distribution of ROS 2, which is supported on Ubuntu 22.04 (Jammy). This compatibility ensured stable deployment on the Raspberry Pi 4 used throughout the project.

4.2.2 iRobot Create

By Jonas Andaya

The iRobot Create is a robotics development platform that integrates with ROS 2, allowing communication through ROS topics for sending commands and receiving sensor data. Developed by iRobot, it is primarily used for education and research and is also compatible with simulation environments such as Gazebo.

This project uses the iRobot Create 3, the latest generation of the platform, which has been used in previous iterations of the mail delivery robot. The notable physical addition to this project's iRobot Create 3 over the past years has been a LiDAR sensor that was attached to the roof of the robot. The iRobot Create 3 was still used this year with no modifications to the robot.

4.2.3 LiDAR Sensor

By Jonas Andaya

Light Detection and Ranging (LiDAR) is a way to determine distances by shooting out laser beams and measuring how long it takes to return. A LiDAR sensor is comprised of a laser to shoot out the pulsed beams and a receiver to detect the pulsed beams [4].

LiDAR is a key sensor for mobile robot navigation because it provides real-time spatial awareness, allowing the system to detect obstacles and build a representation of its surroundings for safe path planning. The LiDAR sensor is primarily used for obstacle detection and navigation support, providing distance measurements that inform the robot's movement decisions.

In the 2023 iteration of the mail-delivery robot, they purchased the Slamtec RPLIDAR sensor to improve the navigation capabilities of the robot. The Slamtec RPLIDAR sensor can scan 360 degrees within 12 meters and is able to change its scan rate from 2-10 Hz. This year's iteration of the mail-delivery robot continued to use the LiDAR sensor. With the introduction of the LLM agent, the team experimented with how well it interacts with the LiDAR sensor.

4.2.4 Gazebo

By Jonas Andaya and Connor Yelle

Gazebo is an open-source robotics simulation platform used to model and test robots in realistic virtual environments. It provides built-in support for physics simulation, sensor modelling, and 3D rendering, allowing robots to be tested in controlled conditions before deployment. Since both ROS and Gazebo are developed by Open Robotics, they are designed to integrate seamlessly, with existing documentation and tools supporting ROS 2 compatibility [3].

In last year's iteration of the project, Gazebo was first introduced to simulate both the mail delivery robot and the Carleton tunnel environment. This allowed for early testing of navigation and control logic without requiring physical deployment.

In this year's implementation, Gazebo continued to be used as a primary simulation tool for development and testing of the robot's behavior with LLMs. ROS2 nodes running within the simulation environment communicate with the same interfaces used on the physical robot. This means that the team can test LLM-driven nodes in a consistent and repeatable manner, allowing for quick iteration. Changes to prompts, models, and decision-making code can be tested efficiently.

The simulation is executed on a more powerful workstation than the Raspberry Pi used on the physical robot, enabling faster iteration and more computationally intensive testing.

4.2.5 Ollama

By Jonas Andaya and Connor Yelle

Ollama is a software platform used to run large language models (LLMs) locally. It simplifies LLM deployment by providing an accessible interface and well-documented tooling for model management and inference [2]. Unlike previous iterations of the project, which did not incorporate LLMs, this year's system uses Ollama to support local model execution and integration with the ROS 2 architecture.

Ollama was used to evaluate and interface with LLMs as part of the robot's decision-making pipeline, with further analysis of model performance presented in section 6.3. It was deployed on the Raspberry Pi 4 to enable direct communication between the ROS nodes and the locally running LLM, allowing real-time prompt handling within the system.

The primary motivation for selecting Ollama was its ability to run models completely locally without internet connection. This approach allows for the LLM based ROS nodes to operate with reduced latency and improved reliability, as they are not dependent on external APIs. This also allows for the LLM queries to work even when the robot has no Wi-Fi connection in certain parts of the Carleton Tunnels.

4.2.6 UV

By Owen McKibbin

UV is a Python tool for setting up virtual environments and Python projects. UV allows for easy setup of Python dependencies to be used through the terminal. In this case, linting and code checking dependencies such as black, isort, and ruff were used for removing unnecessary space, reformatting lines, sorting imports, and checking for syntax errors and unused imports. UV is

also helpful for specifying Python versions and paths. This tool allows the team to maintain a clean working environment during the development process.

4.2.7 BluePy

By Jonas Andaya

BluePy is a Python library that enables communication with Bluetooth devices. BluePy allows developers to easily discover, connect to, and exchange data with nearby wireless devices.

BluePy is commonly used in embedded systems and robotics applications where efficient, short-range communication with sensors or beacons is required. In this project, BluePy was used as part of the system to interact with Bluetooth beacon devices.

5.0 Group Skills

By Connor Yelle, Jonas Andaya, Owen Mckibbon, Tommy Phang, Umniyah Mohammed

This section provides an introduction and overview of team members' skills and experience in relation to this project.

Connor Yelle:

Connor is a fourth-year software engineering student. As such, he has a lot of fundamental knowledge of system design and programming. Connor knows Python, C, and Java, which are all languages that are useful when working on the task at hand. Connor also has prior knowledge of embedded systems programming in C, and web development in CSS/HTML/JavaScript, which are both critical skills that are needed to succeed in the project. Throughout his education at Carleton, Connor learned critical software engineering development lifecycle skills like requirements elicitation and analysis, software architecture, and testing which will be useful for making critical decisions throughout the year.

Jonas Andaya:

Jonas is a fourth-year software engineering student. Jonas has experience programming in languages such as Python, which is used in the code of the mail-delivery robot. Jonas has knowledge of the software development life cycle and has experience working with common software design methodologies. Particularly, Jonas has experience creating sequence diagrams, using case diagrams, and state machine diagrams. All of these will aid in the explanations of how the mail-delivery robot works. Jonas also has experience working on software projects with others. Additionally, Jonas has experience debugging programs and developing structured, maintainable code.

Tommy Phang:

Tommy is a fourth-year software engineering student, with foundational skills in software engineering. Tommy currently has gained proficiency in Python through ECOR 1041, ECOR 1042 and SYSC 2100; Python is utilized in the capstone project. Tommy is familiar with software development lifecycles methodologies such as Agile and DevOps, learned in SYSC 4106. Materials learned from SYSC 4106 will be useful in managing the software development process of the capstone project.

Umniyah Mohammed:

Umniyah is a fifth-year software engineering student driven by a passion for robust software architecture and cybersecurity. Umniyah has experience in Python, C, C++, Java, and Racket, as well as modern web development frameworks including HTML, CSS, JavaScript, TypeScript, React, and related tools such as npm and Vite. Umniyah has applied her skills in a variety of projects, ranging from AI-driven applications and chatbots to full-stack development and team-led software projects, demonstrating abilities in software development, software requirements, software architecture, SDLC methodologies, design patterns, and software validation and verification. Additionally, she has professional experience as a software developer and researcher, including back-end algorithm validation and full-stack development internships, where she collaborated with cross-functional teams to deliver optimized, industry-quality code. Her combination of academic knowledge, applied project experience, and industry exposure provides critical expertise to the capstone project, particularly in software design, system implementation, and secure, maintainable coding practices.

Owen McKibbon:

Owen is a fourth-year software engineering student with a deep curiosity for machine learning, simulation, and robotics. Owen has extensive experience in Python through several projects ranging from data visualization and parsing to machine learning simulations. Owen has also worked with languages such as C, C++, and Java, giving him a solid foundation for systems and embedded programming. Through both coursework and independent projects, he has developed strong software engineering practices, including version control with Git, task delegation, and collaborative communication in team settings. Owen is eager to bring his technical skills, problem-solving mindset, and commitment to planning and research to the mail delivery robot project alongside his teammates.

6.0 Methods

By Owen McKibbin

To accomplish the objective of making the robot fully autonomous through LLM decision making, the group selectively tested and chose small LLMs that can fit on the Raspberry Pi 4 and competently answer simple navigation questions. The group had decided to inject the AI model into the architecture of the robot as the primary decision maker when given data such as LiDAR point clouds, current location, obstacles, etc., to choose the best course of action to reach its destination.

The team decided to follow an Agile methodology for the project through the implementation of testing and verification of multiple iterations of work throughout the year. This process allowed the team to spot errors and resolve conflicts ahead of time before they become detrimental to the project. Agile methodologies have been covered extensively in SYSC4106 (Software Economy and Project Management) which has helped transition quickly into this approach for this project.

To establish the requirements of the project and build the project schedule, the team has applied skills learned through SYSC3120 (Software Requirements Engineering) to formulate a list of requirements specific to the project. In addition, the experience of the team studying Software Architecture and design in SYSC4120 prepared us to make key design decisions on the injection of the AI model into the system.

Lastly, the team decided to build a dashboard and GitHub repository to publish metrics relating to the robot's performance and overall project progress. This allowed the team to have a clear picture of the progress of the project and overviews of the current state of testing and model performance.

7.0 Analysis

By Tommy Phang

In this section, the team discusses the analysis of LLMs and how the team integrated LLMs within last year's code. The team performed an analysis on the feasibility of LLMs on Raspberry Pi 4, the analysis of last year's codebase and how to integrate LLMs into it, comparison on a larger LLM vs a smaller LLM, and the effectiveness of prompt styling.

7.1 LLM Feasibility Evaluation

By Tommy Phang

This section goes over the selection, evaluation, and analysis of LLM models for robotic control on the Raspberry Pi 4. Before the team began modifying or replacing components of last year's codebase, the team first wanted to observe the performance of small LLMs (less than 1 B parameters) to get a sense of what the team is working with. The team wanted to determine whether small LLMs could process data and generate meaningful control commands, rather than producing inconsistent or unusable outputs. This section outlines the selection criteria for the deployment of such LLMs on Raspberry Pi 4, describes the team's evaluation methodology, analysis of small LLMs, and examines the influence of model capacity on navigation decision accuracy.

7.1.1 LLM selection criteria for Raspberry Pi 4

By Tommy Phang

The goal of developing this criterion was to help guide the team's decision on choosing a suitable LLM model to be compatible with Raspberry Pi 4 and help meet the capstone objective. An LLM model must meet all the requirements mentioned to be considered suitable for deployment. The following selection criteria were derived from the hardware limitations of Raspberry Pi 4 and real-time application requirements. By following these criteria, the team can quickly select a large language model that is suitable for the project.

Model Requirements:

- **Model Size (< 3B Parameters):** The model must have fewer than 3 billion parameters, following Raspberry Pi Organization recommendation.
- **Tool Support:** The model must support external tools to enable extended functionality.
- **Quantization Availability:** A Quantized option should be available to reduce RAM usage and improve performance on Raspberry Pi 4.

- **Concise Reasoning Capability:** The model should be able to toggle between verbose and concise reasoning modes; concise outputs are essential for real-time navigation commands.

7.1.2 Evaluation methodology

By Tommy Phang

The team evaluated the following models: smoll2 360M, Gemma3-tools:4b, Qwen3:0.6b, and Qwen2.5:0.5b and determined if they were suitable for the project by following the selection criteria, see *Table 4*. This section evaluates how these models performed with spatial instructions. The team assessed each model’s ability to reason and follow spatial instructions based on this prompt:

- You are a robot navigating a 3x3 grid. The bottom-left corner is position (0,0) and the top-right corner is (2,2). Your goal is to move from (0,0) to (2,2). At each step, describe your movement using simple commands: "move up", "move down", "move left", or "move right". Do not skip steps or jump positions. Output each move in order until you reach (2,2).

An LLM passes the test if it outputs a valid sequence of steps that move from (0,0) to (2,2), using only simple movement commands without skipping any positions. Each model was run on the Raspberry Pi 4.

The team also evaluated Qwen3:0.6b and an alternative version, Qwen2.5:0.5b, using a different prompt. The team tested Qwen3:0.6b and Qwen2.5:0.5b through the following prompt: “Suggest a movement command for the robot ‘forward’, ‘backward’, ‘left’, ‘right’, or ‘stop’. You should send one-word answers.” The team wanted to observe how each model would respond when issuing commands to potentially move the robot in real time. A model would pass the test if it followed the instructions of returning one of the following: ‘forward’, ‘backward’, ‘left’, ‘right’, or ‘stop’. If the models returned anything else, the model has failed the test. The goal of this test was to assess both models’ ability to issue simple navigation commands. This section describes the evaluation methodology; the results of these experiments are presented in the next section.

7.1.3 Results and observations

By Tommy Phang

Following the prompt evaluation outlined in Section 7.1.2, the team observed that Smoll2 360M took 5.7 seconds to start generating a prompt; it began generating python code, failing to adhere to the instructions. Gemma3-tools:4b was unable to run on Raspberry Pi 4 due to it not having the required memory of 5.8 GB of usable RAM. Qwen3:0.6b took 6.5 seconds to start generating

a prompt; it successfully followed instructions and reached (2,2). The team decided to go with some variation of Qwen for the LLM implementation since it passed the prompt test and meets the selection criteria.

The team has observed that both Qwen3:0.6b and Qwen2.5:0.5b could generate navigation commands to be sent to the robot. Based on the recorded data (see Appendix A), Qwen3:0.6b exhibited an average response time of 29,200ms and occasionally failed to generate a response entirely. This is due to the LLM performing chain of thought, where the LLM would output its step-by-step reasoning. In contrast, Qwen2.5:0.5b achieved a significantly faster average response time of 2,416 ms and consistently produced responses without failure. Due to both LLMs large response time the robot moved in a stop and go manner, resulting in choppy motion.

Even though Qwen3:0.6b and Qwen 2.5:0.5b were able to move the robot, the team found that both models would occasionally fail to follow instructions to respond with one-word outputs. The core issues the team encountered with both models were inconsistent response times and an inability to adhere to strict response formats. The robot requires continuous, low latency inputs to avoid choppy motion, but both models produced commands in unpredictable bursts, with response times ranging from several seconds to several milliseconds (see Appendix A).

Based on the tests conducted with Qwen3:0.6b and Qwen2.5:0.5b, the team identified that stable, predictable response behavior and strict adherence to output formatting are important requirements for an LLM used in the robot.

Table 5: Table of LLMs Tested

LLM Model	Model < 3B Parameters	Tool Support	Quantization Availability	Concise reasoning Capability	Model Download Size
smoll2 360M	✓	✓	✓	✗	726 MB
Gemma3-tools:4b	✗	✓	✓	✗	3.3 GB
Qwen3: 0.6b	✓	✓	✓	✗	523 MB

Qwen2.5: 0.5b	✓	✓	✓	✓	398 MB
------------------	---	---	---	---	--------

7.1.4 Conclusion

By Tommy Phang

This section evaluated the feasibility of deploying small LLMs for robotic control on Raspberry Pi 4 by defining the selection criterion, outlining evaluation methodologies, and assessing model performance on spatial reasoning and command generation tasks. The results indicate that, while small LLMs can generate navigation commands, their effectiveness is limited by inconsistent response times and unreliable adherence to strict output formats. These limitations highlight the importance of both response accuracy and latency in real-time robotic applications. In the next section, the team discusses where the team would implement LLMs based on last year's system architecture.

7.2 Analysis of Current System Architecture

By Jonas Andaya, Tommy Phang, Owen McKibbin, Connor Yelle and Umniyah Mohammed

Before beginning the project, the team needed a clear plan for integrating AI throughout the system. Thus, a thorough analysis of the current system architecture was conducted to determine if the integration could be achieved.

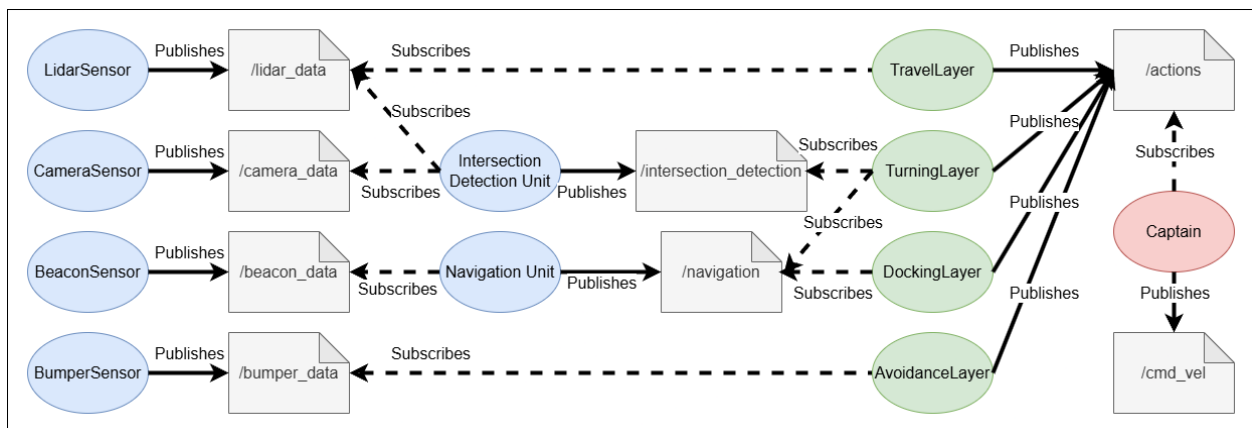


Figure 6: The system architecture from the previous year's version of the system

This diagram illustrates how the system's various sensors publish data to processing nodes, which then communicate with the control layers. The captain node reads data from the layers and sends movement commands to the robot.

It was apparent to the team that to successfully fulfil the project’s objectives, the LLM agent had to be injected into as many aspects of the system as possible. The team proposed several strategies for achieving this, and two approaches were derived: vertical partitioning and horizontal partitioning.

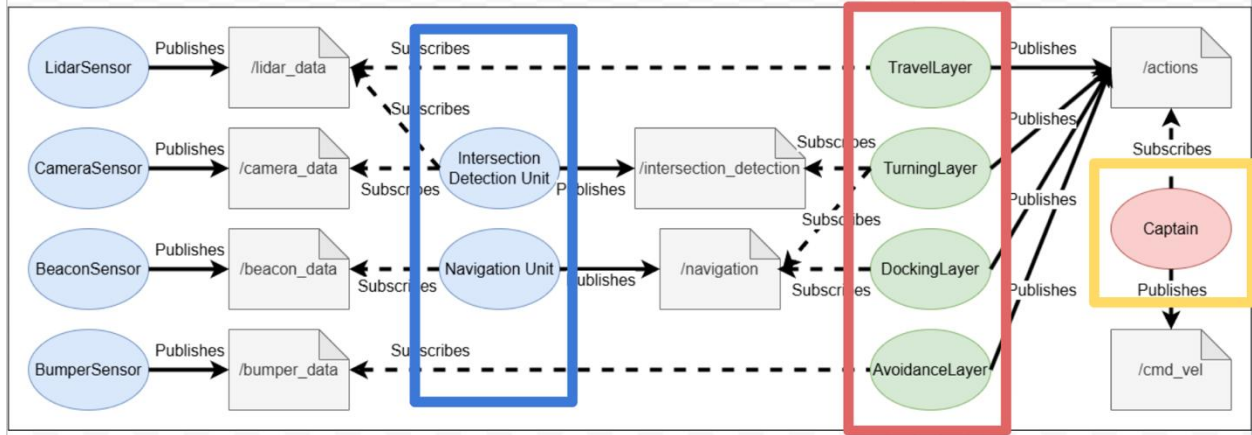


Figure 7: Vertical Partitioning of Architecture

In the illustrated vertical partitioning of the architecture (Figure 7), there are three main components in the system where the LLM agent will be incrementally integrated. The blue box highlights the processing unit nodes, which receive and process sensor data for decision-making. The red box contains the behavior layers responsible for navigation and control. The yellow box includes the captain node, which coordinates data into specific actions for the robot. The LLM agent will be injected into each node sequentially, starting with minimal functionality and gradually expanding its role in each node as confidence in its performance increases.

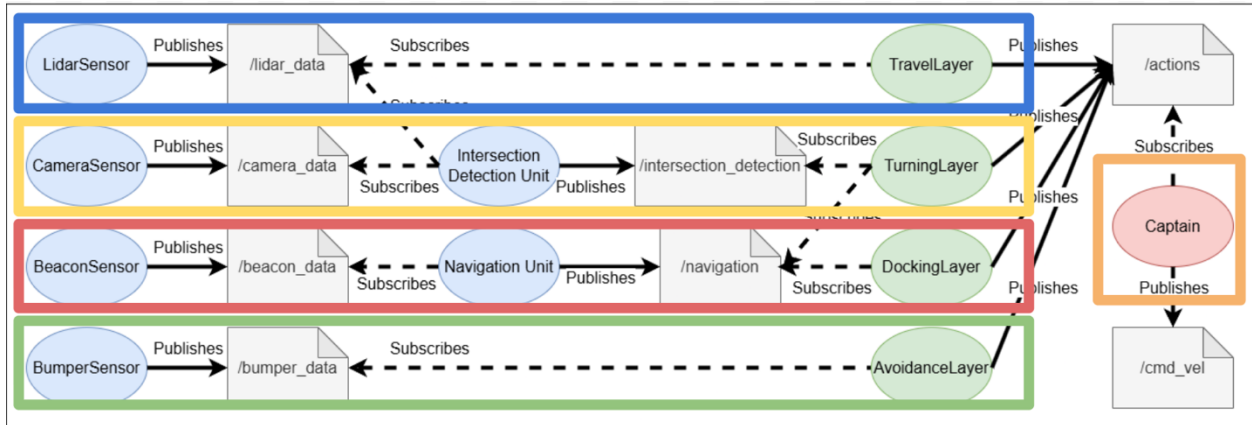


Figure 8: Horizontal Partitioning of Architecture

Figure 8 illustrates the proposed horizontal partitioning of the architecture. In this approach, there are five components where the LLM agent will be incrementally integrated. Each colored box represents a horizontal slice of the system that shows a complete data processing path from sensor input to robot movement. This approach enables the LLM agent to be introduced into

each functionality of the system one at a time, while minimizing the risk of disrupting other parts of the system. It also allows for a gradual scaling of the agent's role across the architecture to maximize its integration throughout the system.

To help facilitate the decision on which partitioning method to adopt, the team created a table comparing the pros and cons of vertical and horizontal partitioning to analyze the trade-offs between the two designs.

Table 6: Table of Pros and Cons for Each Partitioning Method

	Pros	Cons
Horizontal Partitioning	▪ Complete data pipeline for each LLM injection ▪ If an LLM injection in a given slice is completed, it is considered finished and should not interfere with the injection in other pipelines.	▪ Requires more agents ▪ Debugging can be challenging due to the scope of the slice; multiple files may need to be examined to locate bugs
Vertical Partitioning	▪ Easier to debug functionality in a single layer rather than an entire data processing stream ▪ Testing is simpler when adding tests to individual layers versus entire streams, as layers often have very similar code and dependencies ▪ Easier to coordinate agents within a single layer versus the entire horizontal slice ▪ Separation of concerns, for navigation, data processing, and action.	▪ Adding the LLM to every aspect of the system may be delayed until the end of the project ▪ Less granular milestones, which could complicate agile methodology for achieving project objectives

After analyzing the table, the team decided to adopt vertical partitioning for LLM integration. This decision was made primarily to simplify debugging and testing, which are critical to the project's success. If the robot with LLM integration cannot be reliably tested, there is no way to ensure that the desired functionality is achieved. Another factor influencing this decision was the limited resources of the Raspberry Pi; the LLM is very memory-intensive, and only a small

number of agents can run simultaneously. Vertical partitioning reduces the number of active LLM agents at any given time, which makes it more feasible for the hardware.

After injecting the LLM into the navigation unit node, the team decided the next node to implement was the Intersection Detection Unit node. However, upon further review, the team noticed that the decision making inside of this node comes down to a simple true or false statement. The team then decided to not implement the AI inside of this node, as it is too simple for an AI to control it. This was also the case for the Bumper sensor node, so the team again decided not to implement the AI here. Other than this, all AI nodes were implemented following the vertical partitioning, starting with the sensor nodes, and then the layers. The team wanted to compare the performance of Qwen 2.5 0.5B to a stronger model which led us to explore the influence of large language model capacity on navigation decision accuracy. The team wanted to know if the team had access to a stronger model would it improve accuracy.

7.3 Analysis of the Influence of Large Language Model Capacity on Navigation Decision Accuracy

By Tommy Phang

Due to the poor performance of smaller LLMs, this section aims to evaluate if increasing the capacity of a large language model (LLM) improves the accuracy of navigation decisions, assuming no hardware or latency constraints. This experiment isolates model capacity from system latency constraints to measure pure navigation accuracy. The objective of this experiment is to determine whether a stronger/larger LLM produces more correct navigation decisions than a smaller/weaker model when both are used within the same decision-making pipeline. Which then shows that performance of LLMs on the robot will increase if given better hardware.

Question: Does a stronger/larger LLM produce more correct navigation decisions than a smaller/weaker model?

Hypothesis: A stronger/larger LLM will produce more correct navigation decisions than a smaller/weaker model due to improved reasoning and instruction following capabilities. This suggests that, in the absence of hardware constraints, deploying larger models can lead to improved navigation performance.

- Null Hypothesis (H_0): $p_1 = p_2$
 - The accuracy of the smaller model (p_1) is equal to the accuracy of the larger model (p_2).
- Alternative Hypothesis (H_a): $p_1 < p_2$
 - The accuracy of the larger model (p_2) is larger than the accuracy of the smaller model (p_1)

The experiment is run by selecting three LLMs of different sizes:

As stated above, this experiment is run with the assumption that there are no hardware constraints or latency constraints.

- Qwen 2.5 0.5B
- Qwen 2.5 3B
- Gemma3 9B

Experiment methodology can be found in [Appendix F](#).

7.3.1 Experiment results

By Tommy Phang

For the full table of results please see [Appendix D](#).

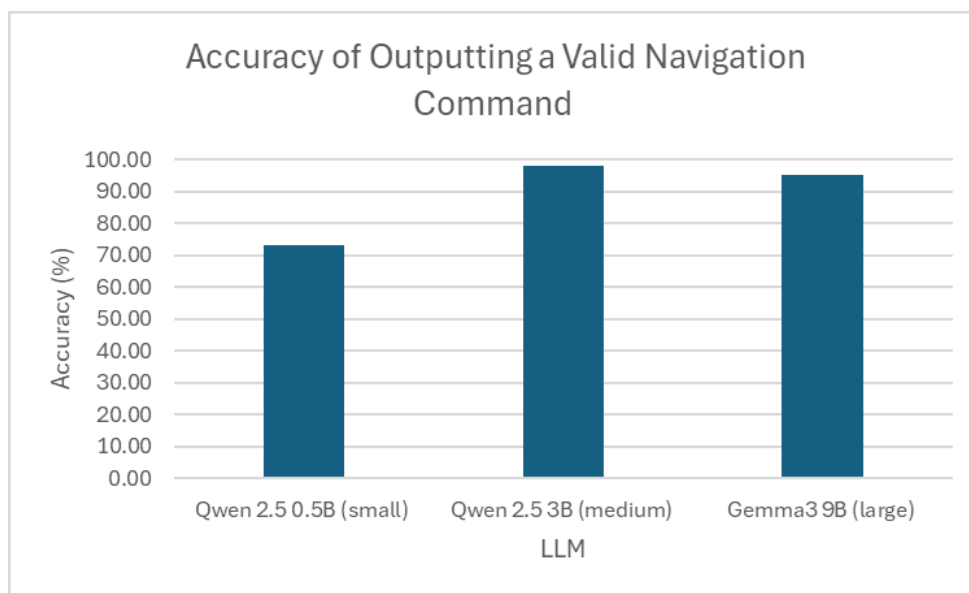


Figure 9: Accuracy of Outputting a Valid Navigation Command

7.3.2 Results and analysis

By Tommy Phang

After running the experiment, Qwen 2.5 0.5B has an accuracy of 73.00%, Qwen 2.5 3B has an accuracy of 98.00%, and Gemma3 9B has an accuracy of 95.00%. Surprisingly even though Qwen 2.5 3B is a smaller model than Gemma3 9B, it outperformed it. When comparing the navigation accuracy between the small and medium model, the comparison yields a z-score of -5.021 and a p-value of 0.0000. Based on the calculated p-value, we reject the null hypothesis.

This shows that there is a statistically significant improvement in navigation accuracy when moving from a small model to a medium model.

When we look at the comparison between the small and large models, we get a z-score of -4.243 with a p-value of 0.0000 . Therefore, we reject the null hypothesis. This shows that there is a statistically significant improvement in navigation accuracy when moving from a small model to a large model.

When we look at the comparison between the medium and large model, we get a z-score of 1.154 with a p-value of 0.2484 . Therefore, we fail to reject the null hypothesis. This shows that there is no statistically significant difference in navigation accuracy when moving from a medium model to a large model.

This shows that a larger capacity does improve performance in navigation but up to a point. Based on the above results, we can see further increases beyond 3B parameters did not yield additional benefits for this task. This shows that the team does not need a large model to accurately provide accurate navigation commands, and that a medium sized model can solve them when provided with the correct context and prompt.

7.3.3 Conclusion

By Tommy Phang

In conclusion, this experiment demonstrates that increasing LLM capacity improves navigation decision accuracy, as the medium and large models achieved greater than 95% accuracy while the small model achieved only 73% accuracy. The results also indicate diminishing returns beyond the 3B model, since the 9B model did not improve navigation accuracy and would require more power and resources on the robot to run it.

The statistical data confirms the hypothesis that larger models improve accuracy, but only to a point. The jump from 0.5B to 3B parameters provides a massive boost in accuracy. However, the move from 3B to 9B shows diminishing returns.

Currently, the robot uses Qwen 2.5 0.5B for navigation and based on statistical evidence, a 3B parameter model should be considered the target for this capstone project. This is only possible if hardware constraints can be addressed. A model with 3B parameters will allow maximum accuracy without the excessive computational overhead, power draw, and latency that the 9B model would require on the robot's hardware. Unfortunately, given with the current hardware the team is unable to run models greater than 2B parameters. This experiment shows that if the team has access to better hardware and/or when stronger models become more efficient in the future, the performance of LLMs on the robot will increase. The team thus relies on trying other smaller models and/or modifying prompts to improve model performance. This leads to the next section analyzing the effects of prompt variations on model performance.

7.4 Effects of Prompt Variations on Model Performance

By Tommy Phang

Since the team relies on experimenting with smaller models and adjusting prompts to improve performance, this section focuses on how prompt design impacts results. By testing different prompt structures and wording, the team evaluated how these changes influence the model's outputs. This analysis helps us identify which prompts lead to more accurate and reliable responses, and whether certain styles consistently improve performance. In the upcoming sections the team analyses the effects of prompt variation, the goal is to determine the most effective prompt for the AI units. The team compared five different prompts and analyzed how each prompt affected the model's performance. Each section describes the methodology used to evaluate the effectiveness of each prompt and their results. From this analysis, the team implemented the best prompt style for the LLM driven nodes and layers aiming to improve overall performance of the robot.

7.4.1 Effects of Prompt Variation on Navigation AI

By Tommy Phang

The team wanted to explore how different prompt formulations affected the accuracy of a small language model specifically qwen2.5: 0.5b in outputting correct navigation commands. This experiment aims to identify how 5 distinct prompt styles (e.g., instructional, concise, role-based, few-shot, and constraint-based) affect the accuracy of qwen2.5: 0.5b. Ultimately identifying which prompt style yields the highest performance.

The experiment was conducted with 50 trials each trial a random starting location and destination is selected. The LLM must output the correct navigation command based on the given starting location to reach the given destination. For each trial the same randomly selected starting location and destination is injected into all five prompt styles to ensure consistency. More details on experiment setup and procedure can be found in [Appendix F](#). Five prompt types, each with a different prompt structure are tested below:

Prompt 1: Instructional Prompt has detailed and structured instructions

```
f"""
    You are a robot navigation system.
    Current Location: {current_node}
    Destination: {target_dest}
    {context}
    INSTRUCTION: Based on the Navigation Rules above, what is the single
    command to go from {current_node} to {target_dest}?
    Reply ONLY with the command (e.g., NAV_LEFT). Do not explain.
    """
```

Prompt 2: Concise Prompt has minimal and technical wording

```
f"""Context: {context}
Task: Move from {current_node} to {target_dest}.
Response: [NAV_ command only]"""
```

Prompt 3: Role-based Prompt frames the model as a navigation system

```
f"""System: Navigation AI.
Rules: {context}
Current State: {current_node} -> {target_dest}
Next Move: """
```

Prompt 4: Few-shot Prompt includes an example demonstration

```
f"""Rules: {context}
Example: Start: Loeb1, Dest: UC -> NAV_U-TURN
Task: Start: {current_node}, Dest: {target_dest}
Command: """
```

Prompt 5: Constraint-focused Prompt emphasizes strict output formatting rules

```
f"""{context}
Find command for {current_node} to {target_dest}.
Rule: Do not provide any conversational text. Use ONLY the NAV_ labels
provided in the rules. """
```

Evaluation Metrics

Success count per prompt type out of 50

Accuracy (%) calculated as: $\text{Accuracy} = (\text{successes} / 50) * 100\%$

Results

Sample Correct Response from qwen2.5: 0.5b

[Prompt Style 1]

Prompt given to qwen2.5: 0.5b:

You are a robot navigation system.

Current Location: Robertson1

Destination: Canal Navigation Rules:

- To Loeb: NAV_PASS
- To Southam: NAV_PASS
- To TunnelJunction: NAV_PASS
- To Library: NAV_PASS
- To Azrieli/Tory: NAV_PASS
- To Steacie: NAV_PASS
- To Richcraft: NAV_PASS
- To UC: NAV_PASS
- To Canal: NAV_PASS
- To Nicol: NAV_PASS

- To Mackenzie/Minto: NAV_PASS
- To Frontenac: NAV_PASS
- To ResComms: NAV_PASS
- To CTTC: NAV_PASS
- To Robertson: NAV_DOCK

INSTRUCTION:

Based on the Navigation Rules above, what is the single command to go from Robertson1 to Canal? Reply ONLY with the command (e.g., NAV_LEFT). Do not explain.

qwen2.5: 0.5b correctly responded with: "NAV_PASS"

Sample Incorrect Response:

Prompt given to qwen2.5: 0.5b:

Context:

Navigation Rules:

- To Loeb: NAV_PASS
- To Southam: NAV_PASS
- To TunnelJunction: NAV_PASS
- To Library: NAV_PASS
- To Azrieli/Tory: NAV_PASS
- To Steacie: NAV_PASS
- To Richcraft: NAV_PASS
- To UC: NAV_PASS
- To Canal: NAV_PASS
- To Nicol: NAV_PASS
- To Mackenzie/Minto: NAV_PASS
- To Frontenac: NAV_PASS
- To ResComms: NAV_PASS
- To CTTC: NAV_PASS
- To Robertson: NAV_DOCK Task: Move from Robertson1 to Canal.

Output JSON format ONLY: {"direction": "NAV_command"}

qwen2.5: 0.5b incorrectly responded with: "NAV_TunnelJunction"(Extracted: NAV_TUNNELJUNCTION, Expected: NAV_PASS)

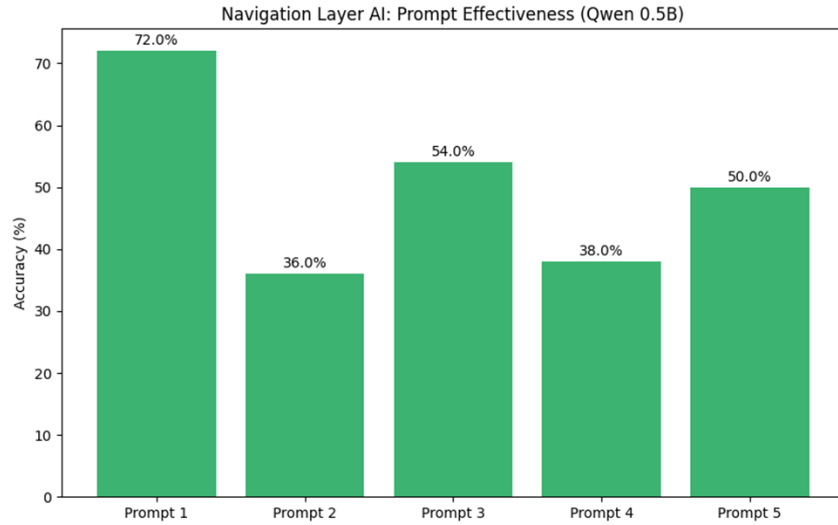


Figure 10: Bar chart of Navigation AI prompt Accuracy

From the results above, we can conclude that formatting the prompt in an instructional style (prompt 1) with detailed and structured instructions can yield better navigation commands from Qwen2.5: 0.5B.

7.4.2 Effects of Prompt Variation on Avoidance Layer AI

By Tommy Phang

The team wanted to explore how different prompt formulations affected the accuracy of a small language model specifically qwen2.5: 0.5b in outputting correct commands in an obstacle avoidance task. This experiment aims to identify how 5 distinct prompt styles (instructional, concise, reasoning, role-based, and constraint-based) affect the accuracy of qwen2.5: 0.5b. Ultimately identifying which prompt style yields the highest performance.

The experiment was conducted with 50 trials, for each trial a random distance is generated for the front, left, and right region of the LiDAR sensor. A LiDAR summary string is then generated that includes the random distances generated, as well as one region containing a small distance representing an obstacle detected in that region and includes the most open region. For each trial the same LiDAR summary is injected into all five prompt styles to ensure consistency. The LLM must output the correct navigation command to avoid collision based on the given LiDAR summary. More details on experiment setup and procedure can be found in [Appendix F](#). Five prompt types, each with a different prompt structure are tested below:

Prompt 1: No style

```
p1 = f"""
The robot has collided with an obstacle.
```

```
{lidar_summary}

Rules:
- Do NOT choose the closest obstacle direction.
- Prefer the most open direction.
- If FRONT is most open, choose GO.

Respond with ONLY ONE word:
BACK, LEFT, RIGHT, or GO
"""
```

Prompt 2: Concise Prompt has minimal and technical wording

```
f"""Task: Obstacle Avoidance.
Lidar Data: {lidar_summary}
Logic: Avoid closest, move to most open. If front is clear, GO.
Response: [BACK/LEFT/RIGHT/GO]"""
```

Prompt 3: Step-by-Step Reasoning

```
f"""Review the sensor data: {lidar_summary}.
Determine the safest path by avoiding the closest obstacle and
choosing the largest gap.
Output the command only."""
```

Prompt 4: Role-based Prompt frames the model as a robot controller

```
f"""System: You are a tunnel robot controller.
Input: {lidar_summary}
Action: Based on safety rules, choose the next move.
Command:"""
```

Prompt 5: Constraint-focused Prompt emphasizes strict output formatting rules

```
f"""{lidar_summary}
Rule: Never move towards the closest obstacle. Select the best
direction from [BACK, LEFT, RIGHT, GO]."""
```

Evaluation Metrics

Success count per prompt type out of 50

Accuracy (%) calculated as: $\text{Accuracy} = (\text{successes} / 50) * 100\%$

Results

Sample Correct Response:

[Prompt Style 3]

Prompt given to qwen2.5: 0.5b:

Review the sensor data: Front: 3.39, Left: 6.80, Right: 7.22. Closest: FRONT, Open: RIGHT.

Determine the safest path by avoiding the closest obstacle and choosing the largest gap. Output the command only.

qwen2.5: 0.5b responded correctly with a response: RIGHT

Sample of Incorrect Response:

[Prompt Style 1]

Prompt given:

The robot collided with an obstacle. Front: 1.67, Left: 5.99, Right: 5.46. Closest: FRONT, Open: LEFT

Rules:

- Do NOT choose the closest obstacle direction.
- Prefer the most open direction.
- If FRONT is most open, choose GO.

Respond with ONLY ONE word: BACK, LEFT, RIGHT, or GO

qwen2.5: 0.5b responded with an incorrect response: BACK

Expected: LEFT

This is a good example of a bad prompt because it lacks clear context and precise instructions created by the team with no style used. Even as a human, it is not immediately clear what the prompt is asking. While it provides distances and rules, it doesn't explain how these numbers relate to collision avoidance or how the robot should prioritize its actions. The mention of a collision at the start can bias both humans and the LLM toward choosing BACK, even though the rules indicate otherwise. The prompt mentions a collision but at the same time tells it to avoid the closest obstacle. An observer might ask when reading the above prompt: did the team hit something, or is the team on the verge of hitting something? This ambiguity shows the importance of structured prompt design: clear rules, explicit intent, and well-defined decision criteria are necessary to guide the model toward the correct output. Judging by this prompt, an LLM would logically choose BACK because of the collision mentioned at the start which contradicts the fact that the team is on the verge of collision. This demonstrates how poor prompt construction can lead to incorrect responses. As a human reading this prompt, I would choose back as well, when I should swerve left or right to avoid the obstacle in front of me as I'm moving.

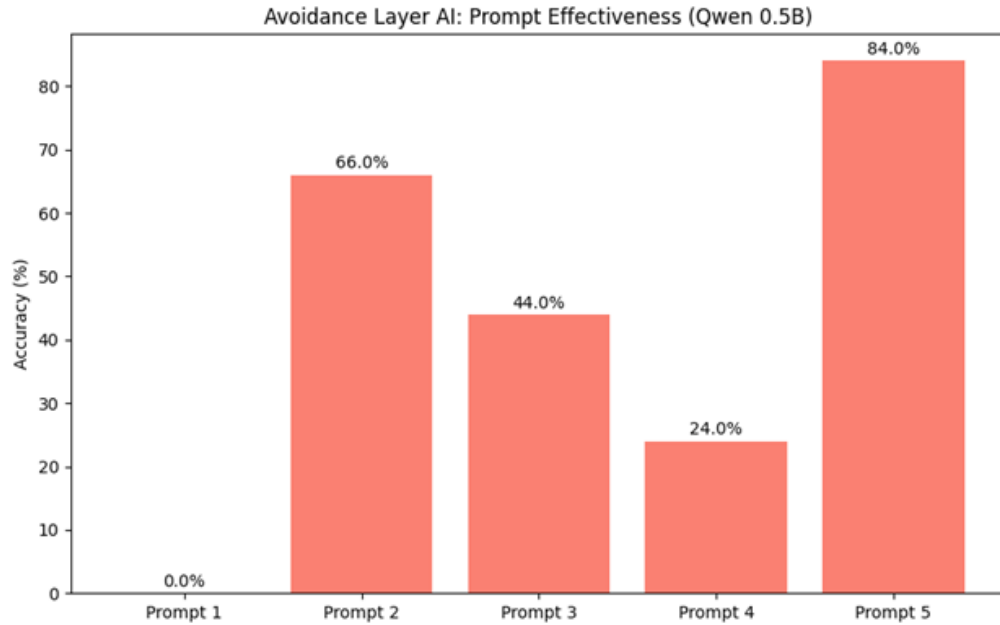


Figure 11: Bar chart of Avoidance Layer AI prompt Accuracy

From the results above, we can conclude that Prompt Style 5, a constraint-focused prompt that emphasizes strict output formatting rules, leads to better avoidance commands from Qwen2.5: 0.5B. Additionally, the results show that Prompt Style 1 (No prompt style), which includes structured instructions, achieved 0.0% accuracy. This shows that constructing a prompt that utilizes a prompt style may increase accuracy.

7.4.3 Effects of Prompt Variation on Beacon Sensor AI

By Tommy Phang

The team wanted to explore how different prompt formulations affected the accuracy of a small language model specifically qwen2.5: 0.5b in outputting correct commands in identifying the strongest beacon based on RSSI (Received Signal Strength Indicator) data. This experiment aims to identify how 5 distinct prompt styles (instructional, concise, reasoning, role-based, and constraint-based) affect the accuracy of qwen2.5: 0.5b. Ultimately identifying which prompt style yields the highest performance.

The experiment was conducted with 50 trials, for each trial three random beacons are selected. For those three beacons that were selected, each will receive three randomly generated RSSI reading in the range of $[-95, -40]$ to simulate RSSI multiple scans and noise. All this information is incorporated into scan_data string and for each trial the same scan_data is injected into all five prompt styles to ensure consistency. The LLM must select the closest beacon based on the strongest RSSI value, values closer to 0 are stronger than values closer to -100. More details on

experiment setup and procedure can be found in [Appendix F](#). Five prompt types, each with a different prompt structure are tested below:

Prompt 1: Instructional Prompt has detailed and structured instructions

```
"Choose the best (strongest) beacon based on RSSI.\n" \
    "Lower RSSI values indicate stronger signals.\n" \
    "Beacon data:\n"
for name, rssi_list in scan_data.items():
    p1 += f"-- {name}: RSSI readings: {rssi_list}\n"
p1 += 'Return only a JSON object with the following structure:
{"best_beacon": "BEACON_NAME"}'
```

Prompt 2: Concise Prompt has minimal and technical wording

```
f"Identify beacon with lowest RSSI value from: {scan_data}. Output JSON:
{{'best_beacon': 'NAME'}}"
```

Prompt 3: Role-based Prompt frames the model as a signal processor

```
f"System: Wireless Signal Processor. Data: {scan_data}. Task: Find
strongest beacon (min RSSI). Format: JSON."
```

Prompt 4: Few-shot prompt that includes an example mapping input to correct output

```
f"""Example: {{'B1': [-50], 'B2': [-90]}} -> {"best_beacon": "B1"}
Task: {scan_data}
Result:"""
```

Prompt 5: Analytical prompt that encourages reasoning about RSSI values

```
f"Analyze these signals: {scan_data}. Remember that -50 is stronger than -
80. Which is strongest? Output JSON only."
```

Evaluation Metrics

Success count per prompt type (out of 50)

Accuracy (%) calculated as: $\text{Accuracy} = (\text{successes} / 50) * 100\%$

Results

Sample Correct Response:

[Prompt Style 2]

Prompt given to qwen2.5: 0.5b:

Identify beacon with lowest RSSI value from: {"Nicol": [-77, -88, -88], "Canal": [-89, -92, -77], "UC": [-80, -40, -74]}.

Output JSON format: {'best_beacon': 'NAME'}

RESPONSE

```
```json { "best_beacon": "Canal" } ```
```

qwen2.5: 0.5b correctly identifies the beacon with the lowest RSSI value which is Canal

Sample Incorrect Response:

[Prompt Style 1]

Prompt given to qwen2.5: 0.5b:

Choose the best (strongest) beacon based on RSSI. Lower RSSI values indicate stronger signals.

Beacon data:

- Nicol: RSSI readings: [-77, -88, -88]

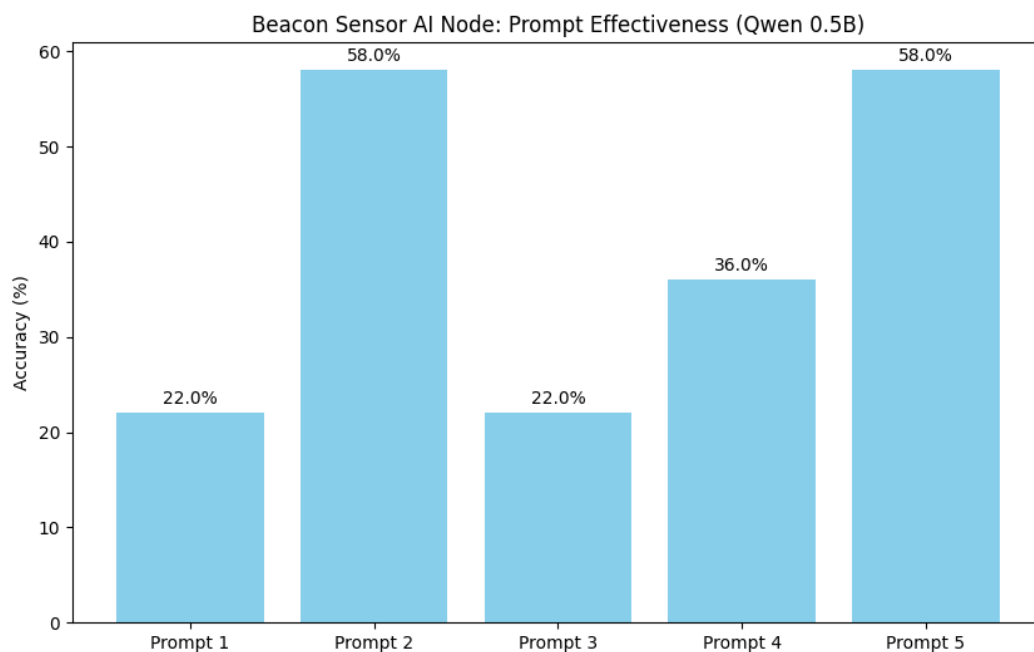
- Canal: RSSI readings: [-89, -92, -77]

- UC: RSSI readings: [-80, -40, -74]

Return only a JSON object with the following structure: {"best\_beacon": "BEACON\_NAME"}

qwen2.5: 0.5b correctly identifies the beacon with the lowest RSSI value: "Nicola"

(Expected: Canal)



*Figure 12: Bar chart of Beacon Sensor AI prompt Accuracy*

From the results above, we can conclude that Prompt Style 5 (analytical prompt), which encourages reasoning on RSSI, had the best accuracy compared to all other prompt styles tested here. From this, the team has learned that prompt style 5 is the best choice for the beacon sensor AI node for increasing qwen2.5:0.5b performance in relaying the best beacon to the robot.

## 7.4.4 Effects of Prompt Variation on Lidar Sensor AI

By Tommy Phang

The team wanted to explore how different prompt formulations affected the accuracy of a small language model specifically qwen2.5: 0.5b in processing raw LiDAR scan data and correctly calculating the minimum distance across spatial sectors. This experiment aims to identify how 5 distinct prompt styles (instructional, concise, reasoning, role-based, and constraint-based) affect the accuracy of qwen2.5: 0.5b. Ultimately identifying which prompt style yields the highest performance.

### Methodology

#### 1. Experimental setup

- Model used: qwen2.5:0.5b via ollama
- Number of trials: 30
- Task: Calculate the minimum distances for predefined LiDAR regions: front, left, right, wf\_dist (wall follow distance)
- Input: Simulated LiDAR scan data in "angle:distance" format

The LiDAR space is divided into angular regions:

- Front:  $\text{angle} \leq 10^\circ$  or  $\geq 350^\circ$
- Right:  $45^\circ \leq \text{angle} < 135^\circ$
- Left:  $225^\circ < \text{angle} \leq 315^\circ$

Here the LLM acts as a mapper that maps angle:distance pairs into the regions defined above, the expected output is a JSON object expected to look like this:

```
{
 "wf_dist": 0.89,
 "wf_angle": 355.0,
 "right": 1.12,
 "left": 4.50,
 "front": 0.89
}
```

The constants below are all predefined for these prompt tests and injected into the prompt before it is sent to the LLM for querying:

```
{
 'WF_MIN': -30, 'WF_MAX': 30,
 'FRONT_MIN': 10, 'FRONT_MAX': 350,
 'RIGHT_MIN': 45, 'RIGHT_MAX': 135,
 'LEFT_MIN': 225, 'LEFT_MAX': 315,
 'DEFAULT_DIST': 10.0
}
```

This is like how on the robot there is a loadConfig file which would load these constants and inject them into the string before passing into the LLM for querying.

## 2. Prompt styles

For each trial, five prompt types are generated:

Prompt 1: Instructional Prompt has detailed and structured instructions

```
f"""
 Analyze these Lidar readings (format \"angle:distance\").
 Data: [{data_str}]

 Task: Find the minimum distance in the following sectors.
 If a sector has no data, use {config['DEFAULT_DIST']}.

 Sectors:
 1. Wall Follow: Angle between {config['WF_MIN']} and
 {config['WF_MAX']}.
 2. Front: Angle <= {config['FRONT_MIN']} OR Angle >=
 {config['FRONT_MAX']}.
 3. Right: Angle >= {config['RIGHT_MIN']} and < {config['RIGHT_MAX']}.
 4. Left: Angle > {config['LEFT_MIN']} and <= {config['LEFT_MAX']}.

 Return ONLY a JSON object with keys: wf_dist, wf_angle, right, left,
 front.
 """
```

Prompt 2: Concise Prompt has minimal and technical wording

```
f"""Lidar Scan: [{data_str}]
 Config: Front(<= {config['FRONT_MIN']} or >= {config['FRONT_MAX']}),
 Right({config['RIGHT_MIN']} to {config['RIGHT_MAX']}),
 Left({config['LEFT_MIN']} to {config['LEFT_MAX']}), WF({config['WF_MIN']}
 to {config['WF_MAX']}).
 Goal: Find min distance per sector. Default: {config['DEFAULT_DIST']}.
 Output: JSON only {'wf_dist', 'wf_angle', 'right', 'left',
 'front'}"""
```

Prompt 3: Structured constraints

```
f"""Input Data: {data_str}
 Rules:
 - front = min dist where angle <= {config['FRONT_MIN']} or angle >=
 {config['FRONT_MAX']}
 - right = min dist where {config['RIGHT_MIN']} <= angle <
 {config['RIGHT_MAX']}
 - left = min dist where {config['LEFT_MIN']} < angle <=
 {config['LEFT_MAX']}
```

```

- wf_dist = min dist where {config['WF_MIN']} <= angle <=
{config['WF_MAX']}
- If no points, result is {config['DEFAULT_DIST']}.
Format: JSON only.

```

Prompt 4: Few-shot prompt that includes an example mapping input to correct output

```

f"""Example: Data: [0.0:1.0, 90.0:2.0] -> {"wf_dist": 10.0, "wf_angle":
0.0, "right": 10.0, "left": 2.0, "front": 1.0}
Task Data: [{data_str}]
Sector Config: Front({config['FRONT_MIN']}/{config['FRONT_MAX']}),
Right({config['RIGHT_MIN']}), Left({config['LEFT_MAX']}).
JSON Result:

```

Prompt 5: Concise data

```

f"""Analyze: {data_str}
Find min dists for sectors: Front (angles around 0), Right (pos
angles), Left (neg angles).
Specific ranges: Right [{config['RIGHT_MIN']}, {config['RIGHT_MAX']}],
Left [{config['LEFT_MIN']}, {config['LEFT_MAX']}].
Default: {config['DEFAULT_DIST']}.
Return JSON.

```

### 3. Procedure

For each of the 50 trials:

Generate 15 random LiDAR points:

- Angle  $\in [0^\circ, 360^\circ]$
- Distance  $\in [0.5, 5.0]$

Construct a string representation of scan data (data\_str)

Define the ground truth: Initialize all regions' distance to 10.0, then update each region with the minimum distance of points falling within its angular range

For each prompt:

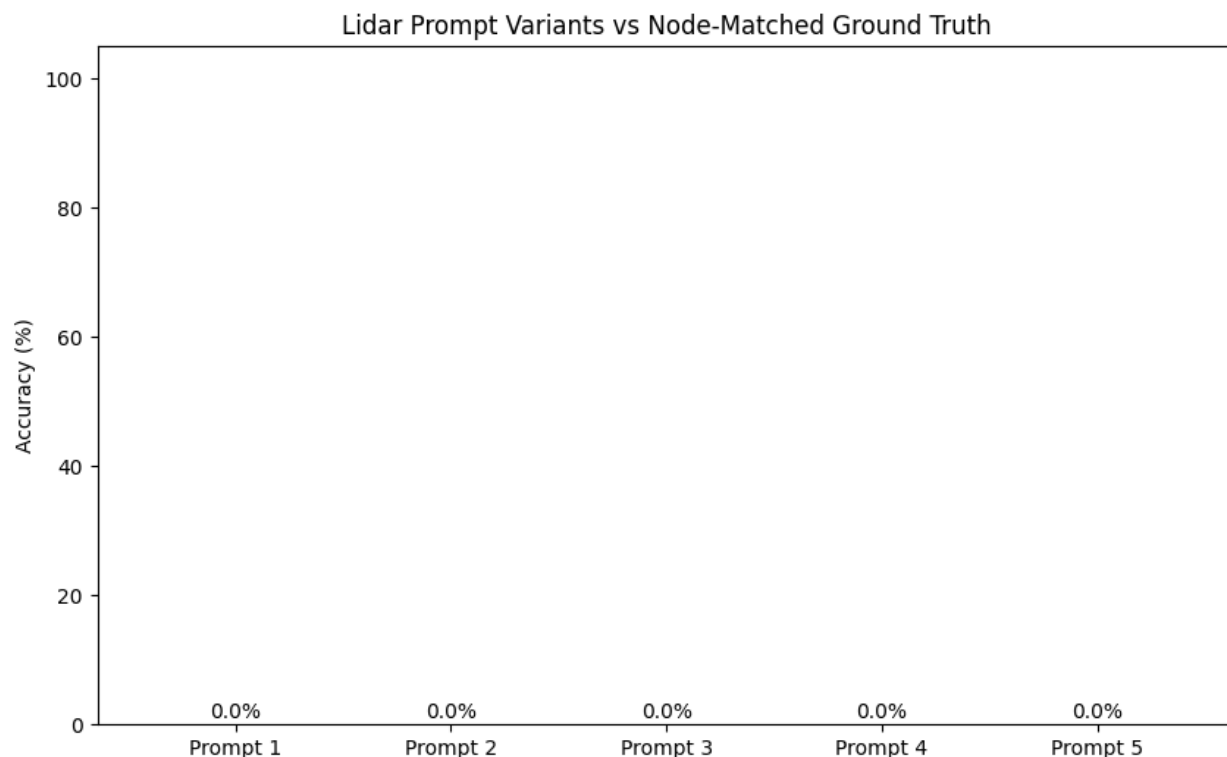
- o Query the model using ollama.generate()
- o Check correctness: If at least one correct region value (front, right, left) appears in the response, then it is a success

### 4. Evaluation Metrics

Success count per prompt type (out of 30)

Accuracy (%) calculated as:  $\text{Accuracy} = (\text{successes} / 30) * 100\%$

## Results



*Figure 13: Bar chart of LiDAR sensor AI prompt Accuracy*

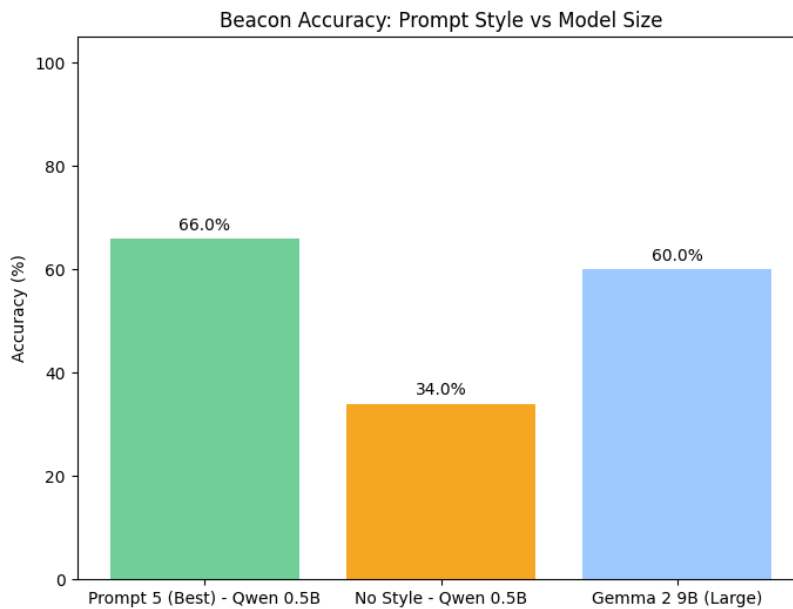
From the results above, we can conclude that utilizing LLMs to process large streams of LiDAR data is infeasible. The low accuracy across all prompts may be due to the additional constraint of requiring JSON formatting, which likely limits the model's performance by increasing cognitive load. Additionally, this task may be particularly challenging for smaller models due to their limited number of parameters, which reduces their ability to simultaneously process complex spatial LiDAR arrays while adhering to JSON formatting requirements. Smaller models, such as qwen2.5:0.5b, often struggle to follow complex instructions, leading to hallucinations or malformed JSON outputs with incorrect navigation commands. In this setup, the LLM is given a large stream of data points and acts as a mapper, converting angle–distance pairs into the predefined regions described above. These results indicate that this is a difficult task for qwen2.5:0.5b. Furthermore, when tested on the robot, it exhibited a 100% failure rate and a 100% fallback rate (falling back to last year's working code, non LLM node). Therefore, utilizing prompt styles is unlikely to resolve the LiDAR sensor node's complete failure that the team observed on the robot.

### 7.4.5 Comparing the Accuracy of a Larger LLM (Gemma 2 9B) with No Prompt Style Against the Accuracy of a Smaller LLM (Qwen 2.5: 0.5B)

By Tommy Phang

Section 7.4.5 aims to explore the comparison between the accuracy of a larger LLM such as Gemma 2 9B versus the accuracy of a smaller model such as Qwen 2.5: 0.5B. The team wants to explore if using a stronger model with no prompt style has better accuracy compared to a smaller model with a smaller model that utilizes prompt style. This test is conducted like Section 7.4.3 (beacon sensor AI) with the difference being that:

- Prompt used for no style is utilized for Gemma 2 9B and Qwen 2.5: 0.5B, no style:  
f"Identify beacon with lowest RSSI value from: scan\_data\_str}. Output JSON: {{'best\_beacon': 'NAME'}}}"
- Prompt used for Qwen 2.5: 0.5B with a tested style (Prompt 5) which was determined to be the best prompt style for Qwen 2.5: 0.5B regarding this task.
- A fixed seed is generated for each experiment to allow consistency, 7.4.3 and 7.4.5 uses the same seed to allow consistent and repeatable results between the two.



*Figure 14: Bar chart of Comparison Between Prompt Style accuracy and Model Size*

The results above demonstrate that prompt design does play a role in model performance and can improve the performance of smaller models. Qwen 2.5: 0.5B achieves the highest accuracy in this test when using Prompt Style 5 significantly outperforming both its non-style version and Gemma 2: 9B. When removing prompt structuring there is a significant drop in Qwen 2.5: 0.5B accuracy, showing the effectiveness of prompt styling. In conclusion, these findings suggest that

utilizing prompt styling can be a more efficient and impactful strategy than relying on larger models.

#### 7.4.6 Summary of Findings on Effects of Prompt Variations on Model Performance

By Tommy Phang

From this analysis, the team discovered that different prompt structures and wordings can increase the accuracy of small LLMs used within the robot. The analysis shows that formatting prompts in an instructional style with detailed and structured instructions gives better navigation results for the navigation unit AI.

The analysis also shows that using constraint focused prompts that highlight strict output formatting rules leads to higher accuracy for avoidance commands from the avoidance layer AI. In addition, structuring prompts in an analytical style that encourages step by step reasoning provides greater accuracy than other prompt styles for the beacon sensor AI node. The analysis show that small LLMs have the potential to outperform larger models when utilizing the correct prompt style, lessening the reliance on better hardware.

Finally, the analysis shows that prompt performance depends on the role of each AI node and layer, since each one works with different types of data and tasks.

Overall, this analysis shows that the team cannot just pick one prompt style and use it everywhere. Different nodes and layers require different prompt structures based on how they operate. There is no one size that fits all prompt that works for all the nodes and layer AIs, as shown in this analysis, since each one performed best with a different prompt style.

## 8.0 Metrics

### 8.1 Preface

By Owen McKibbon

To measure progress of development and performance of the robot and LLM models, metrics were needed to compare the new version's success. By crafting a list of useful metrics of robot performance that align with the project goals, the team was able to have measurable results that gave us the freedom to choose different models and make design decisions to maximize and monitor the performance of the robot.

### 8.2 Metrics Tracking

By: Connor Yelle, Jonas Andaya, Owen McKibbon, Tommy Phang

*Table 7: Table of Metrics and Justifications*

Metric	Reasoning	Method
Time per delivery – sec	This metric can be useful to calculate the speed of the robot, and to see if the trip is ending when it's supposed to end. This metric can also be useful for seeing if the LLM is improving the robot's decision making.	By keeping a log with time stamps of the robot's departure and arrival during a test, the team can measure the time per delivery.
Battery Efficiency – Voltage per delivery or m	This metric is useful to see how much of the battery is being used during delivery. If a high voltage is being used during delivery, it might be smart to change LLMs. Maybe the LLM is too big to be able to run on the robot.	There is a node inside of ROS2 that directly obtains the usage of the battery in voltage. The team must calculate the average voltage per delivery. This number can be queried every second during delivery to ensure the most accurate average possible.
Battery Life – %	This metric can be important to see how the robot uses the battery. If this percentage goes down fast, there is an	This will be tracked by checking the battery status node given to us by ROS2. All the team must do is

	optimization flaw in either the robot, or the LLM. Maybe the LLM is too big or takes too much energy to run.	multiply the node output by 100 to turn it into a percentage.
Docking Rate Success – %	It is important to see the percentage of times the robot succeeds at docking, so that the team can see if the LLM is sending the data correctly, or if the docking layer is not processing data correctly. This would be a good metric to track robot improvements.	When the robot is sent to the dock, the team can measure the time it takes to dock itself and if it is unable to do so within 30 seconds it is considered a failure. If the robot successfully transitions to the docked state, it is noted as a success.
Obstacle Avoidance Accuracy – %	This is important to see how well the Avoidance layer is transmitting data, or if the LLM is transmitting false data to the avoidance layer.	The team will measure the number of times an obstacle has been recorded in the LiDAR data versus the number of times the bumper has been hit to measure the accuracy of the obstacle avoidance.
Stuck Incidents – incidents per delivery	This metric is important to measure the cause of lengthier trips or failures of trips. It allows us to track improvements in the number of incidents to achieve a more efficient run.	If the location of the robot has no change after 30 seconds, it is marked as stuck and tallied.
Distance travel vs Battery	This metric is very important for efficient benchmarking. If the team sees an actual visual graph, the team can utilize the trend lines to see how the robot is transferring energy into physical movement.	The team can use the found odometry and measure it against the battery level the robot has.
Distance vs time	This can be shown as a graph to see either how fast the	Using the known distance between destinations and the

	robot is moving, or to see how optimal the route it is taking is. Lower times and higher distances = better robot.	logged time spent delivering the team can measure these two properties against each other.
Prompt response time	This is a crucial metric that can be used to assess the efficiency of the chosen LLM. If this metric is too high, it might be smart to either change how the LLM is processing data and sending to the different Layers. This metric can be used to analyze if the team selects a sufficient LLM for the robot.	This metric will be tracked by directly checking the Ollama server. The Ollama server sends the response time of the LLM every time it sends a new prompt.
Instruction success rate	The team wants to know how well an LLM performs in each node and layer. For each Node and layer that has AI capabilities, this metric is tracked.	For each node and layer where an LLM is deployed, the team compares the output against the ground truth, which is typically the non-AI version of that layer or node.
Time spent following wall – sec	This can be important to see how the robot is reading LiDAR sensor data and how it's converting it into actual robot movements.	During the robot's trip, the team will log the time when the LiDAR sensor picks up the wall next to it during a trip.

### 8.3 Automated Report Generation for Metrics Tracking

By Tommy Phang and Umniyah Mohammed

To support faster development and testing, the team developed an automated metrics report generation system as part of our CI/CD pipeline. This automated metrics report generation was mainly developed to help us speed up iterations by referencing performance metrics. Running the robot for every code change was not practical because it took a lot of time to set up and run. Additionally, only one person can work on the robot and its ROS2 environment at a time if they

require that environment. These constraints ended up slowing down development and made it harder for the team to iterate quickly by referencing metrics.

To solve this, the team developed a GitHub Actions workflow that uses the Gazebo simulation along with the report generation code to automatically generate performance metrics reports for every pull request and commit. One key benefit of this approach is that it removes the need for team members to set up ROS2 or the robot environment locally. Any team member can just write code and push it, and the GitHub Actions workflow will handle running everything inside a container. This allows everyone to view how their changes affect the robot's performance without waiting for access to the robot or dealing with environment setup. This allows team members to easily get feedback from performance metrics, enabling them to improve their changes.

The metrics report gets generated when the GitHub Action workflow is triggered. The GitHub Action workflow is triggered whenever a pull request is opened or updated. GitHub Actions checks out the repository, builds the Docker image, and runs the simulation inside the container. After the simulation finishes, a report consisting of performance metrics is posted as a comment on the pull request. The report displays the team's tracked metrics and highlights whether performance improved, worsened, or stayed the same.

This system provides fast feedback, so team members can quickly see how their changes affect the robot's performance after each commit. It also ensures consistency since everything runs in the same environment and makes it easier to compare metric results by including historical averages and previous run metrics within the same pull request. Thus, this allows everyone on the team to evaluate their code based on performance metrics, without waiting for access to the physical robot. It speeds up development, shortens the feedback loop, and fits well with the agile workflow by allowing team members easy access to metrics.

Metrics reports from physical robot runs are not generated automatically through our CI/CD pipeline. To generate these, team members must run the robot in Carleton's tunnels. After the run is completed, the metrics data is automatically processed into a report by the team's dashboard report generator program on the robot. The metrics report (.txt) file must be committed to the main branch, so it can automatically be added to the dashboard for easy accessibility and centralize metrics tracking. While this takes more time, comparing real world metrics against metrics generated from the simulation is important because they help confirm that results from the simulation match actual robot performance.

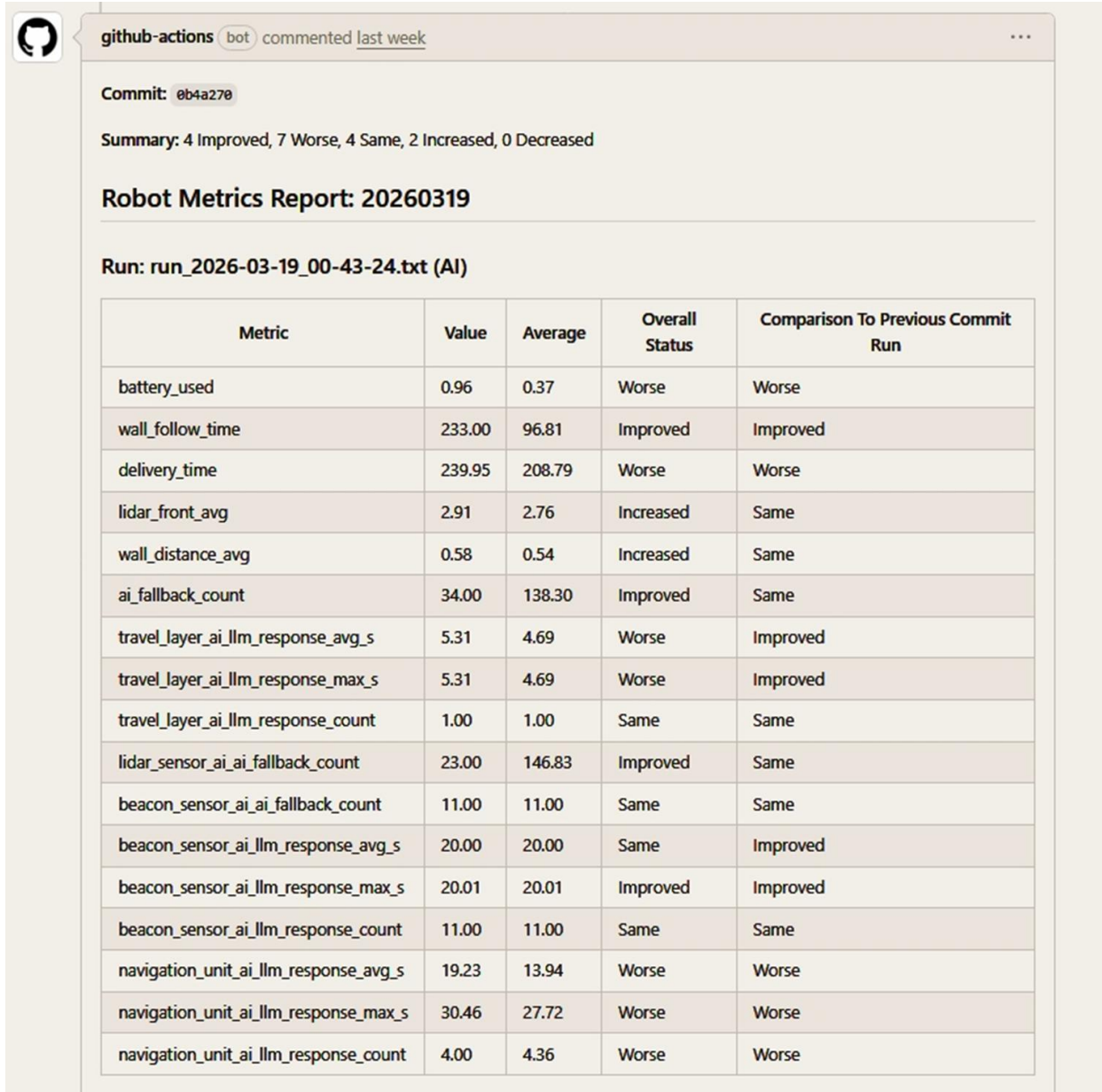


Figure 15: Report generated after creating a commit in an existing pull request.

Figure 15 shows a sample metrics report generated after a simulation run triggered by a pull request. The report includes key performance metrics such as delivery time, battery usage, LLM response times, and fallback counts. Each metric is compared against historical averages and the previous commit. This allows the team to quickly identify regressions or improvements in system performance. The “Overall Status” and comparison columns provide immediate insight into whether changes have positively or negatively impacted the robot’s behavior.

This automated evaluation approach enables data-driven decision-making during development, allowing the team to objectively assess the impact of LLM integration and system changes on overall robot performance.

## 8.4 Dashboard

By Tommy Phang

While the individual metrics reports provide detailed insights into individual runs, they are distributed across pull requests and commits. This makes it difficult to analyze performance trends over time or compare metric results across multiple runs. To address this limitation, the team developed a centralized dashboard.

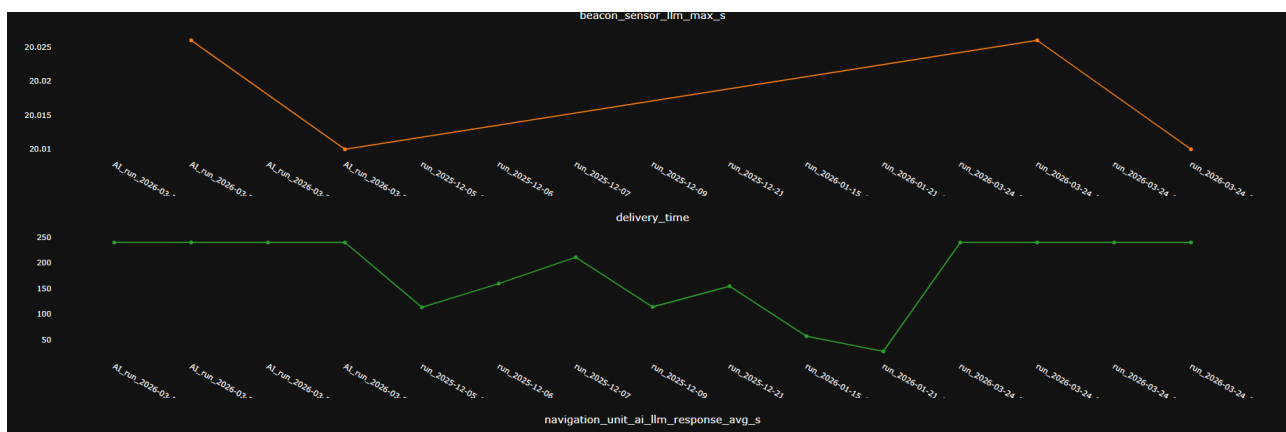
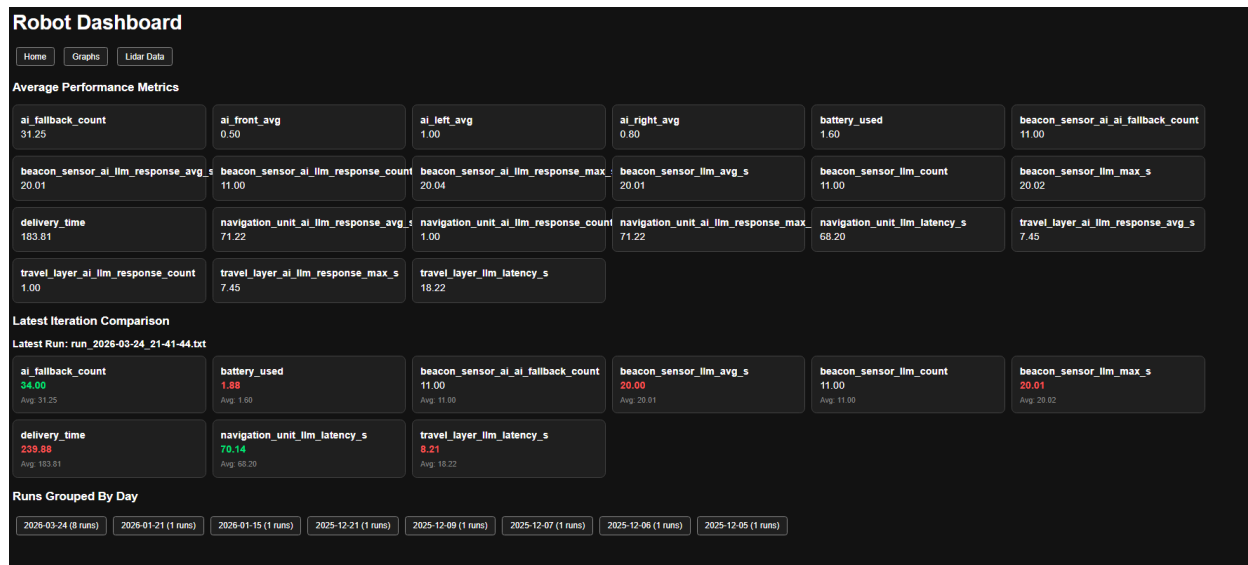
The purpose of the dashboard is to act as a unified hub for all generated metrics reports, enabling the team members to easily explore, compare, and visualize robot performance data over time. Instead of reviewing individual metric reports, the dashboard aggregates all run data and presents it in a structured and interactive format.

The dashboard was designed to meet the following requirements:

- **Centralization:** Provide a single interface to access all simulation and physical robot run metric reports.
- **Trend Visualization:** Display how performance metrics evolve across runs using graphs.
- **Run Organization:** Group runs by date to simplify navigation and comparison.
- **Traceability:** Link each run to its corresponding commits and pull requests to better pinpoint how each change affects the robot's performance.
- **Usability:** Allow team members to quickly access both high-level summaries and detailed metrics data for each run.

The dashboard includes a “Runs Grouped by Day” section, where each entry represents a testing day and indicates the number of runs completed. Selecting a day displays detailed metrics for each run, along with associated commits and pull requests. Additional pages provide performance graphs and lidar data visualizations. The graphs plot metric values across runs, allowing team members to identify trends, improvements, or regressions over time. While the lidar page summarizes sensor-related metrics and their progression, it is important for the robot wall following feature.

By combining automated report generation with a centralized dashboard, the system enables both detailed per-run analysis and long-term performance tracking. This improves the team’s ability to evaluate code changes, monitor progress, and make metrics driven decisions throughout the project's lifecycle.



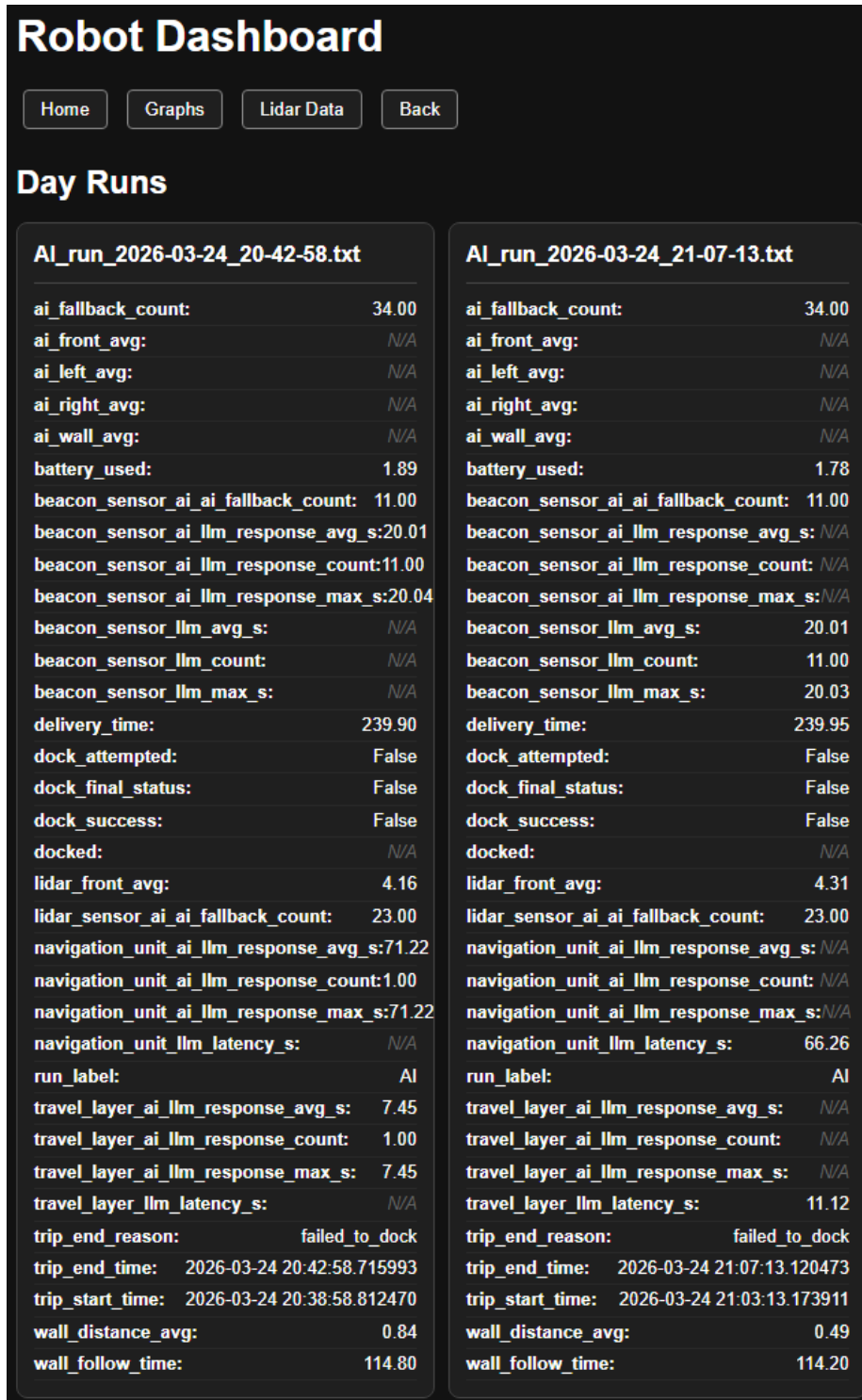


Figure 18: What team members would see if they clicked a run day allowing them to view all metric reports for the day, they clicked

## 9.0 Design

### 9.1 System Overview and ROS 2 Architecture Diagrams

By Jonas Andaya

Figure 19 shows the overall ROS2 architecture of the mail delivery robot system. It illustrates the nodes, the topics they publish and subscribe to, and the data flow between components. The global data space represents the shared memory in the ROS. Arrows going towards the global data space represent publishing. Arrows going to each of the bubbles from the global data space represent subscribing. Each of the labels next to the arrows represent the topics being subscribed to or published, depending on the direction of the arrow. Each bubble in the diagram represents a node in the ROS.

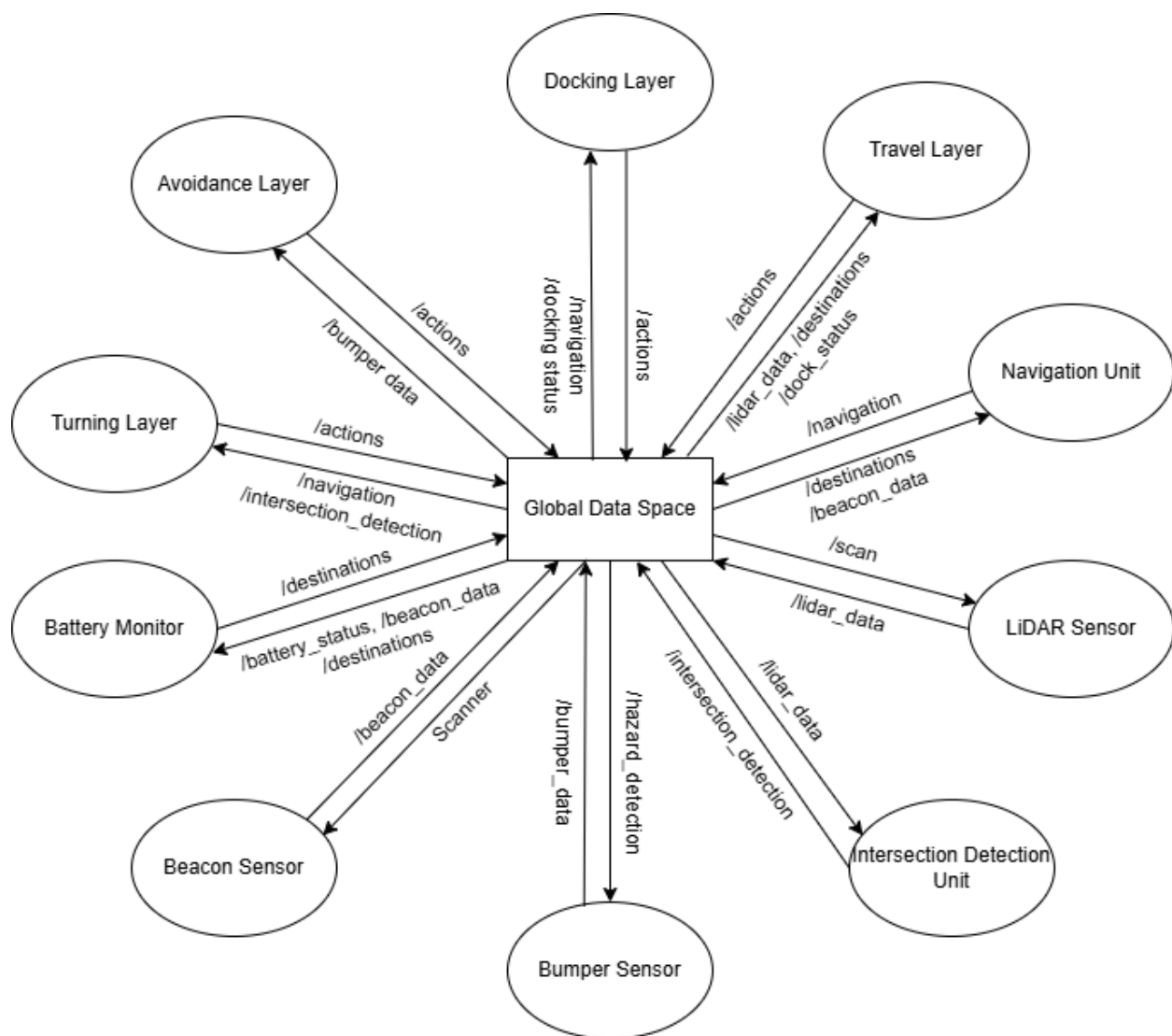


Figure 19: Overview of the ROS for the mail delivery robot.

Currently, there are four nodes in which an LLM implementation exists. The AI nodes are extensions of the normal versions of the nodes. They have the same basic functionality of normal nodes but have specific areas where the LLM takes over. Sections 9.3-9.\_ show each node's implementation with the LLM. The simple explanation as to how all the AI nodes work is that they provide a prompt to the LLM and if it fits the possible expected answers, it replaces the message that was not using an LLM. If it does not fit the possible expected answers, it falls back to the answer of the original, non-AI node.

Figure 20: Overview of the ROS of the mail delivery robot alongside the AI nodes.

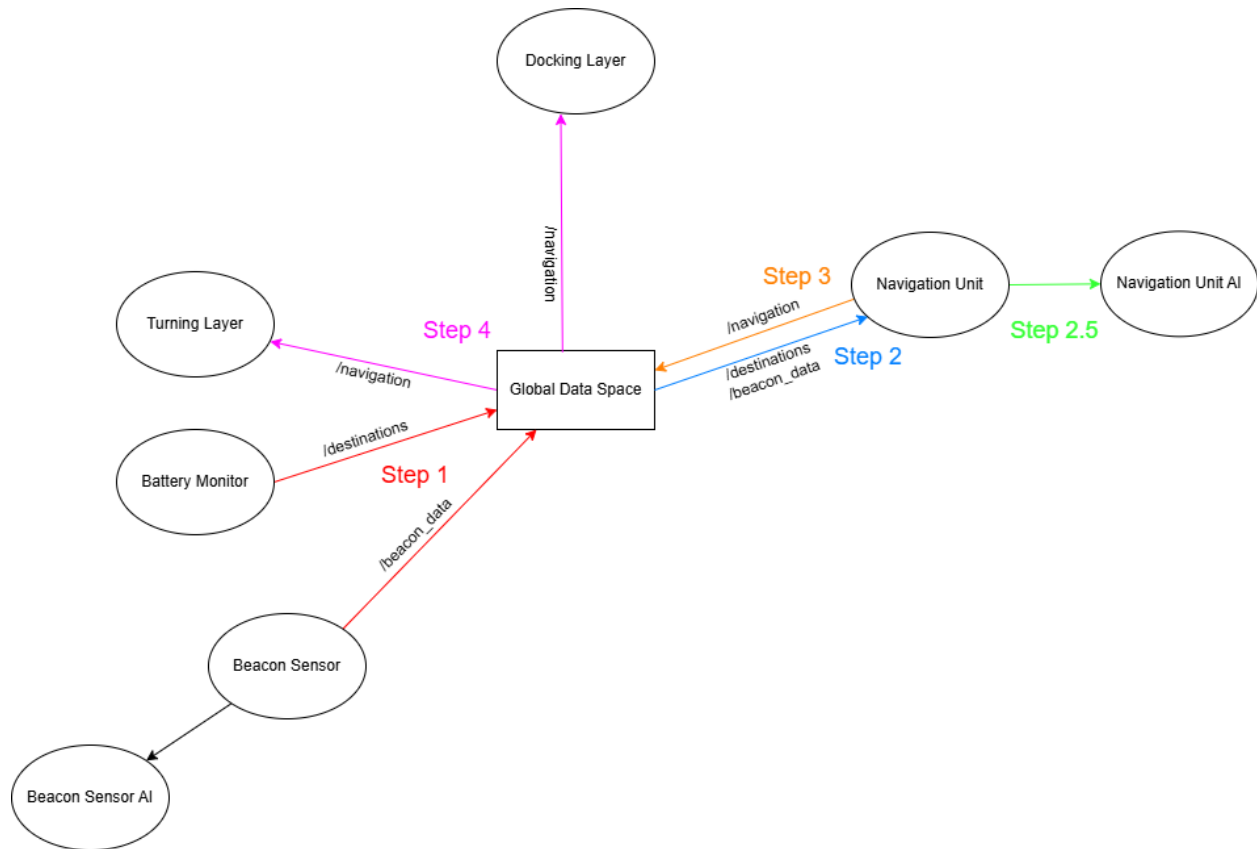


Figure 21: Flow of the /navigation topic and Navigation Unit Node.

The following diagram pictures the added nodes to the architecture of the system. In red are the newly created AI nodes which each rely on the original as a fallback. In certain cases, some AI nodes subscribe to more topics than their original predecessors. For example, the avoidance layer subscribes to both the lidar data and bumper data to make decisions for avoiding and handling collisions, whereas the original only used bumper data to handle a collision that had taken place.

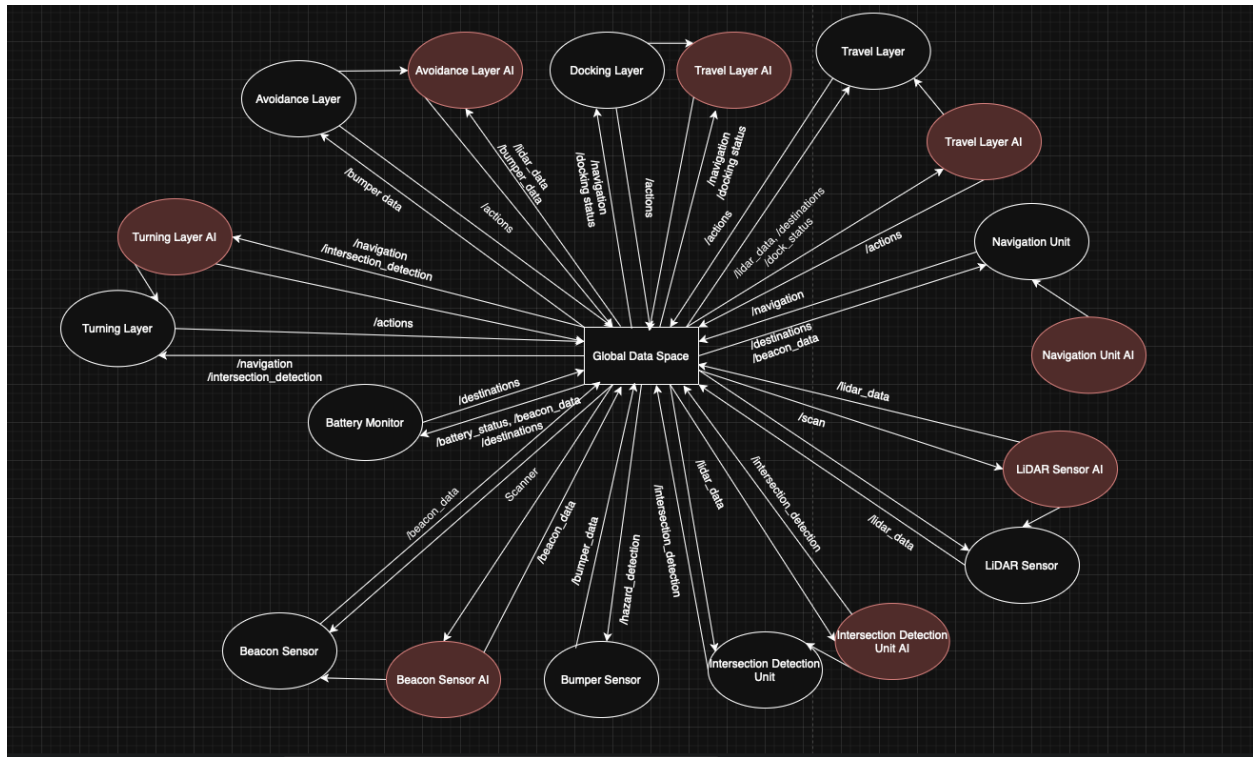


Figure 22: Comparison Overview of the ROS of the mail delivery robot with the AI nodes in Red.

## 9.2 LLM-Augmented Subsumption Architecture

By Umniyah Mohammed

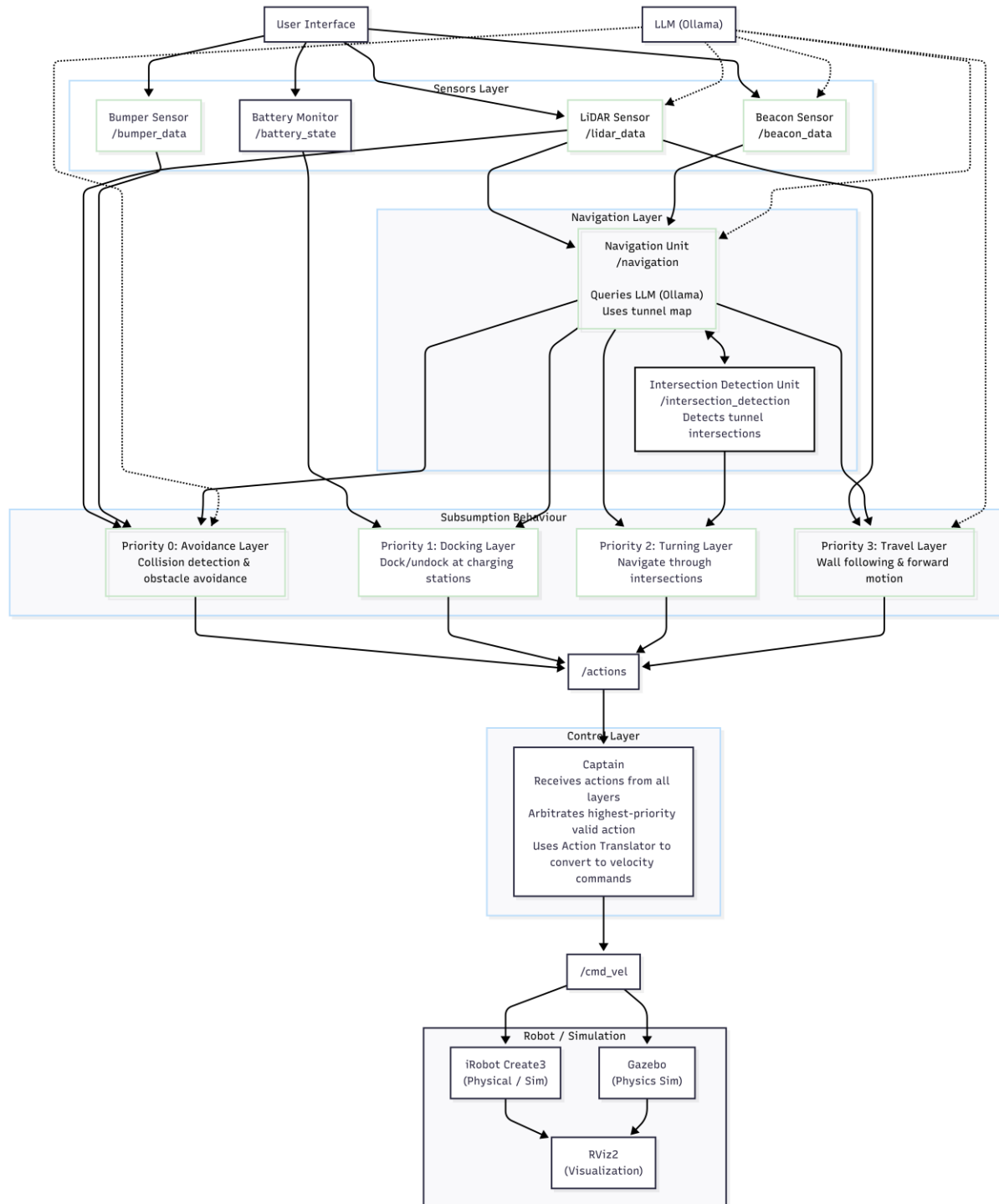


Figure 23: LLM-Augmented Subsumption Architecture

Figure 23 illustrates the subsumption-based architecture used in the mail delivery robot system, with LLM-enhanced components highlighted. The diagram shows how data flows from sensing to action selection through a layered decision-making structure, enabling the robot to operate autonomously within the tunnel environment.

At a high level, the system follows a pipeline in which sensor data is collected, processed into navigation decisions, and translated into control actions. The sensors layer gathers environmental and system data from multiple sources, including LiDAR, beacon signals, bumper sensors, and battery state. Each sensor publishes data through dedicated ROS topics, providing real-time inputs for higher-level processing. This data is then used by the navigation layer, which determines high-level movement decisions. The navigation layer consists of the Navigation Unit, which incorporates LLM-based reasoning to generate navigation commands using contextual information such as tunnel maps, and the Intersection Detection Unit, which identifies tunnel junctions. In addition to the Navigation Unit, LLM-based processing is also applied at the sensor level, where the LiDAR and beacon sensor nodes use LLM queries to interpret raw inputs and produce structured outputs for downstream decision-making.

Once navigation decisions are generated, they are passed to the subsumption behaviour layers, which are organized by priority. Each layer is responsible for a specific behaviour, such as obstacle avoidance, docking, turning, or general travel. In the updated architecture, selected behaviour layers, including the avoidance and travel layers, are augmented with LLM-based decision-making to improve performance in complex or ambiguous situations. The subsumption architecture enforces a priority hierarchy in which higher-priority behaviours override lower-priority ones when necessary. For example, the avoidance layer can override all other behaviours to prevent collisions, while lower-priority layers handle routine navigation tasks such as wall following and movement between destinations. All behaviour layers publish their intended actions to a shared `/actions` topic.

The control layer, represented by the captain node, receives these action commands and determines which action to execute based on priority. It arbitrates between competing actions, ensuring that only the highest-priority valid command is executed. The selected action is then translated into velocity commands and sent to the robot or simulation environment for execution through the `/cmd_vel` topic.

A key aspect of this architecture is its hybrid design. Each LLM-enhanced component validates its outputs against a set of allowable commands and falls back to the original rule-based implementation if the response is invalid or delayed. This allows the system to benefit from the flexibility and contextual reasoning of the LLM while maintaining the reliability and safety guarantees of the original subsumption framework.

Overall, this architecture enables modular and prioritized decision-making across multiple layers of the system. By integrating LLM-based reasoning within a subsumption framework, the design

balances adaptability and robustness, allowing the robot to handle complex scenarios while maintaining safe and predictable behaviour.

## 10.0 Implementation

### 10.1 LLM Integration Process

By Owen McKibbin, Jonas Andaya, and Umniyah Mohammed

The following subsection outlines the key methodology for LLM integration, and in-depth explanations of the process of integration for specific nodes.

#### 10.1.1 Injection Methodology

By Owen McKibbin

The injection of LLMs into the existing nodes of the previous year's system posed two challenges. The first being where to place the LLM query or queries, and the second what to give the model and what to expect from it. Though these challenges seem daunting in the web of a carefully implemented system, it becomes simpler when considering the nature of ROS. ROS is built in a modular style, reducing coupling of components as nodes. As each node is responsible only for publishing and subscribing to topics, the endpoints of each node become clear. For each layer or sensor which requires the injection of an LLM, that node can be stripped down to its subscriptions and publishing destination. Each subscription contains information necessary to derive a message to publish for other nodes to use within the larger system. The model in this scenario becomes the middleman in which the subscription data is passed as context, and a response is pulled from the LLM to be published to the next topic. The subscription data may be formatted for the LLM to achieve high performance, and a list of valid responses may sometimes be used for accurate responses however, the main sequence is subscription data to the LLM, response to the publish destination topic. For example, this is one prompt used for testing the LLM implementation of the beacon sensor node:

```
"Choose the best (strongest) beacon based on RSSI.\n" \
```

```
 "Lower RSSI values indicate stronger signals.\n" \
```

```
 "Beacon data:\n"
```

```
 for name, rssi_list in self.scan.items():
```

```
 prompt += f"- {name}: RSSI readings: {rssi_list}\n"
```

```
 prompt += """"Return only a JSON object with the following structure: {"best_beacon":
"BEACON_NAME"}"""
```

The prompt contains the beacon name and signal readings for the subscriptions of the node and expects a formatted response.

Surrounding this method the only other challenge is deriving when the LLM should be called and when it should wait. In the event of a failure of the model to provide a meaningful response, each node maintains a separate function, the functionality of the original implementation to be used as a fallback and logged for metric collection.

### 10.1.2 LLM fallback

By Owen McKibbin

In the event of the LLM providing a response that does not fit the set of proper responses for its corresponding node, the code includes a fallback to the original implementation. A proper response is one that can be formatted to fit the publishing topic's destination for its node. To be certain that the team knows when the LLM is in the driver's seat making the decisions, the team has included logs during the execution of the trip which include the prompt the LLM was given, the response of the LLM, and a message indicating the fallback was activated. This allows us to scrape the logs for all instances of the LLM providing improper responses which can then be logged as a metric of LLM successful responses.

## 10.2 Avoidance Layer LLM Implementation

By Owen McKibbin

To investigate the benefits of integrating an LLM into the decision making for reacting to and avoiding collisions, a query to an LLM was placed within the avoidance layer. To provide meaningful data for the LLM to make decisions, the avoidance layer has been made to subscribe to lidar data for information on its surroundings. A few simple helper methods format the lidar data into which regions out of the front, right, and left have the most space available and which have the least. Those methods are pictured below:

```
def get_closest_obstacle(self, lidarData):
 distances = {
 "LEFT": lidarData["left"],
 "RIGHT": lidarData["right"],
 "FRONT": lidarData["front"]
 }

 valid = {k: v for k, v in distances.items() if v > 0}
```

```

if not valid:
 return "UNKNOWN"

return min(valid, key=valid.get)

def get_most_space(self, lidarData):
 distances = {
 "LEFT": lidarData["left"],
 "RIGHT": lidarData["right"],
 "FRONT": lidarData["front"]
 }

 valid = {k: v for k, v in distances.items() if v > 0}
 if not valid:
 return "UNKNOWN"

 return max(valid, key=valid.get)

```

This data is presented to the LLM in a query containing context on its collision, the left, right, and front distances, and a list of options to choose from. During the testing of this feature, the team found that the LLM provided proper answers based on the data. The LLMs responses were used to execute a command and steer the robot in the direction away from the collision. An example interaction with the model can be seen below:

```

[captain-1] [INFO] [1770855710.802984422] [captain]: {'0': 'WAIT', '1': 'NONE', '2': 'U_TURN', '3': 'WALL_FOLLOW,0.2,-0.34793358476343356'}
[avoidance_layer_AI-5] [INFO] [1770855710.981222841] [avoidance_layer_AI]: Background:
[avoidance_layer_AI-5] You are an obstacle-avoidance controller
[avoidance_layer_AI-5] for a small robot navigating narrow tunnels.
[avoidance_layer_AI-5] Choose the safest movement."
[avoidance_layer_AI-5] [INFO] [1770855710.984450983] [avoidance_layer_AI]: Prompt:
[avoidance_layer_AI-5] The robot has collided with an obstacle.
[avoidance_layer_AI-5]
[avoidance_layer_AI-5] Lidar summary:
[avoidance_layer_AI-5] Front distance: 0.25
[avoidance_layer_AI-5] Left distance: -1.00
[avoidance_layer_AI-5] Right distance: -1.00
[avoidance_layer_AI-5] Closest obstacle: FRONT
[avoidance_layer_AI-5] Most open direction: FRONT
[avoidance_layer_AI-5]
[avoidance_layer_AI-5] Rules:
[avoidance_layer_AI-5] - Do NOT choose the closest obstacle direction.
[avoidance_layer_AI-5] - Prefer the most open direction.
[avoidance_layer_AI-5] - If FRONT is most open, choose GO.
[avoidance_layer_AI-5] Respond with ONLY ONE word:
[avoidance_layer_AI-5] BACK, LEFT, RIGHT, or GO
[avoidance_layer_AI-5]
[avoidance_layer_AI-5] [INFO] [1770855710.988235916] [avoidance_layer_AI]: Raw LLM Response: model='gwen2:0.5b' created_at='2026-02-12T00:21:50.976205.'
[avoidance_layer_AI-5] [INFO] [1770855710.995209585] [avoidance_layer_AI]: Ollama decided: LEFT
[avoidance_layer_AI-5] [INFO] [1770855711.002780119] [avoidance_layer_AI]: GOT COLLISION
[avoidance_layer_AI-5] [INFO] [1770855711.008103429] [avoidance_layer_AI]: GOT COLLISION
[beacon_sensor-7] [INFO] [1770855710.993498616] [beacon_sensor]: [SCAN RESULT] Beacon: Nicol, RSSI readings: [48]
[beacon_sensor-7] [INFO] [1770855711.003552650] [beacon_sensor]: [BEST] Selected beacon: Nicol with RSSI=48
[captain-1] [INFO] [1770855711.008104059] [captain]: {'0': 'LEFT_TURN', '1': 'NONE', '2': 'U_TURN', '3': 'WALL_FOLLOW,0.2,-0.34793358476343356'}
[avoidance_layer_AI-5] [INFO] [1770855711.012440951] [avoidance_layer_AI]: GOT COLLISION

```

Figure 244: LLM-Driven avoidance layer output

In this scenario, the model performing the response was qwen2:0.5b, which despite being quite small was able to choose meaningful responses with the help of well formatted prompting. If the model responds with an unlisted option, the avoidance layer falls back to the original functionality and logs the event.

### 10.3 Docking Layer LLM Implementation

By Umniyah Mohammed

Following the established injection methodology outlined in Section 10.3.1, the LLM was integrated into the docking layer of the robot's subsumption architecture. The original docking layer was responsible for managing the robot's docking and undocking behaviour using simple conditional logic based on two pieces of information: dock visibility and the robot's current docked status. While functional after the bug fixes described in Section 10.4.1, this logic was rigid. The robot could only choose to dock when the dock was visible, or do nothing otherwise. There was no capacity for the robot to reposition, adjust its approach, or reason about environmental conditions. The goal of this integration was to replace this fixed decision point with an LLM query, allowing the model to make context-aware docking decisions using richer sensor data.

A new node, `docking_layer_AI.py`, was created to house the LLM-enhanced docking layer. The node follows the same ROS subscriber/publisher pattern as the original: it subscribes to `/navigation` for navigation commands and `/dock_status` for dock visibility and docked state. In addition, a new subscription to `/lidar_data` was introduced to provide the LLM with spatial awareness of the robot's surroundings during the docking maneuver.

The decision point where the original logic determined whether to dock was identified within the `update_actions` timer callback. In the original implementation, this was a simple check:

```
elif self.state == DockingLayerStates.HAS_DEST and self.last_navigation_message == 'DOCK':
 if self.dock_visible:
 self.action_publisher.publish(self.dock_msg)
 elif self.is_docked:
 self.state = DockingLayerStates.NO_DEST
 else:
 self.action_publisher.publish(self.no_msg)
```

*Figure 255: Dock decision making code snippet*

This decision point was replaced with a call to the LLM through the `ai_docking_query` method. Before querying the model, a context summary is assembled by `get_docking_context`, which aggregates dock visibility, docked status, the number of docking attempts, the current navigation command, and lidar distances in the front, left, and right directions.

The prompt provided to the LLM includes a system message establishing the model's role as a docking controller, followed by a user message containing the situational context and a set of rules to guide the decision. The prompt concludes with a constrained list of valid responses:

```
response = ollama.chat(model='gemma2:2b-instruct-q4_0', messages=[
 {
 "role": "system",
 "content": (
 "You are a docking controller for a mail delivery robot. "
 "You help the robot dock and undock safely at charging stations. "
 "You must make quick, safe decisions based on sensor data."
)
 },
 {
 "role": "user",
 "content": f"""
 Situation: {situation}

 {context}

 Rules:
 - If dock is visible and path is clear (front > 0.5m), proceed with DOCK
 - If dock is not visible, suggest WAIT or ADJUST (LEFT/RIGHT) to find it
 - If too many failed attempts, suggest BACK to reposition
 - Safety is the priority

 Respond with ONLY ONE word:
 DOCK, UNDOCK, WAIT, LEFT, RIGHT, BACK, or NONE
 """,
 }
])
```

*Figure 266: System prompt for Ollama*

This prompt structure mirrors the pattern established in other LLM-integrated nodes. The list of valid responses (DOCK, UNDOCK, WAIT, LEFT, RIGHT, BACK, or NONE) maps directly to pre-defined ROS String messages that the docking layer publishes to /actions. Compared to the original node, which could only publish DOCK, UNDOCK, or NONE, the LLM-enhanced version expands the action space to include WAIT, LEFT\_TURN, RIGHT\_TURN, and BACK, giving the model richer maneuvering options during the docking approach.

The fallback mechanism described in Section 10.3.2 was also implemented. If the LLM fails to respond or the Ollama connection is unavailable, the `ai_docking_query` method returns "FALLBACK", and the `update_actions` method delegates to `fallback_docking_logic`, which preserves the original conditional behaviour. All LLM queries and decisions are logged for metric collection.

Additionally, a retry mechanism was introduced: if the robot exceeds a configurable maximum number of docking attempts (`max_docking_attempts = 3`), the cached LLM decision is cleared, and the model is re-queried with updated sensor data. This ensures the robot does not persist with a stale decision if conditions have changed.

To register the new node with the ROS package, the entry point `docking_layer_AI = layers.docking_layer_AI: main` was added to `setup.py`, and the AI launch file `AI_robot.launch.py` was updated to use `docking_layer_AI` as the executable in place of `docking_layer`.

To conclude, the integration of the LLM into the docking layer followed the same blueprint established by the earlier navigation and avoidance layer implementations. By subscribing to lidar data in addition to the existing dock status and navigation topics, the docking layer provides the LLM with sufficient context to make spatially aware decisions. The expanded action space allows the model to suggest repositioning maneuvers such as turning or backing up, which the original implementation could not accommodate. The fallback mechanism and logging infrastructure were reused from the existing framework, ensuring consistency across all LLM-integrated nodes and enabling systematic comparison of LLM performance against the original logic during testing.

## 10.4 Turning Layer LLM Implementation

By Umniyah Mohammed

The turning layer is the subsumption layer responsible for executing directional commands (left turns, right turns, U-turns, and going straight) when the robot reaches a hallway intersection. The original turning layer operated deterministically: it received a navigation command from the `/navigation` topic, waited until the `/intersection_detection` topic confirmed the robot was inside an intersection, and then executed the commanded turn for a fixed number of timer cycles. This approach had no awareness of the robot's physical surroundings; if the navigation system commanded a left turn into a blocked corridor, the original layer would execute it regardless. The goal of the LLM integration was to introduce a validation step where the model could confirm or override the planned turn using real-time LIDAR data, improving safety at intersections.

A new node, `turning_layer_AI.py`, was created to implement the LLM-enhanced turning layer. The node retains the original subscriptions to `/navigation` and `/intersection_detection`, and introduces a new subscription to the `/scan` topic to receive raw LIDAR LaserScan data.

Unlike the docking layer which received pre-processed lidar data as a formatted string, the turning layer subscribes directly to the raw LaserScan message from the `/scan` topic. This required a helper method, `lidar_summary`, to partition the 360-degree scan into four directional zones (front, left, right, and back) and extract the minimum distance in each zone:

```

def lidar_summary(self, scan):
 """Summarize LIDAR ranges into zones for the LLM."""
 if scan is None or len(scan.ranges) == 0:
 return "Lidar data unavailable."

 n = len(scan.ranges)
 front = min(scan.ranges[0:n // 8] + scan.ranges[7 * n // 8:n])
 left = min(scan.ranges[n // 8: 3 * n // 8])
 back = min(scan.ranges[3 * n // 8: 5 * n // 8])
 right = min(scan.ranges[5 * n // 8: 7 * n // 8])

 return (
 f"LIDAR distances (meters):\n"
 f" Front: {front:.2f}\n"
 f" Left: {left:.2f}\n"
 f" Right: {right:.2f}\n"
 f" Back: {back:.2f}"
)

```

Figure 277: `lidar_summary()` code snippet

This summary is passed to the LLM as part of the prompt context. The key design distinction of this integration is that the LLM does not replace the navigation command outright; instead, it acts as a validation layer. The navigation system's commanded turn is presented to the model alongside the LIDAR distances, and the model is asked to either confirm the command or override it with a safer alternative. The prompt is constructed within the `ai_turning_query` method:

```

response = ollama.chat(model='qwen2:0.5b', messages=[
 {
 "role": "system",
 "content": (
 "You are a turning controller for a small mail-delivery robot "
 "navigating hallway intersections. Your job is to validate or "
 "override the planned turn based on LIDAR sensor data."
)
 },
 {
 "role": "user",
 "content": (
 f"The navigation system has commanded: {nav_command}.\n\n"
 f"{lidar_text}\n\n"
 "Rules:\n"
 "- If the commanded direction is safe (distance > 0.5m), CONFIRM it.\n"
 "- If the commanded direction is blocked, choose the safest alternative.\n"
 "- Valid responses: LEFT_TURN, RIGHT_TURN, U_TURN, STRAIGHT\n\n"
 "Respond with ONLY ONE of: LEFT_TURN, RIGHT_TURN, U_TURN, STRAIGHT"
)
 }
])

```

Figure 288: System Prompt for the Ollama chat

This integration uses the `qwen2:0.5b` model, which is smaller and faster than the `gemma2:2b-instruct-q4_0` model used in the docking layer. Since the turning layer's timer fires at 200ms intervals and the robot must not stall at intersections, a lighter model was selected to reduce inference latency.

The LLM's response is validated against the set of four valid actions (LEFT\_TURN, RIGHT\_TURN, U\_TURN, STRAIGHT). If the model returns an invalid or unrecognizable response, the fallback behavior trusts the original navigation command. Similarly, if the Ollama connection fails entirely, the navigation command is used as-is. This preserves the fallback-first safety pattern described in Section 10.3.2.

To avoid redundant LLM queries on every timer tick, the decision is cached in `self.current_ai_decision` and only reset when a new navigation command arrives. This means the model is queried once per intersection encounter, and the result is reused across the turn and go cycles that follow. This caching mechanism is an improvement over some of the earlier LLM node integrations where repeated queries on each timer tick could introduce unnecessary latency.

The `update_actions` method preserves the original cycle-based execution model. The robot still performs the turn for a configured number of cycles and then drives forward for additional cycles before releasing control. The only difference is that the action being executed may have been modified by the LLM. Notably, if the model overrides a U-turn command to a different action, the method re-routes execution to the regular turn logic.

To support flexible deployment, the `robot.launch.py` launch file was updated with a `use_ai_turning` launch argument that toggles between the standard and AI turning layers using ROS `IfCondition` and `UnlessCondition` (`robot.launch.py`). This allows the AI turning layer to be enabled selectively with `ros2 launch mail-delivery-robot robot.launch.py use_ai_turning:=true`, without affecting other nodes. The AI launch file (`AI_robot.launch.py`) was also updated to use `turning_layer_AI` by default.

In conclusion, the turning layer LLM integration introduces a safety-oriented validation step into the robot's intersection navigation. Rather than blindly executing the navigation system's command, the LLM evaluates LIDAR data to confirm whether the commanded direction is safe or whether an alternative should be taken. The use of a smaller model (qwen2:0.5b) and per-command decision caching keeps inference overhead minimal while still allowing the model to contribute meaningful safety checks. The conditional launch argument ensures backward compatibility. This enables side-by-side comparison between the original and AI-enhanced turning behaviour during testing and metric collection.

## 10.5 Beacon Sensor LLM Implementation

By Jonas Andaya and Owen McKibbin

The purpose of the beacon sensor node is to identify beacons for the robot to tell what intersection or dock it is closest to. The original beacon sensor node was responsible for scanning for nearby Bluetooth beacons and publishing the strongest detected signal. It did not subscribe to any ROS2 topics, as it operated independently using BluePy's scanner to directly interface with Bluetooth hardware. For more information on BluePy, refer back to section 4.2.7. The node

periodically performed scans based on a configured timer, filtered detected Bluetooth devices and stored their Received Signal Strength Indicator (RSSI) values if they met a specified threshold. After a fixed number of scans, it selected the beacon with the lowest RSSI value, which indicates the strongest signal, and published the result to the `/beacon_data` topic.

The beacon sensor node was extended to incorporate an LLM decision-making component for selecting the strongest beacon. The updated AI-based beacon sensor maintains the same core functionality as the original node and similarly does not subscribe to any ROS2 topics.

The decision point was implemented within the `checkForBeacons()` function, after the required number of scans had been completed. At this stage, the system attempts to determine the best beacon by calling the `pick_beacon_ai()` function. A dedicated function, `pick_beacon_ai()`, was created to handle all interactions with the model. Beacon data collected during scanning is formatted into a structured prompt.

```
"""
```

```
Choose the best (strongest) beacon based on RSSI.\n
```

```
 Lower RSSI values indicate stronger signals.\n
```

```
 Beacon data: {name}: RSSI readings: {rssi_list}\n
```

```
Return only a JSON object with the following structure: {"best_beacon": "BEACON_NAME"}
```

```
"""
```

This prompt includes a description of how RSSI values relate to signal strength and list of detected beacons and their corresponding RSSI readings. The model is instructed to return a JSON object containing the name of the strongest beacon. If the LLM selected beacon is valid, it is published to the `/beacon_data` topic. Otherwise, the fallback method is used.

## 10.6 Navigation Unit LLM Implementation

By Owen McKibbin

A major part of the LLM integration is the navigation unit for robots. The navigation unit is responsible for publishing directional commands based on the robot's relative location to beacons, and its destination. The original implementation of the navigation unit relied on subscriptions to `/destinations` and `/beacon_data`, as well as a map of the tunnels' most optimal directions for each intersection. The navigation unit would use the destination and closest beacon from the beacon data, to look up in a csv file map of the optimal directions, the best direction to take. The navigation unit would then publish its decision derived from the map to `/navigation` for its command to be actuated by the captain.

When considering how an LLM fits into this unit, the endpoints of the subscriptions and publish destination were to remain. In between the publishing and subscribing is where the LLM had freedom to decide based on the subscriptions what command to give the captain. The decision was to provide LLM with general info from the subscriptions as to where it needed to get to and where it was currently. In addition to this the prompt included a list of options to choose from for commands. This list of options was helpful to keep the response in line with the formatting of the command to be published to /navigation for the captain. As mentioned in section 6.6 a selection of prompts were tested to achieve ideal performance, the challenge with including more context for the LLM to produce an accurate choice was the response time increasing significantly. An example prompt that was frequently used for testing is shown below.

""""

The robot is currently at beacon {state["current\_beacon"]} and needs to go to {state["destination"]}.

What direction should it take next from the following list of options based on the current state and map of the tunnels?

- NAV\_LEFT
- NAV\_RIGHT
- NAV\_PASS
- NAV\_U-TURN
- NAV\_DOCK

Provide only one of these directions as the output.

""""

To ensure that the navigation unit is not queried unnecessarily, the navigation unit is made to be prompted only when the current beacon the robot is closest to changes. This signals that the robot has reached a decision point and requires insight into the direction or action to take.

## 10.7 LiDAR Sensor LLM Implementation

By Owen McKibbin

The original implementation of the LiDAR sensor node subscribes to /scan data directly from the physical sensor and published to /lidar\_data with formatted information of the robot's surroundings. The integration of an LLM into this node is a challenge in that raw scan data is large and difficult to read, and that scans are being made of the surroundings constantly. Using the scan data the LLM was often unable to provide meaningful responses that fit an appropriate

format at all. The processing time of the LLM was too large to provide a meaningful response in a timely manner to act on. Formatting the data ahead of submission to the model as context may help with this but would ultimately defeat the purpose of the model formatting the data itself. Below is an example of a frequently used prompt for testing of the LiDAR sensor.

"""

Analyze these Lidar readings (format "angle:distance").

Data: [{data\_str}]

Task: Find the minimum distance in the following sectors.

If a sector has no data, use {default\_dist}.

Sectors:

1. Wall Follow: Angle between {self.config["WALL\_FOLLOW\_MIN\_ANGLE"]} and {self.config["WALL\_FOLLOW\_MAX\_ANGLE"]}.

2. Front: Angle  $\leq$  {self.config["FRONT\_MIN\_ANGLE"]} OR Angle  $\geq$  {self.config["FRONT\_MAX\_ANGLE"]}.

3. Right: Angle  $\geq$  {self.config["RIGHT\_MIN\_ANGLE"]} and  $<$  {self.config["RIGHT\_MAX\_ANGLE"]}.

4. Left: Angle  $>$  {self.config["LEFT\_MIN\_ANGLE"]} and  $\leq$  {self.config["LEFT\_MAX\_ANGLE"]}.

Return ONLY a JSON object with keys: wf\_dist, wf\_angle, right, left, front.

"""

During integration test runs it was found that the LiDAR sensor ai node interfered with other nodes by spending processing time thinking on a response for the LiDAR data. It was found that the integration of the ai LiDAR sensor node with other nodes was a hinderance and not a suitable location for LLM based opportunity of improvement.

## 10.8 AI node Integration

By Owen McKibbin

During the test runs of the LLM nodes working in conjunction with one another, it was found that the combination of the ai beacon sensor node, ai navigation node, and ai docking node, were best suited for running together. These three nodes each follow the pipeline of sensors, decision-making, to actuation. Below is an example snippet from simulation logs of the three nodes working together to successfully decide to dock.

```
[captain-1] [INFO] [1775609730.355878474] [captain]: {'0': 'NONE', '1': 'NONE', '2': 'NONE', '3': 'WALL_FOLLOW,0.2,2.088504355070103'}
[beacon_sensor_AI-3] [INFO] [1775609730.482460816] [beacon_sensor_AI]: Parsed beacon_part: Beacon=ea_2f_93_a6_98_20, signal_part: SignalStrength=-100.
[beacon_sensor_AI-3] [INFO] [1775609730.482890966] [beacon_sensor_AI]: Scan counter: 9, received beacon: ea_2f_93_a6_98_20 with RSSI: 100
[beacon_sensor_AI-3] [INFO] [1775609730.483944297] [beacon_sensor_AI]: Parsed beacon_part: Beacon=e2_77_fc_f9_04_93, signal_part: SignalStrength=-99.8:
[beacon_sensor_AI-3] [INFO] [1775609730.484244131] [beacon_sensor_AI]: Scan counter: 10, received beacon: e2_77_fc_f9_04_93 with RSSI: 99
[beacon_sensor_AI-3] [INFO] [1775609730.485042342] [beacon_sensor_AI]: Parsed beacon_part: Beacon=fb_ef_5c_de_ef_e4, signal_part: SignalStrength=-100.
[beacon_sensor_AI-3] [INFO] [1775609730.485508013] [beacon_sensor_AI]: Scan counter: 11, received beacon: fb_ef_5c_de_ef_e4 with RSSI: 100
[beacon_sensor_AI-3] [INFO] [1775609730.486151050] [beacon_sensor_AI]: Parsed beacon_part: Beacon=fc_e2_2e_62_9b_3d, signal_part: SignalStrength=-99.8:
[beacon_sensor_AI-3] [INFO] [1775609730.486534953] [beacon_sensor_AI]: Scan counter: 12, received beacon: fc_e2_2e_62_9b_3d with RSSI: 99
[beacon_sensor_AI-3] [INFO] [1775609730.486919121] [beacon_sensor_AI]: >>> REACHED AI BLOCK IN checkForBeacons() <<<
[beacon_sensor_AI-3] [INFO] [1775609730.487260350] [beacon_sensor_AI]: >>> pick_beacon_ai CALLED <<<
[beacon_sensor_AI-3] [INFO] [1775609730.487624152] [beacon_sensor_AI]: Preparing prompt for AI with scan data: {'df_2b_70_a8_21_90': [7], 'd0_6a_d2_02_
[beacon_sensor_AI-3] [INFO] [1775609730.487916217] [beacon_sensor_AI]: Sending prompt to Ollama:
[beacon_sensor_AI-3] Choose the best (strongest) beacon based on RSSI.
[beacon_sensor_AI-3] Lower RSSI values indicate stronger signals.
[beacon_sensor_AI-3] Beacon data:
[beacon_sensor_AI-3] - df_2b_70_a8_21_90: RSSI readings: [7]
[beacon_sensor_AI-3] - d0_6a_d2_02_42_eb: RSSI readings: [19]
[beacon_sensor_AI-3] - ee_16_86_9a_c2_a8: RSSI readings: [99]
[beacon_sensor_AI-3] - ea_2f_93_a6_98_20: RSSI readings: [99, 100]
[beacon_sensor_AI-3] - fc_e2_2e_62_9b_3d: RSSI readings: [99, 99]
[beacon_sensor_AI-3] - e4_87_91_3d_1e_d7: RSSI readings: [100]
[beacon_sensor_AI-3] - fb_ef_5c_de_ef_e4: RSSI readings: [99, 100]
[beacon_sensor_AI-3] - e2_77_fc_f9_04_93: RSSI readings: [100, 99]
[beacon_sensor_AI-3] Return only a JSON object with the following structure: {"best_beacon": "BEACON_NAME"}
[beacon_sensor_AI-3] [INFO] [1775609730.489606991] [beacon_sensor_AI]: Waiting for Ollama response...
[captain-1] [INFO] [1775609730.561748817] [captain]: {'0': 'NONE', '1': 'NONE', '2': 'NONE', '3': 'WALL_FOLLOW,0.2,2.088504355070103'}
[navigation_unit_AI-6] [INFO] [1775609730.770242017] [navigation_unit_AI]: Current Direction: NAV_RIGHT, Can Send: False
[captain-1] [INFO] [1775609730.771500087] [captain]: {'0': 'NONE', '1': 'NONE', '2': 'NONE', '3': 'WALL_FOLLOW,0.2,2.088504355070103'}
[captain-1] [INFO] [1775609730.961330580] [captain]: {'0': 'NONE', '1': 'NONE', '2': 'NONE', '3': 'WALL_FOLLOW,0.2,2.088504355070103'}
[captain-1] [INFO] [1775609731.171182366] [captain]: {'0': 'NONE', '1': 'NONE', '2': 'NONE', '3': 'WALL_FOLLOW,0.2,2.088504355070103'}
[beacon_sensor_AI-3] [INFO] [1775609731.342602173] [beacon_sensor_AI]: pick_beacon_ai: latency=0.855s
[beacon_sensor_AI-3] [INFO] [1775609731.342986420] [beacon_sensor_AI]: Ollama response content: {'best_beacon': 'ea_2f_93_a6_98_20'}
[beacon_sensor_AI-3] [INFO] [1775609731.343295979] [beacon_sensor_AI]: [AI RESULT] Selected beacon: ea_2f_93_a6_98_20 with RSSI=100
[beacon_sensor_AI-3] [INFO] [1775609731.344410743] [beacon_sensor_AI]: Parsed beacon_part: Beacon=df_2b_70_a8_21_90, signal_part: SignalStrength=8.142!
[beacon_sensor_AI-3] [INFO] [1775609731.344904915] [beacon_sensor_AI]: Scan counter: 1, received beacon: df_2b_70_a8_21_90 with RSSI: 8
[navigation_unit_AI-6] [INFO] [1775609731.345084769] [navigation_unit_AI]: Received beacon data: ea_2f_93_a6_98_20,100
[beacon_sensor_AI-3] [INFO] [1775609731.345721753] [beacon_sensor_AI]: Scan count: 1 / 12.0
[navigation_unit_AI-6] [INFO] [1775609731.345735219] [navigation_unit_AI]: Current Destination: Canal, Previous Beacon: df_2b_70_a8_21_90
[navigation_unit_AI-6] [INFO] [1775609731.346190663] [navigation_unit_AI]: Current Beacon from data: ea_2f_93_a6_98_20
```

Figure 299: Beacon\_sensor\_AI console output

```
[captain-1] [INFO] [1775609731.744012565] [captain]: irobot_create_msgs.action.Dock_Feedback(sees_dock=True)
[navigation_unit_AI-6] [INFO] [1775609731.779537305] [navigation_unit_AI]: Current Direction: NAV_PASS, Can Send: True
[captain-1] [INFO] [1775609731.779725353] [captain]: {'0': 'NONE', '1': 'NONE', '2': 'NONE', '3': 'WALL_FOLLOW,0.2,2.088504355070103'}
[navigation_unit_AI-6] [INFO] [1775609731.782599503] [navigation_unit_AI]: *****PROMPTING LLM FOR DIRECTION DECISION...*****
[navigation_unit_AI-6] [INFO] [1775609731.782969577] [navigation_unit_AI]: Prompt to LLM:
The robot is currently at beacon ResComms and needs to go to Canal.
Below is a map of the tunnels with the required navigation action to get from each beacon (rows) to each other beacon (i
[navigation_unit_AI-6]
Map of tunnels:
,Loeb,Southam,TunnelJunction,Library,Azrieli/Tory,Steacie,Richcraft,UC,Canal,Nicol,Mackenzie/Minto,Frontenac,ResComms,CTTC,I
Loeb,NAV_DOCK,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV
Southam,NAV_PASS,NAV_DOCK,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_PASS,NAV_PASS,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV
TunnelJunction,NAV_RIGHT,NAV_RIGHT,NAV_DOCK,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV
Library,NAV_LEFT,NAV_LEFT,NAV_LEFT,NAV_DOCK,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV
Azrieli/Tory,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_DOCK,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV
Steacie,NAV_RIGHT,NAV_RIGHT,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_DOCK,NAV_PASS,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV
Richcraft,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_DOCK,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_P
UC,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_PASS,NAV_PASS,NAV_DOCK,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN
Canal,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_DOCK,NAV_U-TURN,NAV_U-TURN,NAV_U-TURN,NAV
Nicol,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_DOCK,NAV_U-TURN,NAV_U-TI
Mackenzie/Minto,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_P
Frontenac,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_DOCK,NAV_U
ResComms,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_DOCK
CTTC,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGHT,NAV_RIGH
Robertson,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_PASS,NAV_P
[navigation_unit_AI-6]
The robot can only take one action at a time, and must follow the actions in the map to navigate correctly.
If the destination is the same as the current location, it should choose NAV_DOCK.
What direction should it take next from the following list of options based on the current state and map of the tunnels:
- NAV_LEFT
- NAV_RIGHT
- NAV_PASS
- NAV_U-TURN
- NAV_DOCK
Provide only one of these directions as the output.
[navigation_unit_AI-6]
[INFO] [dashboard_logger-12]: process has finished cleanly [pid 62007]
[captain-1] [INFO] [1775609731.987799064] [captain]: {'0': 'NONE', '1': 'NONE', '2': 'NONE', '3': 'NONE'}
[navigation_unit_AI-6] [INFO] [1775609732.001519064] [navigation_unit_AI]: Raw LLM Response: model='qwen2:0.5b' created_at='2026-04-08T00:55:31.996869:
[navigation_unit_AI-6] [INFO] [1775609732.002321316] [navigation_unit_AI]: LLM Response: NAV_DOCK
[navigation_unit_AI-6] [INFO] [1775609732.003641331] [navigation_unit_AI]: query_ollama: latency=0.223s
[navigation_unit_AI-6] [INFO] [1775609732.004168077] [navigation_unit_AI]: Ollama decision: NAV_DOCK
[navigation_unit_AI-6] [INFO] [1775609732.004591201] [navigation_unit_AI]: AI Decision: NAV_DOCK
[beacon_sensor_AI-3] [INFO] [1775609732.049352532] [beacon_sensor_AI]: Parsed beacon_part: Beacon=e4_87_91_3d_1e_d7, signal_part: SignalStrength=-100.
[beacon_sensor_AI-3] [INFO] [1775609732.049697164] [beacon_sensor_AI]: Scan counter: 4, received beacon: e4_87_91_3d_1e_d7 with RSSI: 100
```

Figure 300: Navigation\_unit\_AI console output

```
[navigation_unit_AI-6]
[docking_layer_AI-9] [WARN] [1775609733.913939320] [docking_layer_AI]: FALLBACK: Using rule-based docking logic.
[docking_layer_AI-9] [INFO] [1775609734.016825986] [docking_layer_AI]: Querying Ollama for docking decision: Robot needs to dock at charging station
[captain-1] [INFO] [1775609734.018069514] [captain]: {'0': 'NONE', '1': 'DOCK', '2': 'NONE', '3': 'NONE'}
[docking_layer_AI-9] [INFO] [1775609734.021122390] [docking_layer_AI]: ai_docking_query_error: latency=0.003s
```

Figure 31: Docking\_layer\_AI console output

In the above snippet of logs, the ai beacon sensor node had determined based on the signal strengths of surrounding beacons that the current closest beacon was Canal. The ai navigation unit then was queried and recognized that the current location matched the destination and decided to dock, from there the docking layer attempted to query to dock and when failed fell back to the original docking functionality to successfully dock the robot.

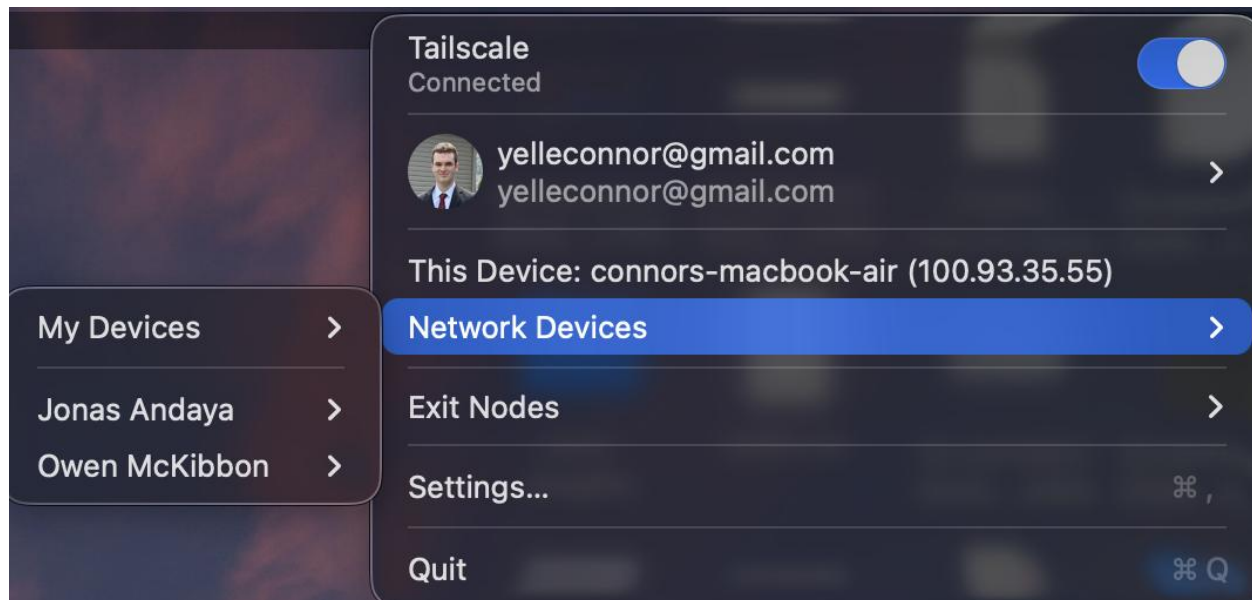
While these nodes were able to complete a trip together, other nodes functioned best on their own as they often interrupted other nodes. For instance, the ai LiDAR sensor node was overloaded with data too often to provide a meaningful response and took too long to generate a response. On the other hand, the ai avoidance layer was able to successfully query and perform avoidance or responsive maneuvers to obstacles. However, avoidance took precedence over other nodes and often interrupted their execution.

## 10.9 Connecting to the Physical Robot

By Connor Yelle

The team faced several challenges when attempting to SSH into the physical robot. The primary issue was that Carleton’s network assigns a new IP address daily, making it impossible to rely on a fixed address for remote access. To address this, the previous year’s group implemented a cloud-based Firebase database that was intended to store and update the robot’s IP address each day. However, when the team attempted to use this system, the database entries were not updated reliably, causing remote access to fail and halting development progress.

To resolve the unreliable SSH access, the team adopted Tailscale, a VPN solution that creates a secure private network between devices. Tailscale assigns each device with a persistent virtual IP address and hostname, allowing the robot to be accessed consistently regardless of changes in the underlying campus network. By installing the Tailscale client on both the robot and the group’s laptops, the team was able to establish stable SSH connections directly using the assigned Tailscale IP address, eliminating the need for the Firebase-based IP lookup.



*Figure 32: Tailscale Client running on a personal computer*

This solution greatly improved development efficiency, as team members could reliably connect to the robot from different locations without additional setup. However, one limitation encountered was that the Tailscale free tier only supports a maximum of three devices within a single tailnet. This constraint required careful coordination among team members regarding who could access the robot at any given time. In practice, team members connected via Tailscale could retrieve the robot’s current local IP address using commands such as ‘ip a’ and share it with others, but this introduced additional coordination overhead.

Overall, this experience highlighted the importance of selecting reliable and scalable infrastructure solutions. While the Firebase-based approach depended on consistent updates that

were not guaranteed, Tailscale provided a more robust and decentralized alternative that reduced system fragility and improved accessibility.

*Table 8: Comparison of Non-Functional Requirements for Tailscale vs. Firebase*

Non-Functional Requirement	Firebase Method	Tailscale Method
Scalability	High	Medium (Maximum of 3 devices per tailnet in the free tier)
Security	Medium (if someone finds out the posted IP on firebase, they can SSH into the robot)	High (robot cannot be entered by unauthorized parties)
Reliability	Low (fails often)	High (very rarely fails)
Maintainability	Low (a lot of maintenance needed to keep the system working)	High (fully automatic once initially setup)
Availability	Medium	High

## 10.10 Docker Usage

By Connor Yelle and Tommy Phang

A major limitation the team experienced during development was the requirement to build and test AI nodes within a Linux Ubuntu environment. This posed a challenge, as none of the team members had prior experience with Linux, and every member’s personal machines primarily ran Windows or macOS. Setting up a compatible development environment on each machine would have introduced significant overhead and potential inconsistencies.

To address this, the team adopted Docker to create a standardized Ubuntu-based development environment that could run on any host machine. By containerizing the ROS 2 workspace along with all required dependencies, this ensured that every team member worked within an identical environment, regardless of their operating system. This eliminated the need for manual dependency installation and reduced configuration-related issues. Additionally, this approach improves the long-term maintainability of the project, as future teams can quickly replicate the environment using the provided Docker configuration.

From a technical perspective, Docker images were used to define the complete software environment that the robot runs in, including ROS2 libraries, and system dependencies. The team

used Docker containers to provide isolated runtime instances for the development and testing of the robot.

The team also used Docker within the CI/CD pipeline to automatically build the project and execute tests on pull requests and commits. The team runs simulation-based tests to validate functionality before integrating the code with the main branch. This eliminates discrepancies between local and CI environments and increased the team's confidence that the system would behave correctly when deployed to the physical robot. This configuration emphasized the fast iteration philosophy while working on the robot.

## 10.11 Implementation Challenges

By Owen McKibbin

During the implementation process the team faced many challenges that limited or delayed the progress that needed to be overcome. This subsection outlines examples of such challenges faced for each of the above implementation processes.

### 10.11.1 Debugging the previous year's version of the codebase

By Connor Yelle

The codebase inherited from the previous year contained several issues that made it difficult to run the simulation reliably in Gazebo. When the team initially attempted to run the previous year's version of the system, the robot's behavior would differ from the intended logic. The project cloned directly from the GitHub repository contained a combination of syntax and logic errors that prevented the navigation and docking subsystems from functioning correctly. These problems had to be resolved before the team could attempt to integrate the LLM queries into different parts of the system.

One of the first major issues involved in this process is docking behavior. In the implementation of last year, the robot would never successfully dock at the end of its route. Instead, the robot would pass its starting beacon and continue towards the next navigation beacon. As soon as the navigation unit detected that the nearest beacon was the destination beacon, the system immediately issued a dock command. This caused the robot to attempt docking while still at a significant distance from the docking station. The servo dock request failed every time, preventing the robot from completing its trip properly and causing the robot to continue driving past the dock.

```
elif action == 'DOCK':
 if self.can_send_goal:
 self.can_send_goal = False
```

*Figure 33: Faulty Docking code snippet*

This part of the logic shown in Figure 11 above behaves as expected. This first dock command is sent, and the flag correctly prevents duplicate dock requests. However, the issue was that `can_send_goal` is never reset to `True` anywhere else in the code. As a result, when the system attempted to send a second dock command when it reaches the dock, the condition `if self.can_send_goal:` is never satisfied again. This causes a dock command to never be sent and the robot to just drive past the dock. This error causes the docking logic to effectively shut down permanently after the first attempt.

```
elif action == 'DOCK':

 if self.current_dock_state:
 self.can_send_goal = True
 break

 if self.can_send_goal:
 self.get_logger().info("Sending dock goal")
 self.can_send_goal = False
 self.dock_goal_future = self.docking_client.send_goal_async(
 self.dock_msg,
 feedback_callback=self.feedback_callback
)
 self.dock_goal_future.add_done_callback(self.dock_goal_callback)
 break
```

*Figure 345: Fixed Docking code snippet*

To resolve this issue, the team added logic to reset `can_send_goal` once the robot was actually in the dock radius. This change ensures that `can_send_goal` is only reset to `True` when the robot is in the docking motion. This fix resolved the intended behavior, allowing the docking layer to properly manage multiple docking events throughout the robot's route.

### 10.11.2 Integrating AI Nodes

By Owen McKibbin

Integrating the AI nodes together to work cooperatively posed many challenges in timing, responsibility, and resource use. The pi must spend a significant amount of processing power querying LLMs. Within that time, other processes must either wait or continue before getting a response. Some nodes depended on each other for topic information to make decisions and were thus made to wait for responses to be able to make their own queries and decisions. Certain nodes such as the ai lidar sensor, spent so much time querying, and so often that they interfered with the other ai nodes. A strategy for mitigating this was to set a fallback timer to ensure responses are timely enough for other nodes and rely on the original implementation. It was found that the most problematic nodes were those which spent too much time processing to allow other nodes to operate.

## 11.0 Verification and Validation

### 11.1 Continuous Integration

By Umniyah Mohammed

To ensure that the unit tests developed for the robot remain reliable and automatically validated throughout the project, a continuous integration (CI) pipeline was created using GitHub Actions. The purpose of this CI pipeline is to automatically run the test suite on every commit, branch update, and pull request, ensuring that no changes are merged into the main system unless they pass the required unit tests.

A key challenge encountered during CI setup was that the project relies on ROS 2 packages such as `rcipy`, `sensor_msgs`, and `std_msgs`, which are not available on GitHub's default Ubuntu runners. To overcome this limitation, a complete ROS-free mocking layer was implemented inside the unit test environment. This mocking system simulates the behavior of ROS 2 nodes, publishers, subscriptions, and message types closely enough for the sensor unit tests (particularly the LiDAR sensor tests) to execute without requiring a physical robot, ROS installation, or Gazebo.

The CI setup involved several major components:

- **GitHub Actions Workflow**
  - A new workflow (`unit-tests.yml`) was added to `.github/workflows/`. This workflow installs Python, installs test dependencies, and automatically runs the test suite on pushes, pull requests, and manual triggers.

- **Mocking the ROS Environment**

A custom `confest.py` file was created to simulate ROS functionality. This included:

- Mock implementations of `rcipy.init()`, `rcipy.shutdown()`, and `rcipy.node.Node`
- Mock publishers and subscriptions capable of capturing published messages
- Mock message types such as `LaserScan` and `String`
- Support for ROS-style keyword arguments such as `qos_profile`

These mocks allowed the LiDAR sensor unit test to run realistically, even though no ROS environment is present in CI.

- **Refinement of Tests for Compatibility with CI**

Tests that depend on unmocked ROS features (e.g., bumper sensor tests requiring `irobot_create_msgs`) were excluded from CI execution until proper mocks are developed. This ensured that the pipeline remains stable while still providing automated verification of the robot's most critical early-stage sensor logic.

The result of this work is a robust, automated testing environment that functions independently of ROS and ensures repeatability across the team. This CI pipeline significantly reduces the risk of regression, supports safer collaboration, and provides immediate feedback when changes

break existing behavior. A GitHub status badge can now be included in the project's README.md to indicate whether the latest commit passes all unit tests, improving transparency and code quality for future contributors.

## 11.2 Robot Testing

By Owen McKibbin

To meet the requirement of downloading and using the model on the Raspberry Pi to enable model instructed movement, the team evaluated two models on the Pi. During testing, instead of starting with the standard 3x3 grid navigation task outline in section 7.1.2, the team began with a simple prompt formatted to produce a one-word response. Using this approach, the robot would be able to interpret the response from the LLM model to run a movement command.

Specifically, the team tested the Qwen3.0:0.6b and Qwen2.5:0.5b models, using the prompt mentioned in 7.1.3. This prompt allowed us to view the robot's ability to respond and follow the model's movement commands, and the integration of the model and robot. The results showed that the model produced appropriate responses, which were well read by the robot, and produced proper movement in the prompted direction.

## 11.3 Unit Testing

By Connor Yelle and Umniyah Mohammed

To ensure a smooth transition from stabilizing the robot and simulator to integrating AI-based decision-making into the system, the team implemented unit tests for critical nodes from the previous year's codebase, beginning with the sensor layer. The goal of this effort was to follow sound software engineering practices and establish a reliable baseline system before introducing additional complexity. Developing these unit tests encouraged a more modular design approach, making it easier to integrate new AI components without breaking existing system behavior. Additionally, the test suite acts as a safety net during refactoring and feature development, allowing the team to iterate on AI logic while preserving correctness in key nodes such as navigation and control.

Unit testing the legacy codebase also provided valuable insight into how data flows through the system and how information is transmitted across ROS topics. By explicitly defining each node's inputs, outputs, and edge cases within tests, the team was able to uncover hidden assumptions and formally document expected behaviour. This not only improved the reliability of the system but also created useful documentation for future teams working with the codebase.

The team adopted a bottom-up testing strategy, beginning with sensor nodes where data is first introduced into the system. This ensures that the raw data entering the pipeline is accurate, consistent, and well-understood before it is consumed by downstream components. Validating

data at its source reduces the risk of propagating incorrect or malformed data through the system, which could otherwise lead to compounding errors in higher-level decision-making layers.

```
def make_mock_scan():
 scan = LaserScan()
 scan.ranges = [10.0] * 360 #initialize all distances to 10.0, no object in sight
 for i in range(175, 185):
 scan.ranges[i] = 0.5 #object 0.5m in front of the robot within angles 175 - 185
 scan.angle_min = 0.0
 scan.angle_increment = math.pi / 180 #1 degree in radians
 return scan
```

*Figure 356: Mock Data creation for the LiDAR sensor unit test*

Figure 33 shows an example of mock data generation used for the LiDAR sensor unit test. Since physical sensors and simulation environments are not available within unit tests, mock messages were created to approximate realistic sensor inputs. A helper function was implemented to construct a `LaserScan` object representing an obstacle approximately 0.5 m in front of the robot. The angle limits and increments were chosen to be compatible with the node's processing logic and to approximate real sensor behavior, enabling meaningful validation of the sensor's computation functions.

## 11.4 System-Level Validation

By Ummiyah Mohammed

In addition to unit testing, the team implemented a system-level validation approach to evaluate how multiple ROS nodes, system layers, and LLM components behave when operating together. Rather than using traditional integration tests with pass/fail assertions, this approach relies on simulation-based execution and performance metric analysis to validate overall system behavior.

This validation is achieved using dedicated runtime analysis nodes, including `metric_analyzer.py` and `travel_analyzer.py`, which operate alongside the robot within the simulation environment. These nodes subscribe to multiple ROS topics, such as LiDAR data, navigation outputs, intersection detection, and docking status, allowing them to monitor how data flows through the system in real time. By observing interactions between components across different layers, the system can be evaluated as a whole rather than in isolation.

In addition, the team developed a CI pipeline (`pr-sim-metrics.yml`) that runs full simulations inside a Docker container for each pull request. During these runs, performance metrics are automatically collected and reported, including delivery time, battery usage, LLM response

latency, and fallback occurrences. These metrics are then compared against previous runs to identify regressions or improvements in system performance.

This approach provides a practical form of system-level validation by ensuring that changes to individual components do not negatively impact overall system behavior. It is particularly important for validating LLM-integrated functionality, where issues may arise from interactions between multiple nodes rather than isolated logic errors. While this method does not use formal integration test assertions, it enables continuous monitoring and evaluation of system performance in a realistic operational context.

## 12.0 Required Components and Facilities

By Connor Yelle and Umniyah Mohammed

The table below outlines the required components for the functionality of the robot and their respective costs. Each component is paired with a description of its purpose and relation to the robot's functionality.

*Table 9: List of required Components/Facilities, Purposes, and Estimated Costs*

Required Component/Facility	Purpose	Cost
iCreate3	The robot itself. It will be performing the mail delivery using the system designed by the team. Already Purchased in previous years	0\$
LiDAR Sensor	The main sensor that makes the robot detect walls. Crucial for standard robot operation. Already purchased in previous years.	0\$
Raspberry Pi 4b	The computer board that is used for the robot. Runs the system, and the AI. Already purchased in previous years	0\$
Beacon Sensors	The Bluetooth sensors placed at every intersection in the tunnels that help the robot orient itself. Already purchased in previous years.	0\$
Carleton Tunnels	The robot is designed to deliver mail inside of the tunnel system under Carleton. Tunnel access is completely free	0\$
5 pk CR2450 Lithium Batteries x2	The beacons are powered by lithium batteries for use in robot navigation experiments	22.80\$



## 13.0 Conclusion

By Connor Yelle

The goal of this project was to explore the use of LLM agents to control an iRobot for autonomous mail delivery within the Carleton tunnels. This included embedding an LLM directly into a Raspberry Pi and integrating it inside of the iCreate3 robot to enable AI decision making. The goal was to leverage the LLM for a wide range of tasks within the robot's data pipeline, including interpreting sensor inputs and making critical movement decisions. By incorporating LLM agents into the already modular ROS2 architecture, the project aimed to evaluate whether this new decision-making system was feasible in any way possible.

By leveraging Gazebo for simulation, Ollama for local LLM deployment, and ROS 2 for system integration, the group was able to develop and evaluate primitive LLM-driven nodes within an agile and iterative development environment. This setup enabled rapid prototyping and testing of different model configurations, prompt designs, and integration strategies. Furthermore, the consistency between the simulation environment and the physical system allowed for efficient transitions from testing to deployment, supporting systematic evaluation of LLM-based behavior.

While it is possible to integrate LLMs into the Raspberry Pi for decision-making, the results of this project show that current hardware limitations significantly impact their performance. Limited processing power and slow response times lead to increased latency and reduced reliability during the robot's mail delivery tasks. These challenges are further shown when attempting to run multiple LLM-driven nodes in parallel, as the constrained resources of the Raspberry Pi make task scheduling and concurrency management difficult. As a result, coordinating multiple LLM-based nodes introduces significant overhead and unpredictability in system behavior. Overall, further improvements in embedded hardware and model optimization are necessary before this approach can become a viable solution for autonomous robotic control.

## 14.0 Recommendations

By Tommy Phang, Connor Yelle, Owen McKibbon, Umniyah Mohammed, and Jonas Andaya

For any subsequent additions to this project, the team suggest that the modularity of ROS is utilized to its fullest extent. The publishing and subscription endpoints allow for the interior content of nodes to be used as an open box for an LLM to use. A further exploration of AI use in this project could be the formatting and interpretation of topic data. For example, instead of formatting output from LLMs manually for publishing a topic or scraping subscription data to feed a model, there are opportunities to use LLMs to format the data for one another. This could allow for the LLMs to interact in plain English. Another area of improvement could be tool use with better models if the hardware is upgraded with a Raspberry Pi 5 and hat. In terms of improvements for LLM functionality, the avoidance layer showed the most promise for its dynamic use. Further utilization of the LLM in the avoidance layer could allow for the robot to respond more dynamically to obstacles in its path. Another area for improvement or further research could be providing the model with a graph-based map or picture of the tunnels instead of a chart of most optimal directions to take. An LLM could choose the best direction based on nodes and edges of the graph and pathfinding instead of being told where to go. While it is true that the tunnels at Carleton University do not change often, part of the goal is to observe how well an LLM can function dynamically in these environments. This could be another method for testing model performance in its ability to navigate complex spaces. Finally the use of Github actions and workflow tools is strongly encouraged to follow an agile methodology and measure progress in project versions.

## 15.0 Project Reflections

By Tommy Phang, Connor Yelle, Owen McKibbon, Umniyah Mohammed, and Jonas Andaya

The team faced many challenges over the past two terms. One big challenge the team faced was trying to get last year's code up and running. The instructions left behind by last year's group had some errors present, which caused repeated delays week by week. These setbacks made it difficult for the team to follow the original project timeline. Debugging the system often requires deep dives into the topics between the nodes to learn the dataflow of the system. These setbacks initially slowed the team's progress, but once the team got the robot working, the team started meeting the milestones in the new project timeline and made steady progress towards achieving the original project objectives.

The numerous challenges that the team faced throughout the project allowed for some valuable opportunities for the team to collaborate. The team utilized GitHub to manage the code, allowing team members to review each other's work before altering the main version. This gave us valuable insight and experience to work as a team in a collaborative code environment with an agile methodology. The improvement and maintenance of legacy code improved the team's

skills in refactoring and testing. The team's collective experience in SYSC4806 gave us the building blocks to implement CI/CD pipelines and utilize workflows for testing.

If the team undertook this project again, the team wouldn't make the same initial decision about the tools and methods. The team would find an alternative to connect to the robot instead of relying on Tailscale, so the team is not restricted to three members only having access to the robot. This restriction imposed by Tailscale made it hard for all members to access the robot making some members rely on other members to be there with them to give them access to the robot.

## 16.0 References

- [1] S. Macenski et al., "Robot Operating System 2: Design, architecture, and uses in the wild," science.org, 2022 <https://www.science.org/doi/epdf/10.1126/scirobotics.abm6074> (accessed Oct. 1, 2025)
- [2] F. S. Marcondes et al., "Using Ollama," SpringerBriefs in Computer Science, pp. 23–35, 2025, doi: [https://doi.org/10.1007/978-3-031-76631-2\\_3](https://doi.org/10.1007/978-3-031-76631-2_3). (accessed Oct. 8, 2025)
- [3] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566), Sendai, Japan, 2004, pp. 2149-2154 vol.3, (accessed Oct. 8, 2025)
- [4] T. S. Taylor, Introduction to Laser Science and Engineering. CRC Press, 2019, pp. 239-240, (accessed Oct. 9, 2025).
- [5] R. A. Brooks, A Robust Layered Control System For A Mobile Robot, IEEE, 1986, pp. 14-23, (accessed Jan. 21, 2026)
- [6] D. Cengu et al., "*Mail Delivery Robot in Carleton Tunnels*," Capstone Project, University of Ottawa, Ottawa, ON, Canada, 2025.

## Appendix A: Test Data

[GIN]	2025/10/01 - 20:17:25	200	33.90205981s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:18:26	500	1m0s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:18:57	200	31.494387153s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:19:15	500	17.772581109s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:20:25	500	1m0s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:21:24	200	59.00035088s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:22:16	200	52.672553414s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:22:50	200	33.981436072s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:23:34	200	43.79380651s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:24:20	200	45.548441586s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:25:20	500	1m0s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:26:20	500	1m0s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:26:57	200	36.976111605s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:27:27	200	30.267523216s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:28:20	200	52.564717554s	127.0.0.1	POST	"/api/generate"

Figure A.367: Qwen 3.0 response times

[GIN]	2025/10/01 - 20:56:52	200	4.193984198s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:56:53	200	763.101204ms	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:02	200	8.444220096s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:03	200	605.446691ms	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:04	200	1.040949417s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:07	200	1.371202894s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:10	200	811.418978ms	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:13	200	1.030716211s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:16	200	583.893471ms	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:24	200	6.481548895s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:29	200	4.133978523s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:29	200	582.675645ms	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:59	200	29.068850954s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:58:01	200	2.080365011s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:58:02	200	1.225444077s	127.0.0.1	POST	"/api/generate"

Figure A.378: Qwen 2.5:0.5b

## Appendix B: Use Case Tables

Table 10: Use Case Description Table of Simulating in Gazebo

<b>Use Case Name</b>	Simulate in Gazebo
<b>Brief Description</b>	The user simulates the robots' movements and travel path in Gazebo for testing.
<b>Primary Actor</b>	Gazebo
<b>Secondary Actor</b>	N/A
<b>Precondition</b>	The system is running, the LLM server is running, and the simulation is turned on.
<b>Dependency</b>	N/A
<b>Basic Flow</b>	<b>Steps:</b>      1. The user submits a mail delivery request to the console.    2. The user watches the simulation in Gazebo for successful robot decisions.      <b>Postcondition:</b>   Gazebo successfully simulated the path the robot would take in a real-life scenario.
<b>Specific Alternate Flows</b>	RFS Basic Flow 1   <b>Steps:</b>      1. If the system is not running, then send an error to the user explaining what happened.

	<b>Postcondition:</b>  The user is informed that a specific part of the system is not running.
--	------------------------------------------------------------------------------------------------------

Table 11: Use Case Description Table of Viewing Historical Performance

<b>Use Case Name</b>	View Historical Performance
<b>Brief Description</b>	The user utilizes the dashboard to view historical versus current performance to analyze how well the LLM is doing in the current implementation.
<b>Primary Actor</b>	User
<b>Secondary Actor</b>	N/A
<b>Precondition</b>	A pull request was sent to the GitHub page. The system is running, and the dashboard is active.
<b>Dependency</b>	N/A
<b>Basic Flow</b>	<b>Steps:</b>       1. The user submits a command to the console telling the robot to perform a trip.     2. The dashboard updates the tracked metrics based on the data it receives from the sensors and previous version of the metrics.     3. The user presses the metrics tab and analyses the new metrics       <b>Postcondition:</b> The dashboard displays the tracked metrics with either red or green bars based on if the metric improved or got worse.
<b>Specific Alternate Flows</b>	N/A

Table 12: Use Case Description Table of Monitoring the System Dashboard

<b>Use Case Name</b>	Monitor System Dashboard
<b>Brief Description</b>	The user utilizes the dashboard to view current robot metrics such as battery life, or the Jira Kanban board for project objectives
<b>Primary Actor</b>	User
<b>Secondary Actor</b>	N/A
<b>Precondition</b>	The dashboard is active and running.
<b>Dependency</b>	N/A
<b>Basic Flow</b>	<b>Steps:</b>       1. The user opens the URL that links to the dashboard web server.     2. The user analyses the Kanban board to see which tasks need to be performed in this sprint.       <b>Postcondition:</b>   The dashboard displays all tasks useful to the user during the current sprint.
<b>Specific Alternate Flows</b>	RFS Basic Flow 2   <b>Steps:</b>       1. The user monitors the various metrics tracked by the robot, like battery life or current trip.       <b>Postcondition:</b>

	The dashboard displays all metrics relevant to the usage of the robot.
--	------------------------------------------------------------------------

Table 13: Use Case Description Table of Executing the Navigation Plan

<b>Use Case Name</b>	Execute Navigation Plan
<b>Brief Description</b>	The ROS system uses the command sent to the system, create and execute a navigation plan to reach the targeted destination.
<b>Primary Actor</b>	ROS
<b>Secondary Actor</b>	N/A
<b>Precondition</b>	The system is running, the robot is turned on, and the Bluetooth beacons are active and installed.
<b>Dependency</b>	INCLUDE USE CASE Avoid Obstacles
<b>Basic Flow</b>	<b>Steps:</b>       1. The Robot Controller obtains a command from the user, parses it, and sends the command into the ROS node     2. The ROS node reads the parsed command to create a navigation plan with the goal of reaching the destination in the most efficient way possible.     3. Once completed, ROS executes the created navigation plan by sending commands throughout the various nodes that communicate with the hardware inside the robot.       <b>Postcondition:</b>

	The robot is in transit to its desired destination using the created navigation plan.
<b>Specific Alternate Flows</b>	RFS Basic Flow 1   <b>Steps:</b>       1. If the user submits an invalid command, the system promptly sends an error explaining what happened.       <b>Postcondition:</b>   ROS is idle waiting for the user to submit a command.

Table 14: Use Case Description Table of Navigating the Tunnel Network

<b>Use Case Name</b>	Navigate Tunnel Network
<b>Brief Description</b>	The robot uses its sensors and Bluetooth beacons to navigate the tunnel network to reach its destination.
<b>Primary Actor</b>	Robot Controller
<b>Secondary Actor</b>	ROS
<b>Precondition</b>	The system is running, the robot is turned on, and the beacons are installed throughout the tunnels.
<b>Dependency</b>	INCLUDE USE CASE Execute Navigation Plan
<b>Basic Flow</b>	<b>Steps:</b>       1. The robot controller receives a command from the user and checks if it has enough battery to perform the trip     2. The robot controller creates a navigation plan based on the current robot location and the proposed destination     3. If the robot has enough battery, the robot starts navigating the tunnels using the navigation plan.       <b>Postcondition:</b>   The robot controller is reading sensor data and using the LLM agent to process data to

	make correct movements so it can reach its destination.
<b>Specific Alternate Flows</b>	N/A

Table 15: Use Case Description Table of Docking for Charging

<b>Use Case Name</b>	Dock for Charging
<b>Brief Description</b>	The robot docks itself once it notices it is low on battery
<b>Primary Actor</b>	Charging Station
<b>Secondary Actor</b>	LLM Agent
<b>Precondition</b>	The robot has low charge
<b>Dependency</b>	INCLUDE Use Case Assist with Sensor Data Processing
<b>Basic Flow</b>	<b>Steps:</b>       1. The user submits a command to the console telling the robot to perform a mail delivery     2. The LLM agent notices that the robot does not have enough charge to perform the trip, so it changes its current destination to the closest charging station.     3. The robot locates the charging station and docks itself to charge its battery       <b>Postcondition:</b>   The robot changes states to charging and is positioned on the charging dock.
<b>Specific Alternate Flows</b>	RFS Basic Flow 1   <b>Steps:</b>

	1. The robot finishes the mail delivery and has no new trips to perform.     2. The robot locates and changes its location to the nearest charging station.     3. The robot arrives at the charging station and docks itself to charge its battery       <b>Postcondition:</b>   The robot changes states to charging and is positioned on the charging dock.
--	----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------

Table 16: Use Case Description Table of Avoiding Obstacles

<b>Use Case Name</b>	Avoid Obstacles
<b>Brief Description</b>	The robot controller utilizes the LLM agent to read sensor data and make correct movements to avoid various obstacles.
<b>Primary Actor</b>	Robot Controller
<b>Secondary Actor</b>	LLM Agent
<b>Precondition</b>	The robot is in transit to its desired destination
<b>Dependency</b>	INCLUDE USE CASE Assist with Sensor Data Processing
<b>Basic Flow</b>	<b>Steps:</b>       1. While is robot is in transit to its destination, the LiDAR sensor finds an obstacle in the robot's path.     2. The sensor data is sent to the robot controller, where it forwards the data to the LLM agent.     3. The LLM agent analyses the data and decides what to do to avoid the obstacle and continue the desired path.

	4. The LLM decision is sent to the Robot Controller, where it sends commands to the various ROS nodes to make the robot avoid the obstacle.   <b>Postcondition:</b>   The robot successfully avoids the obstacle and continues its journey to the destination.
<b>Specific Alternate Flows</b>	RFS Basic Flow 1   <b>Steps:</b>       1. The LiDAR sensor finds a wall that the robot is about to run into.     2. The sensor data is sent to the robot controller, where it forwards the data to the LLM agent.     3. The LLM agent analyses the data and decides what to do to avoid the wall. It uses the navigation plan to know which way to turn     4. The LLM decision is sent to the Robot Controller, where it sends commands to the various ROS nodes to make the robot turn to avoid the wall.       <b>Postcondition:</b>   The robot successfully performs a turn to avoid the wall and continues its journey to the destination.

Table 17: Use Case Description Table of Update Dashboard on pull request

<b>Use Case Name</b>	Update Dashboard on pull request
<b>Brief Description</b>	Upon pull request, the dashboard will update with the latest simulation results for metrics.
<b>Primary Actor</b>	Gazebo Simulator
<b>Secondary Actor</b>	N/A
<b>Precondition</b>	A pull request has been initiated
<b>Dependency</b>	INCLUDE Use Case Monitor System Dashboard
<b>Basic Flow</b>	<b>Steps:</b>       1. Simulation is run in gazebo environment.     2. User submits pull request to repository.     3. Dashboard is updated with simulation data.       <b>Postcondition:</b>   The Dashboard contains the latest simulation results and metrics for the robot.
<b>Specific Alternate Flows</b>	N/A

Table 18: Use Case Description Table of Assist with sensor data processing

<b>Use Case Name</b>	Assist with sensor data processing
<b>Brief Description</b>	The LLM agent assists in processing data from various sensors into information that can be used for navigation decision making.
<b>Primary Actor</b>	LLM Agent
<b>Secondary Actor</b>	N/A
<b>Precondition</b>	Sensor Data is Received from robot's sensors
<b>Dependency</b>	N/A
<b>Basic Flow</b>	<b>Steps:</b>       1. Sensor data is received from LiDAR sensor/collision detector.     2. Data is processed by LLM agent.     3. Processed data is used for navigation decisions.       <b>Postcondition:</b>   The robot makes navigation decisions based on processed sensor data.
<b>Specific Alternate Flows</b>	N/A

## Appendix C: Create 3 Hardware Setup

1. Connect the micro-USB into the LiDAR's USB adapter board and to the Raspberry Pi 4B.
2. Connect the USB-C cable between the USB-C port in the robot's cargo bay to the Raspberry Pi 4B's USB power input port.
3. Ensure the LiDAR Serial Cable is connected to the USB adapter board.
4. Ensure the mail holder and camera bracket are firmly in place, tighten with screwdriver if needed.

## Appendix D: Create 3 Software Setup

For additional guidance or situations not covered here, refer to the Create 3 and ROS documentation:

[https://iroboteducation.github.io/create3\\_docs/](https://iroboteducation.github.io/create3_docs/)

<https://docs.ros.org/en/humble/index.html>

### Create 3 Firmware Setup

1. Download version H.2.4 of the Create 3 firmware for Humble:  
[https://iroboteducation.github.io/create3\\_docs/releases/overview/](https://iroboteducation.github.io/create3_docs/releases/overview/).
2. Turn on the robot by placing it on a charging dock that is plugged in.
3. Press and hold both side buttons until the lights around the middle button turn blue.
4. Connect to the Create-[xxx] Wi-Fi network on your device.
5. Navigate to 192.168.10.1 to access the Create 3 web interface.
6. Navigate to the Update tab and follow the steps to upload the downloaded firmware.
7. Wait until the robot chimes and then navigate to the Application → Configuration tab.
8. Ensure the RMW\_IMPLEMENTATION dropdown is set to `rmw_cyclonedds_cpp`.
9. Restart the application if the RMW implementation is changed.
10. Navigate to Beta Features → NTP sources tab and add  
`server 192.168.186.3 iburst`  
so the robot can receive NTP info from the Raspberry Pi.
11. Reboot the robot and disconnect from the Wi-Fi network.

## Installing the Ubuntu Image

\*Note that this requires a device with a microSD slot.

1. Download the Raspberry Pi imager: <https://www.raspberrypi.com/software/>
2. Open the Raspberry Pi imager software.
3. Select Ubuntu Server 22.04x LTS (64-bit).
4. Select the gear icon and ensure SSH is enabled.
5. Set a new username and password.
6. Write the image to the Raspberry Pi 4B's microSD card.

## Setting up Wi-Fi

\*Note that this requires a device with a microSD slot.

\*The wireless LAN setup included will not work with the Create 3 and Carleton network.

1. Eject and reinsert the microSD card into your device to access the system-boot partition.
2. Edit the "config.txt" file to add `dtoverlay=dwc2,dr_mode=peripheral` at the end.
3. Edit the "cmdline.txt" file to add `modules-load=dwc2,g_ether` after `rootwait`.
4. Edit the "network-config" file to add Wi-Fi information:

```
version: 2
ethernets:
 eth0:
 dhcp4: true
 optional: true
 usb0:
 dhcp4: false
 optional: false
 addresses: [192.168.186.3/24]
wifis:
 renderer: networkd
 wlan0:
 dhcp4: true
 optional: true
 access-points:
 "eduroam":
 auth:
 key-management: eap
 method: peap
 identity: <school wifi login username>
 password: <wifi password>
```

5. Remove the microSD card from the device and insert the microSD card into the Raspberry Pi.

## Creating an Account on the Image

\*An account should have been created when the image was first created in the Installing the Ubuntu Image steps.

Run the following commands in succession:

1. `sudo su - root`
2. `hostnamectl set-hostname`
3. `adduser <accountname>`
4. `su - <accountname>`

## Setting up the NTP Server

1. Go to [https://iroboteducation.github.io/create3\\_docs/setup/compute-ntp/](https://iroboteducation.github.io/create3_docs/setup/compute-ntp/) and follow the steps to configure an NTP server on the Raspberry Pi. Steps 6 and 7 were completed during firmware setup.
2. Run `timedatectl` and check if “System clock synchronized” has the value “yes”. If it does, move on to the locale configuration.
3. If not, run `sudo nano /etc/systemd/timesyncd.conf`
4. Add `NTP=ntp.ubuntu.com` and `FallbackNTP=0.us.pool.ntp.org` if not already configured.
5. Run `systemctl restart systemd-timesyncd.service`

## ROS2 Humble Installation

1. Install ROS Humble using this link as a guide:  
<https://docs.ros.org/en/humble/Installation.html>
2. Run `sudo apt install -y ros-humble-irobot-CREATE-3-msgs`
3. Run `sudo apt install -y build-essential python3-colcon-common-extensions python3-rosdep ros-humble-rmw-cyclonedds-cpp`
4. Run `sudo rosdep init`
5. Run `echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc`
6. Run `echo "export RMW_Implementation=rmw_cyclonedds_cpp" >> ~/.bashrc`
7. Reboot the robot by holding the power button for 10 seconds and then place it onto the charging dock.
8. Run `ros2 topic list` and ensure the Create 3 API topics are displayed.

## Creating a Workspace and Cloning the Necessary Code

1. Run `mkdir -p ~/cmdr_ws/src`
2. Run `cd ~/cmdr_ws/src`
3. Clone the repository: `git clone https://github.com/ConnorYelle/Carleton-Mail-Delivery-Robot.git`
4. Clone the LiDAR package: `git clone https://github.com/Slamtec/sllidar_ros2.git`
5. Install bluepy by running: `sudo pip install bluepy`
6. Install requests by running: `sudo pip install requests`
7. Run `cd ~/cmdr_ws`
8. Run `colcon build --symlink-install`

## Running the Code

1. Run `sudo -s`
2. Run `cd ~/cmdr_ws`
3. Run `source /opt/ros/humble/setup.bash` (This was set to run on boot but just in case)
4. Run `source install/setup.bash`
5. Run `ros2 launch mail-delivery-robot robot.launch.py use_sim:=false use_ai_sensors:=true`
6. In another terminal, publish a destination in the following format: `ros2 topic pub /destinations std_msgs/msg/String "data: 'Canal:Nicol'" --once`

\*If you encounter an error while running the code which states something like ‘lidar node has died’, or ‘SL\_RESULT\_OPERATION\_TIMEOUT’, try unplugging the lidar from the robot and plugging it back in.

## Tailscale Installation and Usage

### Laptop Installation

1. Go to: <https://tailscale.com/download>
2. Download the version for your OS
3. Sign in with Google / GitHub / Microsoft
4. You should now see your laptop listed in the Tailscale dashboard

### ICreate3 Installation

1. Plug the Raspberry Pi into the monitor for a visual interface.
2. Open the console and enter `ip a`
3. Locate the wlan0 network interface and look at the inet ip address. This is the ip address you will use to SSH into the robot, so that you can download Tailscale.

```

 valid_lft forever preferred_lft forever
4: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
 link/ether dc:a6:32:07:9b:a0 brd ff:ff:ff:ff:ff:ff
 inet 172.17.130.105/19 metric 600 brd 172.17.159.255 scope global dynamic wlan0
 valid_lft 6194sec preferred_lft 6194sec
 inet6 2620:22:4000:1202:1f3b:c5b7:9c91:10ea/128 scope global dynamic noprefi
xroute

```

4. Enter the console on your laptop and enter: `ssh <username>@<ip>`. You will be prompted to enter the password you decided to use while flashing the operating system onto the pi.
5. From the robot's console, enter: `curl -fsSL https://tailscale.com/install.sh | sh`. This will install Tailscale directly from the website using cURL.
6. Once installed, enter: `sudo tailscale up`. This will start tailscale.
7. You must now authenticate the robot to the 'tailnet' you created on your computer. You should have been prompted with an authentication URL:  
To authenticate, visit: <https://login.tailscale.com/a/xxxxxxxxx>
8. Open the link and log in with the same credentials as earlier.

To ensure the connection is confirmed, enter: `tailscale status` into the console on the robot. If your computer appears, everything is connected, and you should be able to connect to the robot using the new Tailscale IP.

## Appendix E: Gazebo Setup

\*To set up and run Gazebo you will need an Ubuntu 22.04x system.

### Installing ROS2 Humble

1. Install ROS Humble Desktop and Dev tools using this link as a guide:  
<https://docs.ros.org/en/humble/Installation.html>
2. Run `sudo apt install -y ros-humble-irobot-Create-msgs`
3. Run `sudo apt install -y build-essential python3-colcon-common-extensions python3-rosdep ros-humble-rmw-cyclonedds-cpp`
4. Run `sudo rosdep init`

5. Run `echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc`
6. Run `echo "export RMW_Implementation=rmw_cyclonedds_cpp">> ~/.bashrc`

## Installing Gazebo

1. Run `curl -sSL http://get.gazebosim.org | sh`
2. Try running `gazebo`
3. If that doesn't work follow the instructions from the following page:  
[https://classic.gazebosim.org/tutorials?tut=install\\_ubuntu](https://classic.gazebosim.org/tutorials?tut=install_ubuntu)
4. Follow the instructions on this page to install Ignition Fortress:  
[https://gazebosim.org/docs/fortress/install\\_ubuntu/](https://gazebosim.org/docs/fortress/install_ubuntu/)
5. Run `sudo apt-get install ros-humble-ros-gz`

## Creating a Workspace and Cloning the Necessary Code

1. Run `mkdir -p ~/testing_ws/src`
2. Run `cd ~/testing_ws/src`
3. Clone the repository: `git clone https://github.com/ConnorYelle/Carleton-Mail-Delivery-Robot.git`
4. Move `dashboard.py` from `external_files` into your home directory
5. Move the `demo_video.world` file to your `.gazebo/worlds/` directory
6. Move all the mac address folders and intersection marker folder to `.gazebo/models`.
7. Run `cd ~/testing_ws`
8. Run `colcon build --symlink-install`

## Robot Bringup Process

1. Run `cd ~/testing_ws`
2. Run `source /opt/ros/humble/setup.bash`
3. Run `source install/setup.bash`
4. Run `ros2 launch irobot_create_gazebo_bringup create3_gazebo.launch.py`

## Running Code in Gazebo

Code is run as you would normally for the robot, just make sure that Gazebo is running and the robot is correctly behaving. There are some bugs with the system though most are fixed by closing Gazebo and running the bring up process once more.

To run the metric analyzer, make sure to use the launch argument as follows: `ros2 launch mail-delivery-robot robot.launch.py enable_metrics:=true`

Following the end of your delivery simulation, shut down the ROS2 node and run your `dashboard.py` as follows from the home directory: `python3 dashboard.py`

## Appendix F: Experiment methodology

### 7.3 Experiment methodology

By Tommy Phang

- The experiment is run using Google Colab
- Load the map data; this will allow the LLMs to access map data just like on the robot.
- Run 100 trials for each LLM with the following prompt:

"""

You are a robot navigation system.

Current Location: {current\_node}

Destination: {target\_dest}

{context}

INSTRUCTION: Based on the Navigation Rules above, what is the single command to go from {current\_node} to {target\_dest}?

Reply ONLY with the command (e.g., NAV\_LEFT). Do not explain.

"""

- {current\_node} = the robots starting position, for example “Loeb1”
- {target\_dest} = The place the robot wants to reach, for example “Steacie”
- {context} = Is the row provided by the map.csv made developed from previous groups working on the project. This allows the LLM to navigate to the correct place by following instructions presented in the map.csv.

- In each trial, the “Global” map is hidden from the LLM, and is only provided with a context string, which gives it relevant rules corresponding to its current location. This is like simulating the local lookup table housed in the robot.
- Each LLM performance is measured by its accuracy of the models output against the map.csv file. Since the tunnels at Carleton do not change, map.csv provides a reliable baseline for comparison.

Accuracy = Number of Correct Answers / Number of Runs

- A model's decision is counted as “correct” if the output matches what is in the map.csv.

## 7.4 Effects of Prompt Variation on Navigation AI

By Tommy Phang

### 1. Experimental setup

- Model used: qwen2.5:0.5b via ollama
- Number of trials: 50
- Task: Predict the correct navigation command (e.g., NAV\_LEFT, NAV\_FORWARD)
- given:
  - o A starting node
  - o A destination node
  - o A set of navigation rules (context)

### 2. Procedure

For each of the 50 trials:

Randomly select:

- o A starting location (start\_node)
- o A destination (dest\_node)

Generate:

- o Context describing navigation rules (context\_str)
- o Five prompt variants using identical inputs

For each prompt:

- o Query the model using `ollama.generate()`
- o Check correctness: If the ground truth appears in the response, then it is a success

## 7.4 Effects of Prompt Variation on Avoidance Layer AI

By Tommy Phang

### Methodology

1. Experimental setup
  - Model used: qwen2.5:0.5b via ollama
  - Number of trials: 50
  - Task: Select the safest movement command: BACK, LEFT, RIGHT, or GO (move forward in the current direction its facing)
  - Input: Simulated LiDAR readings (distances in FRONT, LEFT, RIGHT directions)

Sample LiDAR summary (`lidar_summary`) given to the LLM:

### 2. Procedure

For each of the 50 trials:

Generate random LiDAR distances: FRONT, LEFT, RIGHT  $\in [0.5, 10.0]$

Calculation: Closest direction (minimum distance) and most open direction (maximum distance)

Define ground truth:

- If FRONT is open, then GO
- Otherwise, move toward the most open direction (LEFT, RIGHT, or BACK)

Create a LiDAR summary string that consists of:

- Distances
- Closest direction
- Most open direction

Generate 5 prompt variants using the same scenario

For each prompt:

- o Query the model using `ollama.generate()`
- o Check correctness: If the ground truth appears in the response, then it is a success

## 7.4 Effects of Prompt Variation on Beacon Sensor AI

By Tommy Phang

### 1. Experimental setup

- Model used: qwen2.5:0.5b via ollama
- Number of trials: 50
- Task: Identify the beacon with the strongest signal
- Input: Simulated RSSI readings for multiple beacons

### 2. Procedure

For each of the 50 trials:

Randomly select 3 beacons from: UC, ResComms, CTTC, Nicol, Canal

Generate RSSI readings: each beacon receives 3 random values in the range [-95, -40]

Then, calculate the average RSSI for each beacon

Define the ground truth: the ground truth beacon = beacon with the highest (least negative) average RSSI; values closer to 0 are stronger, and closer to -100 are weaker.

Generate 5 prompt variants using the same scenario

For each prompt:

- o Query the model using `ollama.generate()`

- o Check correctness: If the ground truth appears in the response, then it is a success

#### 7.4.4 Effects of Prompt Variation on Lidar Sensor AI

By Tommy Phang

1. Experimental setup
  - Model used: qwen2.5:0.5b via ollama
  - Number of trials: 30
  - Task: Calculate the minimum distances for predefined LiDAR regions: front, left, right, wf\_dist (wall follow distance)
  - Input: Simulated LiDAR scan data in "angle:distance" format

The LiDAR space is divided into angular regions:

- Front:  $\text{angle} \leq 10^\circ$  or  $\geq 350^\circ$
- Right:  $45^\circ \leq \text{angle} < 135^\circ$
- Left:  $225^\circ < \text{angle} \leq 315^\circ$
- Wall-follow (wf\_dist):  $-30^\circ$  to  $30^\circ$
- Default distance: 10.0 (used if no points exist in a region)

#### 2. Prompt styles

For each trial, five prompt types are generated:

Prompt 1: Instructional Prompt has detailed and structured instructions

```
f"""
```

```
Analyze these Lidar readings (format \"angle:distance\").
```

```
Data: [{data_str}]
```

Task: Find the minimum distance in the following sectors.

If a sector has no data, use `{config['DEFAULT_DIST']}`.

#### Procedure

For each of the 50 trials:

Generate 15 random LiDAR points:

- Angle  $\in [0^\circ, 360^\circ]$
- Distance  $\in [0.5, 5.0]$

Construct a string representation of scan data (`data_str`)

Define the ground truth: Initialize all regions' distance to 10.0, then update each region with the minimum distance of points falling within its angular range

For each prompt:

- o Query the model using `ollama.generate()`
- o Check correctness: If at least one correct region value (front, right, left) appears in the response, then it is a success

## **Appendix G: GitHub Repository**

<https://github.com/ConnorYelle/Carleton-Mail-Delivery-Robot>

## **Appendix H: Original Project Proposal**

Mail-Delivery Robot in Carleton Tunnels using LLM Agents

Project Proposal

Tommy Phang - 101211316

Owen McKibbon - 101269794

Connor Yelle - 101260641

Jonas Andaya - 101224271

Umniah Mohammed - 101158792

Dr. Babak Esfandiari

Friday, October 17<sup>th</sup>, 2025

## Table of Contents

List of Figures .....	<b>Error! Bookmark not defined.</b>
List of Tables .....	<b>Error! Bookmark not defined.</b>
List of Abbreviations .....	8
Abstract .....	9
1.0 Introduction.....	10
1.1 Project Background.....	10
1.2 Project Motivation .....	10
2.0 The Engineering Project .....	13
2.1 Health and Safety .....	13
2.2 Engineering Professionalism .....	14
2.3 Project Management .....	14
2.3.1 Agile methodology.....	14
2.3.2 Proposed Gantt Chart.....	15
2.3.3 Complete timeline.....	16
2.3.4 Risks and Mitigation Strategies .....	18
2.4 Justification of Suitability for Degree Program .....	19
2.5 Individual Contributions .....	20
2.5.1 Project Contributions .....	20
2.5.2 Report Contributions.....	22
3.0 Requirements .....	23
3.1 Requirements Process .....	23
3.2 Function Requirements .....	24
3.2 Non-Functional Requirements .....	25
3.3 Use Case Diagram.....	27
3.4 Use Case Tables .....	28
4.0 Background .....	29
4.1 Contributions from Last Year .....	29
4.1.1 Subsumption architecture.....	29
4.2 Tools and Software .....	30
4.2.1 Robot Operating System.....	30

4.2.2 iRobot Create .....	31
4.2.3 LiDAR Sensor.....	31
4.2.4 Gazebo .....	31
4.2.5 Ollama.....	32
4.2.6 UV .....	32
5.0 Group Skills .....	34
6.0 Methods.....	36
7.0 Analysis.....	37
7.1 LLM Feasibility Evaluation.....	37
7.1.1 LLM selection criteria for Raspberry Pi 4 .....	37
7.1.2 Evaluation methodology .....	38
7.1.3 Results and observations.....	38
7.1.4 Conclusion .....	40
7.2 Analysis of Current System Architecture .....	40
7.3 Analysis of the Influence of Large Language Model Capacity on Navigation Decision Accuracy .....	43
7.3.1 Experiment results .....	44
7.3.3 Results and analysis .....	44
7.3.4 Conclusion .....	45
7.4 Effects of Prompt Variations on Model Performance.....	46
7.4.1 Effects of Prompt Variation on Navigation AI.....	46
7.4.2 Effects of Prompt Variation on Avoidance Layer AI .....	49
7.4.3 Effects of Prompt Variation on Beacon Sensor AI.....	52
7.4.4 Effects of Prompt Variation on Lidar Sensor AI .....	55
7.4.5 Comparing the Accuracy of a Larger LLM (Gemma 2 9B) with No Prompt Style Against the Accuracy of a Smaller LLM (Qwen 2.5: 0.5B).....	59
7.4.6 Summary of Findings on Effects of Prompt Variations on Model Performance.....	60
8.0 Metrics .....	61
8.1 Preface.....	61
8.2 Metrics Tracking.....	61
8.3 Automated Report Generation for Metrics Tracking.....	63

8.4 Dashboard .....	66
9.0 Design .....	69
9.1 System Overview and ROS 2 Architecture Diagrams .....	69
9.2 LLM-Augmented Subsumption Architecture .....	73
10.0 Implementation .....	75
10.1 Connecting to the Physical Robot.....	<b>Error! Bookmark not defined.</b>
10.2 Docker Usage.....	<b>Error! Bookmark not defined.</b>
10.3 LLM Integration Process .....	75
10.3.1 Injection Methodology.....	75
10.3.2 LLM fallback .....	76
10.3.3 Model comparison and behavior.....	<b>Error! Bookmark not defined.</b>
10.4 Simulation.....	<b>Error! Bookmark not defined.</b>
10.4.1 Debugging the previous year's version of the codebase .....	<b>Error! Bookmark not defined.</b>
10.5 Avoidance Layer LLM Implementation .....	76
10.6 Docking Layer LLM Implementation.....	78
10.7 Turning Layer LLM Implementation.....	80
10.8 Beacon Sensor LLM Implementation .....	82
10.9 Navigation Unit LLM Implementation.....	83
10.10 LiDAR Sensor LLM Implementation.....	84
10.11 AI node Integration .....	86
11.0 Verification and Validation.....	93
11.1 Continuous Integration.....	93
11.2 Robot Testing.....	94
11.3 Unit Testing .....	94
11.4 System-Level Validation .....	95
11.5 ROS Modules Testing.....	<b>Error! Bookmark not defined.</b>
11.3.1 Lidar Sensor Testing.....	<b>Error! Bookmark not defined.</b>
12.0 Required Components and Facilities .....	97
13.0 Conclusion .....	99
14.0 Recommendations.....	100

15.0 References .....	101
Appendix A: Test Data .....	102
Appendix B: Use Case Tables .....	103
Appendix C: Create 3 Hardware Setup.....	118
Appendix D: Create 3 Software Setup.....	118
Appendix E: Gazebo Setup.....	<b>Error! Bookmark not defined.</b>
Appendix F: Experiment methodology.....	124
Appendix G: Original Project Proposal .....	130
Table 4: Use Case Description Table of Viewing Historical Performance .....	143
Table 5: Use Case Description Table of Monitoring the System Dashboard .....	145
Table 6: Use Case Description Table of Executing the Navigation Plan .....	147
Table 7: Use Case Description Table of Navigating the Tunnel Network .....	148
Table 8: Use Case Description Table of Docking for Charging.....	150
Table 9: Use Case Description Table of Avoiding Obstacles.....	151
Table 10: Use Case Description Table of Update Dashboard on pull request.....	154
Table 11: Use Case Description Table of assisting with sensor data processing .....	155

## List of Figures

Figure 1: Use Case Diagram of Robot Mail-Delivery System	140
Figure 2: Dashboard Homepage Concept	161
Figure 3: LLM Metrics Tracking	162
Figure 4: Performance Over Time Visualization	163
Figure 5: The system architecture from the previous year's version of the system	171
Figure 6: Vertical Partitioning of the Architecture	172
Figure 7: Horizontal Partitioning of the Architecture	172
Figure 8: Gantt Chart of the Milestones	176
Figure Appendix A9: Qwen 3.0 response times	180
Figure Appendix A 10: Qwen 2.5:0.5b	180

## List of Tables

Table 1: Functional Requirements	138
Table 2: Non-Functional Requirements	139
Table 3: Use Case Description Table of Simulating in Gazebo	141
Table 4: Use Case Description Table of Viewing Historical Performance	143
Table 5: Use Case Description Table of Monitoring the System Dashboard	145
Table 6: Use Case Description Table of Executing the Navigation Plan	147
Table 7: Use Case Description Table of Navigating the Tunnel Network	148
Table 8: Use Case Description Table of Docking for Charging	150
Table 9: Use Case Description Table of Avoiding Obstacles	151
Table 10: Use Case Description Table of Update Dashboard on pull request	154
Table 11: Use Case Description Table of	155
Table 12: Table of Metrics and Justifications	163
Table 13: Table of LLMs Tested	169
Table 14: Table of Pros and Cons for Each Partitioning Method	173
Table 15: Table of Milestones and Target Date of Completion	174
Table 16: Project Risks and Mitigation Strategies	176
Table 17: List of required Components/Facilities, Purposes, and Estimated Costs	177

## **1.0 Description**

By Jonas Andaya and Tommy Phang

The purpose of the mail-delivery robot project is to introduce reliable mail-delivery inside the tunnels of Carleton University. This would provide Carleton University with an alternative method to deliver mail to other buildings and faculty. It would also give faculty more free time to focus on other important tasks.

The current project explores the use of large language model (LLM) agents to program an iRobot for mail delivery in the Carleton tunnels, with an emphasis on maximizing LLM involvement within the system. The LLM may be used for tasks such as processing sensor input, planning, and action execution. Each iteration of the system is evaluated using a dashboard that displays performance metrics, alongside a simulation tool to support testing and analysis.

## **2.0 Project Objectives**

By Jonas Andaya, Connor Yelle, Umniyah Mohammed, and Tommy Phang

The objective of this project is to explore the use of LLM agents to program an iRobot for mail delivery in the Carleton tunnels. To achieve this, the project will maximize the involvement of LLMs in planning, control, and data processing tasks. It will also assess the feasibility of running an LLM on the Raspberry Pi 4 and extend the previous year's codebase to support LLM functionality. Additionally, our team plans to develop a web-based dashboard to track system performance and changes, monitor LLM decision-making, and evaluate project progress.

The first portion of the project will be dedicated to developing the dashboard and integrating the LLM agent into the preexisting code from previous years. Initially, development will focus on a basic integration of the dashboard, robot system, and LLM implementation to establish a connection between the robot and the dashboard. Metrics from the ROS nodes will then be incrementally added to the dashboard. Getting the dashboard operational as early as possible is important, as it will serve as a valuable indicator of whether the implemented changes are improving or degrading robot behavior. The dashboard will provide a clear visual representation of the tracked metrics, aiding in monitoring and analysis throughout the project.

Throughout the year, the project will follow an agile methodology to support an iterative software development lifecycle. Agile practices will help maintain organization and ensure the team progresses at a consistent pace. Jira will be used for comprehensive issue tracking, sprint planning, and task management to ensure transparent monitoring of both functional and non-functional requirements. Maintaining steady progress, meeting sprint goals, and incorporating the project supervisor's feedback will be essential to ensuring the project's success.

## 2.1 Requirements

### 2.1.1 Functional Requirements

By Umniyah Mohammed

*Table 1: Functional Requirements*

ID	Requirement Statement
FR-1.0	The <b>System</b> shall autonomously navigate the Carleton University tunnel network to deliver mail between predefined locations.
FR-2.0	The <b>LLM Agent</b> shall utilize a locally hosted Large Language Model (LLM) to assist with sensor data processing throughout as many areas of the robot as possible.
FR-3.0	The <b>Robot</b> shall execute the navigation plan provided by the Bluetooth beacons or the ROS Navigation Stack, maintaining real-time control.
FR-4.0	The <b>Robot</b> shall avoid static and dynamic obstacles using data from its LiDAR and bump sensors.
FR-5.0	The <b>System</b> shall autonomously dock at its charging station upon low battery detection or task completion.
FR-6.0	The <b>Dashboard</b> shall provide a web-based interface for monitoring the robot status, performance metrics, and LLM decision-making rationale.
FR-6.1	The <b>Dashboard</b> shall display historical data and performance comparisons between test runs with and without the LLM enabled.
FR-7.0	The <b>System</b> shall be operable within a Gazebo simulation environment for development and testing purposes.

### 2.1.2 Non-Functional Requirements

By Umniyah Mohammed

*Table 2: Non-Functional Requirements*

ID	Requirement Statement	Category	Measurement/Constraint
NFR-1.0	The <b>Robot</b> shall maintain a minimum safe distance of 1 meter from all detected dynamic obstacles during travel.	Safety	Measured by LiDAR sensor input during motion.
NFR-2.0	The <b>System</b> shall successfully complete a standard mail delivery route with an acceptable success rate to be determined in future testing.	Reliability	Over the test runs on a predefined route, the robot must reach the correct destination without manual intervention in an acceptable number of runs.
NFR-3.0	The <b>LLM Agent</b> shall return a valid plan to the Robot within a system-defined acceptable response time threshold of receiving an intervention request.	Performance	System-wide latency under complex conditions.
NFR-4.0	The <b>System</b> shall revert to a Safe Fallback State if the LLM Agent returns an invalid or null response.	Reliability	Verified by state machine transitions.
NFR-5.0	The <b>System</b> shall run on the existing Raspberry Pi 4 hardware and be compatible with ROS 2.	Compatibility/ Architectural	Verified by successful compilation and operation on the specified hardware/software stack.
NFR-6.0	The selected <b>LLM</b> must be able to run on the existing Raspberry Pi 4.	Development Constraint	Verified by model file size and published parameter count before deployment to Raspberry Pi 4.

NFR-7.0	The <b>Dashboard</b> shall be accessible via a standard web browser (Chrome, Firefox, etc.) on a desktop operating system.	Compliance/ Usability	Verified by testing accessibility on specified browsers
---------	----------------------------------------------------------------------------------------------------------------------------	--------------------------	---------------------------------------------------------

## 2.2.1 Use Case Diagram

By Umniyah Mohammed

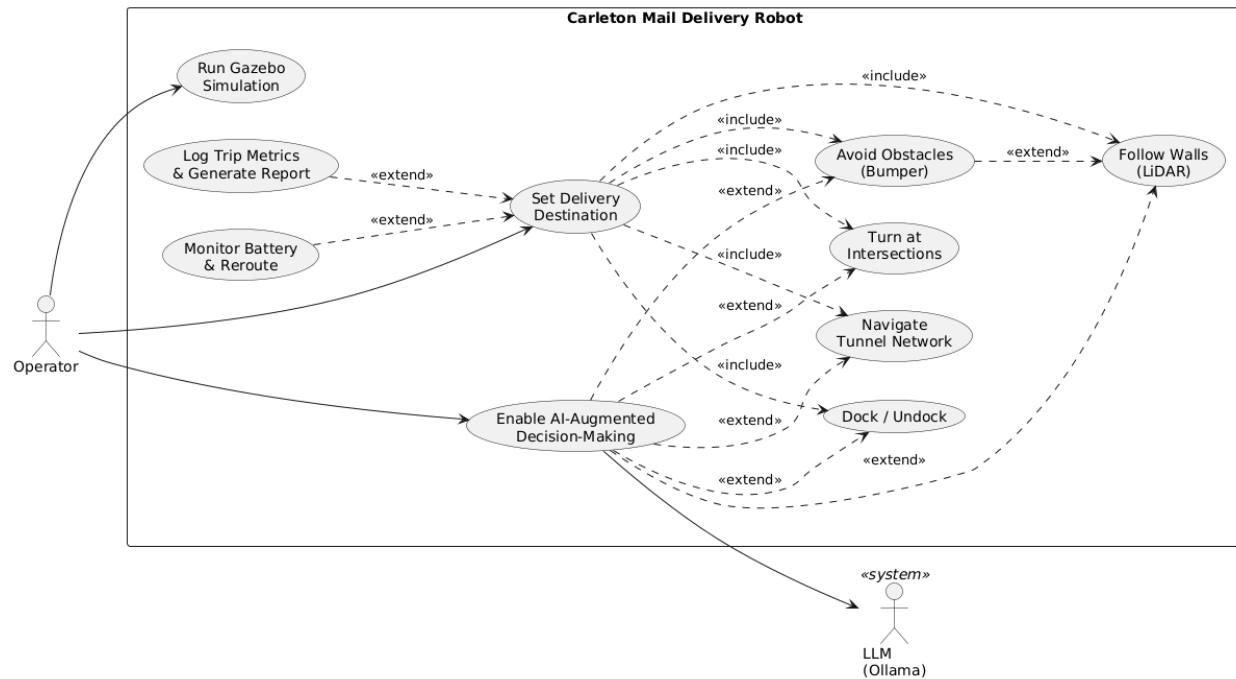


Figure 1: Use Case Diagram of Robot Mail-Delivery System

### 2.2.2 Use Case Tables

By Connor Yelle, Owen McKibbon

Table 3: Use Case Description Table of Simulating in Gazebo

<b>Use Case Name</b>	Simulate in Gazebo
<b>Brief Description</b>	The user simulates the robots' movements and travel path in Gazebo for testing.
<b>Primary Actor</b>	Gazebo
<b>Secondary Actor</b>	N/A
<b>Precondition</b>	The system is running, the LLM server is running, and the simulation is turned on.
<b>Dependency</b>	N/A
<b>Basic Flow</b>	<b>Steps:</b>      3. The user submits a mail delivery request to the console.    4. The user watches the simulation in Gazebo for successful robot decisions.      <b>Postcondition:</b>   Gazebo successfully simulated the path the robot would take in a real-life scenario.
<b>Specific Alternate Flows</b>	RFS Basic Flow 1   <b>Steps:</b>

2. If the system is not running, then  
send an error to the user explaining  
what happened.

**Postcondition:**

The user is informed that a specific part of  
the system is not running.

Table 4: Use Case Description Table of Viewing Historical Performance

<b>Use Case Name</b>	View Historical Performance
<b>Brief Description</b>	The user utilizes the dashboard to view historical versus current performance to analyze how well the LLM is doing in the current implementation.
<b>Primary Actor</b>	User
<b>Secondary Actor</b>	N/A
<b>Precondition</b>	A pull request was sent to the GitHub page. The system is running, and the dashboard is active.
<b>Dependency</b>	N/A
<b>Basic Flow</b>	<b>Steps:</b>       4. The user submits a command to the console telling the robot to perform a trip.     5. The dashboard updates the tracked metrics based on the data it receives from the sensors and previous version of the metrics.     6. The user presses the metrics tab and analyses the new metrics       <b>Postcondition:</b> The dashboard displays the tracked metrics with either red or green bars based on if the metric improved or got worse.
<b>Specific Alternate Flows</b>	N/A



Table 5: Use Case Description Table of Monitoring the System Dashboard

<b>Use Case Name</b>	Monitor System Dashboard
<b>Brief Description</b>	The user utilizes the dashboard to view current robot metrics such as battery life, or the Jira Kanban board for project objectives
<b>Primary Actor</b>	User
<b>Secondary Actor</b>	N/A
<b>Precondition</b>	The dashboard is active and running.
<b>Dependency</b>	N/A
<b>Basic Flow</b>	<b>Steps:</b>       3. The user opens the URL that links to the dashboard web server.     4. The user analyses the Kanban board to see which tasks need to be performed in this sprint.       <b>Postcondition:</b>   The dashboard displays all tasks useful to the user during the current sprint.
<b>Specific Alternate Flows</b>	RFS Basic Flow 2   <b>Steps:</b>       2. The user monitors the various metrics tracked by the robot, like battery life or current trip.       <b>Postcondition:</b>

The dashboard displays all metrics relevant to the usage of the robot.

Table 6: Use Case Description Table of Executing the Navigation Plan

<b>Use Case Name</b>	Execute Navigation Plan
<b>Brief Description</b>	The ROS system uses the command sent to the system, create and execute a navigation plan to reach the targeted destination.
<b>Primary Actor</b>	ROS
<b>Secondary Actor</b>	N/A
<b>Precondition</b>	The system is running, the robot is turned on, and the Bluetooth beacons are active and installed.
<b>Dependency</b>	INCLUDE USE CASE Avoid Obstacles
<b>Basic Flow</b>	<b>Steps:</b>       4. The Robot Controller obtains a command from the user, parses it, and sends the command into the ROS node     5. The ROS node reads the parsed command to create a navigation plan with the goal of reaching the destination in the most efficient way possible.     6. Once completed, ROS executes the created navigation plan by sending commands throughout the various nodes that communicate with the hardware inside the robot.       <b>Postcondition:</b>

The robot is in transit to its desired destination using the created navigation plan.

### Specific Alternate Flows

#### RFS Basic Flow 1

##### Steps:

2. If the user submits an invalid command, the system promptly sends an error explaining what happened.

##### Postcondition:

ROS is idle waiting for the user to submit a command.

Table 7: Use Case Description Table of Navigating the Tunnel Network

<b>Use Case Name</b>	Navigate Tunnel Network
<b>Brief Description</b>	The robot uses its sensors and Bluetooth beacons to navigate the tunnel network to reach its destination.
<b>Primary Actor</b>	Robot Controller
<b>Secondary Actor</b>	ROS
<b>Precondition</b>	The system is running, the robot is turned on, and the beacons are installed throughout the tunnels.
<b>Dependency</b>	INCLUDE USE CASE Execute Navigation Plan
<b>Basic Flow</b>	<b>Steps:</b>

4. The robot controller receives a command from the user and checks if it has enough battery to perform the trip
5. The robot controller creates a navigation plan based on the current robot location and the proposed destination
6. If the robot has enough battery, the robot starts navigating the tunnels using the navigation plan.

**Postcondition:**

The robot controller is reading sensor data and using the LLM agent to process data to make correct movements so it can reach its destination.

**Specific Alternate Flows**

N/A

Table 8: Use Case Description Table of Docking for Charging

<b>Use Case Name</b>	Dock for Charging
<b>Brief Description</b>	The robot docks itself once it notices it is low on battery
<b>Primary Actor</b>	Charging Station
<b>Secondary Actor</b>	LLM Agent
<b>Precondition</b>	The robot has low charge
<b>Dependency</b>	INCLUDE Use Case Assist with Sensor Data Processing
<b>Basic Flow</b>	<b>Steps:</b>       4. The user submits a command to the console telling the robot to perform a mail delivery     5. The LLM agent notices that the robot does not have enough charge to perform the trip, so it changes its current destination to the closest charging station.     6. The robot locates the charging station and docks itself to charge its battery       <b>Postcondition:</b>   The robot changes states to charging and is positioned on the charging dock.
<b>Specific Alternate Flows</b>	RFS Basic Flow 1

**Steps:**

4. The robot finishes the mail delivery and has no new trips to perform.
5. The robot locates and changes its location to the nearest charging station.
6. The robot arrives at the charging station and docks itself to charge its battery

**Postcondition:**

The robot changes states to charging and is positioned on the charging dock.

Table 9: Use Case Description Table of Avoiding Obstacles

<b>Use Case Name</b>	Avoid Obstacles
<b>Brief Description</b>	The robot controller utilizes the LLM agent to read sensor data and make correct movements to avoid various obstacles.
<b>Primary Actor</b>	Robot Controller
<b>Secondary Actor</b>	LLM Agent
<b>Precondition</b>	The robot is in transit to its desired destination
<b>Dependency</b>	INCLUDE USE CASE Assist with Sensor Data Processing
<b>Basic Flow</b>	<b>Steps:</b>

5. While the robot is in transit to its destination, the LiDAR sensor finds an obstacle in the robot's path.
6. The sensor data is sent to the robot controller, where it forwards the data to the LLM agent.
7. The LLM agent analyses the data and decides what to do to avoid the obstacle and continue the desired path.
8. The LLM decision is sent to the Robot Controller, where it sends commands to the various ROS nodes to make the robot avoid the obstacle.

**Postcondition:**

The robot successfully avoids the obstacle and continues its journey to the destination.

**Specific Alternate Flows**

RFS Basic Flow 1

**Steps:**

5. The LiDAR sensor finds a wall that the robot is about to run into.
6. The sensor data is sent to the robot controller, where it forwards the data to the LLM agent.
7. The LLM agent analyses the data and decides what to do to avoid the wall. It uses the navigation plan to know which way to turn.
8. The LLM decision is sent to the Robot Controller, where it sends commands to the various ROS nodes

to make the robot turn to avoid the wall.

**Postcondition:**

The robot successfully performs a turn to avoid the wall and continues its journey to the destination.

Table 10: Use Case Description Table of Update Dashboard on pull request

<b>Use Case Name</b>	Update Dashboard on pull request
<b>Brief Description</b>	Upon pull request, the dashboard will update with the latest simulation results for metrics.
<b>Primary Actor</b>	Gazebo Simulator
<b>Secondary Actor</b>	N/A
<b>Precondition</b>	A pull request has been initiated
<b>Dependency</b>	INCLUDE Use Case Monitor System Dashboard
<b>Basic Flow</b>	<b>Steps:</b>       4. Simulation is run in gazebo environment.     5. User submits pull request to repository.     6. Dashboard is updated with simulation data.       <b>Postcondition:</b>   The Dashboard contains the latest simulation results and metrics for the robot.
<b>Specific Alternate Flows</b>	N/A

Table 11: Use Case Description Table of assisting with sensor data processing

<b>Use Case Name</b>	Assist with sensor data processing
<b>Brief Description</b>	The LLM agent assists in processing data from various sensors into information that can be used for navigation decision making.
<b>Primary Actor</b>	LLM Agent
<b>Secondary Actor</b>	N/A
<b>Precondition</b>	Sensor Data is Received from robot's sensors
<b>Dependency</b>	N/A
<b>Basic Flow</b>	<b>Steps:</b>       4. Sensor data is received from LiDAR sensor/collision detector.     5. Data is processed by LLM agent.     6. Processed data is used for navigation decisions.       <b>Postcondition:</b>   The robot makes navigation decisions based on processed sensor data.
<b>Specific Alternate Flows</b>	N/A

### 3.0 Background

By Jonas Andaya

This will be the fifth year of the project. Last year, the team shifted the system to a subsumption architecture and provided a simulator for a more scalable system for future iterations. They also introduced new “layer” nodes for the robot to coincide with the subsumption architecture.

This year’s work will primarily focus on implementing and testing the LLM agent, rather than fixing the known issues from the previous iteration. Last year’s report mentioned that there is room for improvement of the Gazebo simulation and suggested introduction a new web application in the form of a dashboard. Both the updated Gazebo simulation and the new web application will support testing and evaluation of the LLM agent.

#### 3.1 Robot Operating System

By Jonas Andaya

The Robot Operating System is a framework for the development of robotics software. In this project, ROS 2 will be used. ROS 2 is the second generation of the ROS framework and aims to fix some of the issues that riddled ROS 1. ROS 1 has data delivery issues, a single point of failure, and no built-in security mechanisms [1]. The robot will use the Humble Hawksbill distribution of ROS 2. Humble Hawksbill supports Ubuntu 22.04 (Jammy), which is what will be used in the Raspberry Pi 4.

In last year’s implementation, ROS 2 was used to control all the functions of the robot. This was done using ROS 2’s library of drivers and algorithms, nodes, and the subsumption architecture mentioned in Section 1.0. This year, ROS 2 will be used for more of the same. However, this year’s implementation will have a new node called “AI” which should handle the LLM and its prompts with Ollama.

#### 3.2 iRobot Create

By Jonas Andaya

iRobot Create is a development platform for robotics that runs on ROS 2. This means that you can send commands and receive data using ROS 2 topics. iRobot Create was developed by iRobot and is used for educational and research purposes. It also has compatibility with Gazebo.

The newest generation of the iRobot Create is the iRobot Create 3 and it has been used in past versions of the mail delivery robot project. The notable additions to the iRobot Create 3 over the past years have been a camera and a LiDAR sensor. The iRobot Create 3 will still be used this year with no modifications to the robot.

### 3.3 LiDAR Sensor

By Jonas Andaya

Light Detection and Ranging (LiDAR) is a way to determine distances by shooting out laser beams and measuring how long it takes to return. A LiDAR sensor is comprised of a laser to shoot out the pulsed beams and a receiver to detect the pulsed beams [4].

In the 2023 iteration of the mail-delivery robot, they purchased the Slamtec RPLIDAR sensor to improve the navigation capabilities of the robot. The Slamtec RPLIDAR sensor can scan 360 degrees within 12 meters and is able to change its scan rate from 2-10 Hz. This year's iteration of the mail-delivery robot will continue to use the LiDAR sensor. With the introduction of the LLM agent, the team would like to experiment with how well it interacts with the LiDAR sensor or if it can even interact with the LiDAR sensor.

### 3.4 Gazebo

By Jonas Andaya

Gazebo is an open-source simulation software that is primarily used to simulate robots in an outdoor setting. Gazebo comes with open-source libraries for the math, physics, sensors and rendering of the simulator. Since ROS and Gazebo were both developed by Open Robotics, the developers included many documents for getting Gazebo to interact with ROS 2 [3].

Last year was the first year that Gazebo was used to simulate the mail-delivery robot and the Carleton tunnels. This year, Gazebo will continue to be used for simulating the mail-delivery robot. There should be 2 types of simulations for the robot: one where the Raspberry Pi is used and one where a more powerful computer is used.

### 3.5 Ollama

By Jonas Andaya

Ollama is software that is used to run Large Language Models (LLMs) locally. Ollama makes it easy to get started on interaction with LLMs because it has clear documentation and a beginner friendly user interface [2]. Previous iterations of the project did not need Ollama because they did not need the use of an LLM. This year, we will be using Ollama to test LLMs and to interact with our robot and ROS system. There is an analysis of the LLMs in Ollama in the proposal which can be seen in section 6.4. Ollama should be downloaded to the Raspberry Pi so that the AI ROS node can interact with Ollama and the LLM.

### 3.6 LangGraph

By Tommy Phang

Our team is exploring the potential use of LangGraph in our project to help customize existing models to better fit our needs. LangGraph is an open-source framework for building modular AI agents. Our team is considering the use of LangGraph because it allows more control over how the used LLM behaves. LangGraph would allow our team to control how, when, and why the large language model is called. LangGraph will allow our team to configure what context gets passed in, which model to use based on the task or input, and how the large language model's outputs are filtered.

### 4.0 Individual Relation of Degree to Project

By Connor Yelle and Umniyah Mohammed

Our team, consisting of five members, possesses a complimentary skill set that will contribute to a successful year of working on this capstone project. All five team members are students in the Software Engineering program at Carleton University, each bringing a unique combination of experience, programming proficiency, problem-solving ability, and communication skills. These strengths will support the design and development of a thoughtful and well-designed system for the robot. The team is confident that the knowledge gained through both academic coursework and professional experience will be sufficient to address the technical and organizational challenges presented by this capstone project.

The codebase inherited from previous years is primarily written in Python. Since every team member has prior experience with Python through ECOR 1041 (Computation and Programming), ECOR 1042 (Data Management), and SYSC 2100 (Data Structures and Algorithms), becoming familiar with and extending the existing system should be manageable and efficient. Because the robot operates using a Raspberry Pi, having team members with an understanding of low-level programming languages such as C (developed through SYSC 2006: Foundations of Imperative Programming) will be crucial for ensuring efficient iteration throughout the project.

The robot's codebase utilizes middleware called ROS2 (Robot Operating System 2). ROS provides a variety of frameworks, libraries, and tools that assist in developing robot functionality. Since most team members have taken SYSC 4001 (Operating Systems), becoming familiar with ROS should be straightforward. ROS heavily relies on state machines to capture robot states and support decision-making. Therefore, the experience gained from SYSC 3303 (Real-Time Concurrent Systems) will be a valuable asset throughout the project.

To ensure that weekly iterations do not disrupt the overall system, the team is implementing automated testing and automatic dashboard updates through GitHub Actions, a tool introduced in

SYSC 4806 (Software Engineering Lab). Testing remains a primary focus throughout the project and will be conducted using techniques learned in SYSC 4101 (Software Validation and Verification).

## **5.0 Group Skills**

By Connor Yelle, Jonas Andaya, Owen Mckibbon, Tommy Phang, Umniyah Mohammed

### **Connor Yelle:**

Connor is a fourth-year software engineering student. As such he has a lot of fundamental knowledge of system design and programming. Connor knows Python, C, and Java, which are all languages that are useful when working on the task at hand. Connor also has prior knowledge of embedded systems programming in C, and web development in CSS/HTML/JavaScript, which are both critical skills that are needed to succeed in the project. Throughout his education at Carleton, Connor learned critical software engineering development lifecycle skills like requirements elicitation and analysis, software architecture, and testing which will be useful for making critical decisions throughout the year.

### **Jonas Andaya:**

Jonas is a fourth-year software engineering student. Jonas has experience programming in languages such as Python, which is used in the code of the mail-delivery robot. Jonas has knowledge of the planning stage of the software development life cycle. Particularly, Jonas has experience with creating sequence diagrams, use case diagrams, and state machine diagrams. All of these will aid in the explanations of how the mail-delivery robot works.

### **Tommy Phang:**

Tommy is a fourth-year software engineering student, with foundational skills in software engineering. Tommy currently has gained proficiency in Python through ECOR 1041, ECOR 1042 and SYSC 2100, Python is utilized in the capstone project. Tommy is familiar with software development lifecycles methodologies such as Agile and DevOps, learned in SYSC 4106. Material learned from SYSC 4106 will be useful in managing the software development process of the capstone project.

**Umniyah Mohammed:**

Umniyah is a fifth-year software engineering student driven by a passion for robust software architecture and cybersecurity. Umniyah has experience in Python, C, C++, Java, and Racket, as well as modern web development frameworks including HTML, CSS, JavaScript, TypeScript, React, and related tools such as npm and Vite. Umniyah has applied her skills in a variety of projects, ranging from AI-driven applications and chatbots to full-stack development and team-led software projects, demonstrating abilities in software development, software requirements, software architecture, SDLC methodologies, design patterns, and software validation and verification. Additionally, she has professional experience as a software developer and researcher, including back-end algorithm validation and full-stack development internships, where she collaborated with cross-functional teams to deliver optimized, industry-quality code. Her combination of academic knowledge, applied project experience, and industry exposure provides critical expertise to the capstone project, particularly in software design, system implementation, and secure, maintainable coding practices.

**Owen McKibbin:**

Owen is a fourth-year software engineering student with a deep curiosity for machine learning, simulation, and robotics. Owen has extensive experience in Python through several projects ranging from data visualization and parsing to machine learning simulations. Owen has also worked with languages such as C, C++, and Java, giving him a solid foundation for systems and embedded programming. Through both coursework and independent projects, he has developed strong software engineering practices, including version control with Git, task delegation, and collaborative communication in team settings. Owen is eager to bring his technical skills, problem-solving mindset, and commitment to planning and research to the mail delivery robot project alongside his teammates.

**6.0 Testing and Metrics****6.1 Dashboard**

By Umniyah Mohammed and Connor Yelle

As mentioned in Section 2.0, the dashboard serves as the central hub for tracking robot metrics. It will act as the primary point of reference after each system iteration to determine whether the robot's performance is improving or regressing as a result of the LLM integration. The dashboard will also function as a gateway to the project's management Kanban board.

The following figures present proof-of-concept designs for the dashboard. All figures were created using Figma, a web-based interface design tool that enables collaborative and flexible creation of digital interfaces. Figma was selected for its precision, scalability, and ease of editing, which support rapid iteration, team collaboration, and the integration of real UI components and design systems. These features make the dashboard prototypes closely aligned with the intended final implementation.

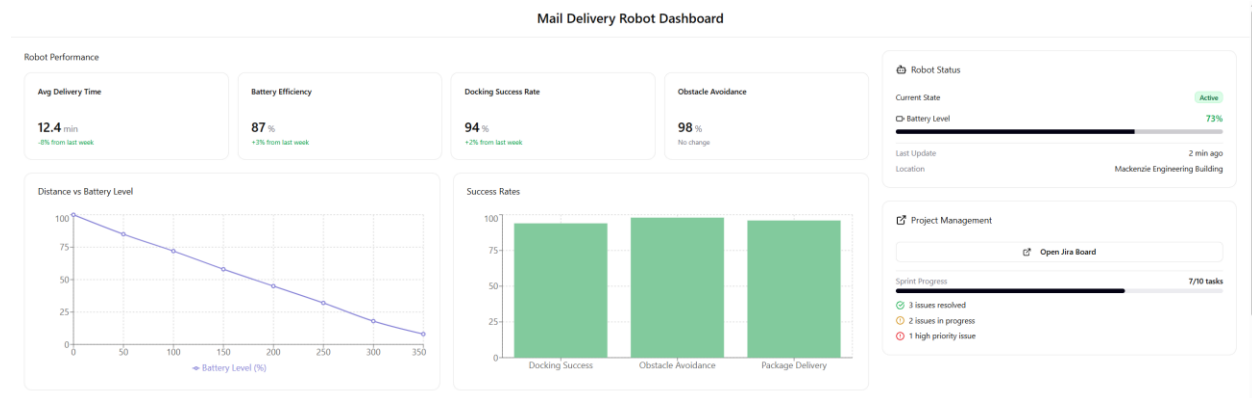


Figure 2: Dashboard Homepage Concept

Figure 2 shows the proof-of-concept for the dashboard homepage. Many important sections are easily accessible and immediately visible from this page.

### Robot Performance:

This panel displays four key metrics that will need to be tracked after every successful system build. These metrics were selected because they are expected to be most affected by the changing role and placement of the LLM within the system. For example, if the LLM assumes a greater role in decision-making, it may negatively impact battery efficiency. The metrics also capture the robot's primary operational behaviors (such as obstacle avoidance, docking, and general movement), allowing for effective monitoring of performance trends over time.

### Robot Status:

This panel shows the robot's current battery level and activity status. This information is valuable because the robot may continue performing background tasks or maintain communication with the Ollama server even when it is not in motion. Distinguishing between an idle state and an inactive state is therefore crucial for accurate performance assessment.

## Project Management

This panel displays the current status of the project, including high-priority, in-progress, and completed tasks, presented in an organized and dynamic format that updates automatically with each pull request on GitHub. This automation ensures that the team remains aware of active tasks in each sprint, identifies tasks that carry over to subsequent sprints, and tracks completed work. The panel serves a crucial role in project management by highlighting potential bottlenecks that may impede weekly progress.



Figure 3: LLM Metrics Tracking

Tracking LLM metrics such as response time and instruction accuracy is essential for evaluating system performance. This tab within the dashboard provides real-time visualizations of these metrics, which enable continuous monitoring of how efficiently the LLM processes data and communicates with the robot's sensors.

The dashboard separates Robot Metrics and LLM Metrics to distinguish between hardware performance and decision-making performance. This separation helps isolate whether an issue (e.g., poor docking) is due to LLM decision-making or sensor response. The goal is to build by combining reusable chunks; the split view allows us to debug and analyze the system modularly.

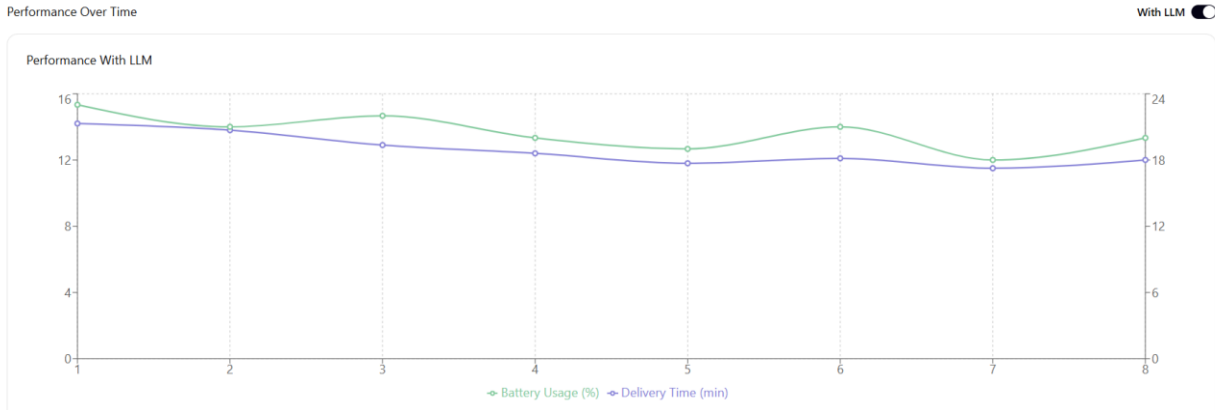


Figure 4: Performance Over Time Visualization

Figure 4 presents a combined history graph that tracks system performance over time. A toggle allows users to switch between “With LLM” and “Without LLM” data, while the line chart compares average delivery time and battery usage across multiple runs. This visualization provides an at-a-glance overview of overall system performance and efficiency. The graph is particularly valuable for analyzing the impact of the LLM agent on the robot’s operation. By comparing performance with and without the intelligent agent, the team can evaluate trade-offs and identify the optimal points in the system architecture where the LLM should be integrated.

## 6.2 Metrics Tracking

By: Connor Yelle, Jonas Andaya, Owen Mckibbon, Tommy Phang

Table 12: Table of Metrics and Justifications

Metric	Reasoning	Method
Missed package/Did not reach destination/Success rate – Count	This metric is important because it will measure how reliable the system is. Specifically, the robot movements and obstacle detection would be some important parts of making this project successful that would be impacted by more missed packages.	During the testing of the robot with predefined routes, if the robot ends up in a different location than desired it will be tabulated as a failure and measured against successes.
Time per delivery – sec	This metric can be useful to calculate the speed of the robot, and also to see if the	By keeping a log with time stamps of the robot's departure and arrival during a

	trip is actually ending when it's supposed to end. This metric can also be useful for seeing if the LLM is improving the robot's decision making.	test we can measure the time per delivery.
Battery Efficiency – Voltage per delivery or m	This metric is useful to see how much of the battery is being used during delivery. If a high voltage is being used during delivery, it might be smart to change LLMs. Maybe the LLM is too big to be able to run on the robot.	There is a node inside of ROS2 that directly obtains the usage of the battery in voltage. We must calculate the average voltage per delivery. This number can be queried every second during delivery to ensure the most accurate average possible.
Battery Life – %	This metric can be important to see how the robot uses the battery. If this percentage goes down fast, there is an optimization flaw in either the robot, or the LLM. Maybe the LLM is too big or takes too much energy to run.	This will be tracked by checking the battery status node given to us by ROS2. All we must do is multiply the node output by 100 to turn it into a percentage.
Docking Rate Success – %	It is important to see the percentage of times the robot actually succeeds at docking, so that we can see if the LLM is sending the data correctly, or if the docking layer is not processing data correctly. This would be a good metric to track robot improvements.	When the robot is sent to dock the team can measure the time it takes to dock itself and if it is unable to do so within 30 seconds it is considered a failure. If the robot successfully transitions to the docked state, it is noted as a success.
Obstacle Avoidance Accuracy – %	This is important to see how well the Avoidance layer is transmitting data, or if the LLM is transmitting false data to the avoidance layer.	The team will measure the number of times an obstacle has been recorded in the LiDAR data versus the number of times the bumper has been hit to measure the

		accuracy of the obstacle avoidance.
Stuck Incidents – incidents per delivery	This metric is important to measure the cause of lengthier trips or failures of trips. It allows us to track improvements in the number of incidents to achieve a more efficient run.	If the location of the robot has no change after 30 seconds, it is marked as stuck and tallied.
Path Optimality – actual delivery path: optimal delivery path	Measuring the actual path versus the known optimal path allows us to gain insight into the number of incorrect decisions our model is making when deciding what to do at intersections to reach the destination.	Using our known set of most optimal routes between locations, the team can log each decision at an intersection and map out the chosen path versus the known optimal path.
Robot Odometry	Measuring the distance travelled by the robot allows us to track distance over battery life and expected distance versus the actual distance.	The team can use the average speed of the robot over the time spent travelling to find the total distance travelled for a trip.
Distance travel vs Battery	This metric is very important for efficient benchmarking. If we see an actual visual graph, we can utilize the trend lines to see how our robot is transferring energy into physical movement.	The team can use the found odometry and measure it against the battery level the robot has.
Distance vs time	This can be shown as a graph to see either how fast the robot is moving, or to see how optimal the route it is taking is. Lower times and higher distances = better robot.	Using the known distance between destinations and the logged time spent delivering the team can measure these two properties against each other.

Prompt response time	This is a crucial metric that can be used to assess the efficiency of our chosen LLM. If this metric is too high, it might be smart to either change how the LLM is processing data and sending to the different Layers. This metric can be used to analyze if we select a sufficient LLM for the robot.	This metric will be tracked by directly checking the Ollama server. The Ollama server sends the response time of the LLM every time it sends a new prompt.
Instruction success rate	Is the LLM making correct decisions based on the HashMap outlined	Using the HashMap of the known optimal routes versus the logged chosen turns of the robot for a trip, the team can see if the instructions follow the ideal path.
Intersection Navigation Time – sec	This metric is important because it will be used to track how long the LLM takes to think in an intersection. This will indicate to us whether the LLM is a good choice for our robot or if the LLM is controlling more of the system than it can handle.	By logging the timestamps for the first entrance of an intersection and when the robot returns to the wall following, the team can measure the difference to find the intersection navigation time.
Time spent following wall – sec	This can be important to see how the robot is reading LiDAR sensor data and how it's converting it into actual robot movements. Is the turning layer and travel layer working? Are they passing the right data from the LLM?	During the robot's trip the team will log the time when the LiDAR sensor picks up the wall next to it during a trip.

### 6.3 LLM Testing

By Tommy Phang

Our team so far evaluated four different language models based on the selection criteria outlined in Section 6.3.1. A model must meet all these criteria to be considered suitable for testing.

### 6.3.1 LLM selection criteria for Raspberry Pi 4

(subject to change as the project progresses)

The following selection criteria were derived from the hardware limitations of the Raspberry Pi 4 and real-time application requirements. This helps our team select a large language model that is suitable for our project.

- **Model Size (< 3B Parameters):** The model must have fewer than 3 billion parameters, following Raspberry Pi Organization recommendation.
- **Tool Support:** The model must support external tools to enable extended functionality.
- **Quantization Availability:** Quantized versions should be available to reduce RAM usage and improve performance on Raspberry Pi 4.
- **Able to run on Raspberry Pi 4:** The model must run within the Pi 4's hardware limits.
- **Concise Reasoning Capability:** The model should be able to toggle between verbose and concise reasoning modes; concise outputs are essential for real-time navigation commands.
- **Models download size:** The Raspberry Pi 4 relies on a 32 GB SD card for storage, so it's important that models are lightweight and take up minimal space.

### 6.3.2 LLM Evaluation

Our team evaluated the following models: *smoll2 360M*, *Gemma3-tools:4b*, *Qwen3:0.6b*, and *Qwen2.5:0.5b* based on our selection criteria, see *Table 13*. Our team assessed each model's ability to reason and follow spatial instructions based on this prompt:

- You are a robot navigating a 3x3 grid. The bottom-left corner is position (0,0) and the top-right corner is (2,2). Your goal is to move from (0,0) to (2,2). At each step, describe your movement using simple commands: "move up", "move down", "move left", or "move right". Do not skip steps or jump positions. Output each move in order until you reach (2,2).

An LLM passes the test if it outputs a valid sequence of steps that move from (0,0) to (2,2), using only simple movement commands without skipping any positions. Each model was run on Raspberry Pi 4, and each model's responses were collected and analyzed for quality. Each model was also evaluated based on our selection criteria.

### 6.3.3 Qwen:0.6b and Qwen2.5:0.5b Evaluation

We also evaluated Qwen3:0.6b and an alternative version, Qwen2.5:0.5b, using a different prompt. This test did not follow the standard 3x3 grid navigation task outlined in Section 6.4.2, and the results are therefore reported separately.

We tested Qwen3.0:0.6b and Qwen2.5:05b through the following prompt: “Suggest a movement command for the robot ‘forward’, ‘backward’, ‘left’, ‘right’, or ‘stop’. You should send one-word answers.” Our team wanted to observe how each model would respond when issuing commands to potentially move the robot in real time. A model would pass the test if it followed the instructions of returning one of the following: ‘forward’, ‘backward’, ‘left’, ‘right’, or ‘stop’. If the models returned anything else, it has failed the test. The goal of this test was to assess both model’s ability to issue simple navigation commands.

Table 13: Table of LLMs Tested

LLM Model	Model < 3B Parameters	Tool Support	Quantization Availability	Able to run on Raspberry Pi 4	Concise reasoning Capability	Model Download Size
smoll2 360M	✓	✓	✓	✓	✗	726 MB
Gemma3-tools:4b	✗	✓	✓	✗	✗	3.3 GB
Qwen3: 0.6b	✓	✓	✓	✓	✗	523 MB
Qwen2.5: 0.5b	✓	✓	✓	✓	✓	398 MB

## 6.4 Robot Testing

By Owen McKibbin

To meet the requirement of downloading and using the model on the Raspberry Pi to enable model instructed movement, the team evaluated two models on the Pi. During testing, instead of starting with the standard 3x3 grid navigation task outline in section 6.4.2, we began with a simple prompt formatted to produce a one-word response. Using this approach, the robot would be able to interpret the response from the LLM model to run a movement command.

Specifically, we tested the Qwen3.0:0.6b and Qwen2.5:05b models, using the prompt mentioned in 6.3.3. This prompt allowed us to view the robot's ability to respond and follow the model's movement commands, and the integration of the model and robot. The results showed that the model produced appropriate responses, which were well read by the robot, and produced proper movement in the prompted direction.

## 7.0 Methods

By Owen McKibbin

To accomplish our project objective of making the robot fully autonomous through LLM decision making, the group has selectively tested and chosen small LLMs that can fit on the Raspberry Pi and competently answer simple navigation questions. Our group has decided to

inject our AI model into the architecture of the robot as the primary decision maker when given data such as LiDAR point clouds, current location, obstacles, etc., to choose the best course of action to reach its destination.

Our team has decided to follow an Agile methodology for our project through the implementation of testing and verification of multiple iterations of work throughout the year. This process will allow our team to spot errors and resolve conflicts ahead of time before they become detrimental to the project. Agile methodologies have been covered extensively in SYSC4106 (Software Economy and Project Management) which has helped us transition quickly into this approach for our own project.

To establish the requirements of the project and build our project schedule, we have applied skills learned through SYSC3120 (Software Requirements Engineering) to formulate a list of requirements specific to the project. In addition, our experience studying Software Architecture and design in SYSC4120 has prepared us to make key design decisions on the injection of our ai model into the system.

Lastly, we have decided to build a dashboard web app linked to the robot and GitHub repository to publish metrics relating to the robot's performance and overall project progress. This allows us to have a clear picture of the progress of our project and overviews of the current state of testing and model performance.

## **8.0 Analysis**

### **8.1 Analysis of Qwen3:0.6b and Qwen2.5:0.5b Performance**

By Tommy Phang

Following the prompt evaluation outlined in Section 6.3.2, we observed that Smoll2 360M took 5.7 seconds to start generating a prompt; it began generating python code, failing to adhere to the instructions. Gemma3-tools:4b was unable to run on Raspberry Pi 4 due to it not having the required memory of 5.8 GB of usable RAM. Gwen3:0.6b took 6.5 seconds to start generating a prompt; it successfully followed instructions and reached (2,2). For now, we have decided to go with some form of Qwen since it passed the prompt test and met our selection criteria.

Following the prompt evaluation outlined in Section 6.3.3, through observation, our team has observed that both Qwen3:0.6b and Qwen2.5:0.5b could generate navigation commands to be sent to the robot. Based on the recorded data (see Appendix A), Qwen3:0.6b exhibited an average response time of 29,200 ms and occasionally failed to generate a response entirely. In contrast, Qwen2.5:0.5b achieved a significantly faster average response time of 2,416 ms and consistently produced responses without failure.

As mentioned in section 6.4, our team did robot testing evaluating how both Qwen3:0.6b and Qwen2.5:05b would perform in the robot. Due to both LLMs large response time the robot moved in a stop and go manner, resulting in choppy motion.

Even though Qwen3:0.6b and Qwen 2.5:0.5b were able to move the robot, our team found that both models would occasionally fail to follow instructions to respond with one-word outputs. The core issues we encountered with both models were inconsistent response times and an inability to adhere to strict response formats. The robot requires continuous, low latency inputs to avoid choppy motion, but both models produced commands in unpredictable bursts, with response times ranging from several seconds to several milliseconds (see Appendix A).

Based on the tests conducted with Qwen3:0.6b and Qwen2.5:0.5b, we identified that stable, predictable response behavior and strict adherence to output formatting are important qualities for an LLM when used to control navigation on our robot. From our observations, we hypothesize that for an LLM to be viable for full scale navigation, in addition of the mentioned qualities it must also achieve response times within the low hundreds of milliseconds. However, further testing is required to validate this hypothesis. Given the current performance limitations, our team is now shifting focus toward leveraging LLMs for smaller, more contained tasks rather than full autonomous navigation or other applications requiring low response times.

## 8.2 Analysis of Current System Architecture

By Jonas Andaya, Tommy Phang, Owen McKibbon, Connor Yelle and Umniyah Mohammed

Before beginning the project, the team needed a clear plan for integrating AI throughout the system. Thus, a thorough analysis of the current system architecture was conducted to determine if the integration could be achieved.

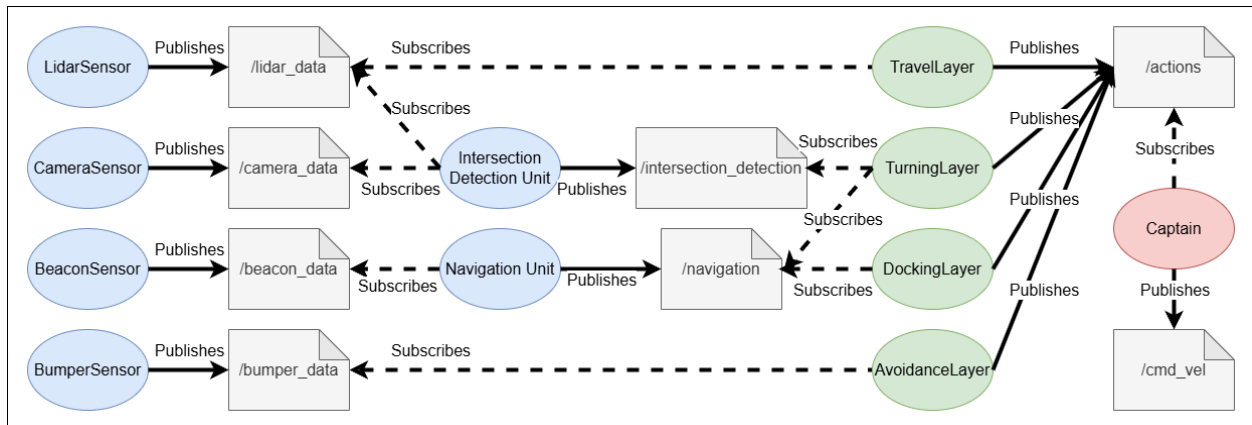


Figure 5: The system architecture from the previous year's version of the system

This diagram illustrates how the system’s various sensors publish data to processing nodes, which then communicate with the control layers. The captain node reads data from the layers and sends movement commands to the robot.

It was apparent to the team that to successfully fulfil the project’s objectives, the LLM agent had to be injected into as many aspects of the system as possible. The team proposed several strategies for achieving this, and two approaches were derived: vertical partitioning and horizontal partitioning.

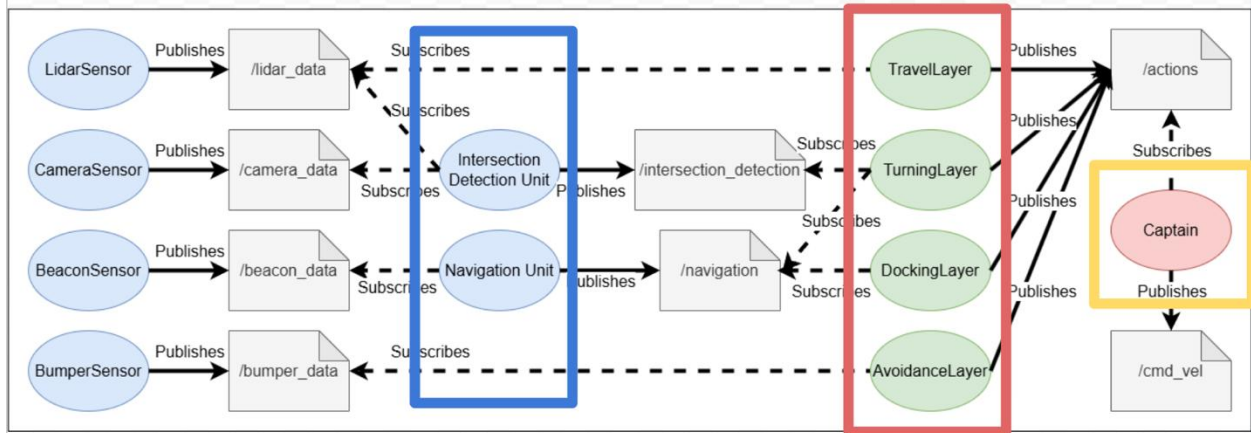


Figure 6: Vertical Partitioning of the Architecture

In the illustrated vertical partitioning of the architecture (Figure 6), there are three main components in the system where the LLM agent will be incrementally integrated. The blue box highlights the processing unit nodes, which receive and process sensor data for decision-making. The red box contains the behavior layers responsible for navigation and control. The yellow box includes the captain node, which coordinates data into specific actions for the robot. The LLM agent will be injected into each node sequentially, starting with minimal functionality and gradually expanding its role in each node as confidence in its performance increases.

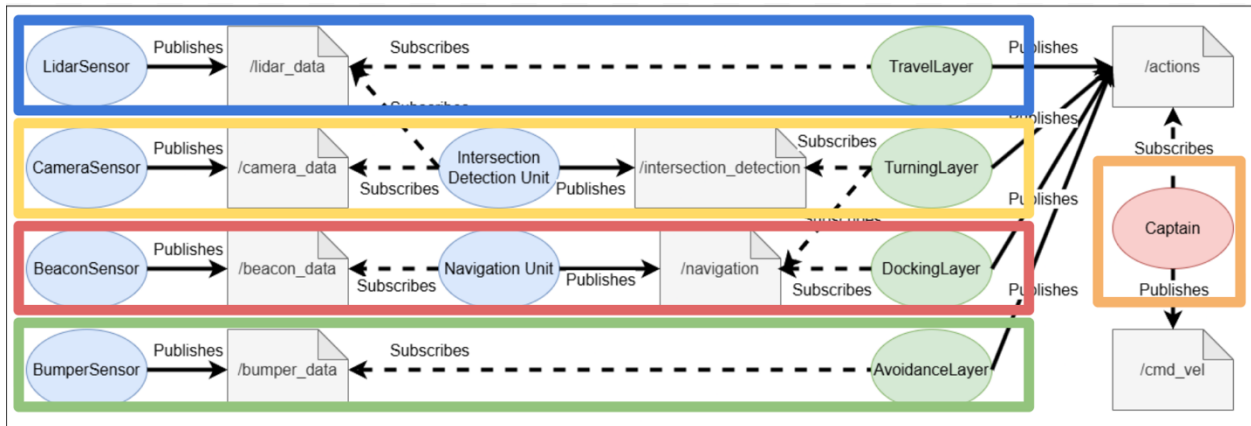


Figure 7: Horizontal Partitioning of the Architecture

Figure 7 illustrates the proposed horizontal partitioning of the architecture. In this approach, there are five components where the LLM agent will be incrementally integrated. Each colored box represents a horizontal slice of the system that shows a complete data processing path from sensor input to robot movement. This approach enables the LLM agent to be introduced into each functionality of the system one at a time, while minimizing the risk of disrupting other parts of the system. It also allows for a gradual scaling of the agent's role across the architecture to maximize its integration throughout the system.

To help facilitate the decision on which partitioning method to adopt, the team created a table comparing the pros and cons of vertical and horizontal partitioning to analyze the trade-offs between the two designs.

*Table 14: Table of Pros and Cons for Each Partitioning Method*

	Pros	Cons
Horizontal Partitioning	▪ Complete data pipeline for each LLM injection     ▪ If an LLM injection in a given slice is completed, it is considered finished and should not interfere with the injection in other pipelines.	▪ Requires more agents     ▪ Debugging can be challenging due to the scope of the slice; multiple files may need to be examined to locate bugs
Vertical Partitioning	▪ Easier to debug functionality in a single layer rather than an entire data processing stream     ▪ Testing is simpler when adding tests to individual layers versus entire streams, as layers often have very similar code and dependencies     ▪ Easier to coordinate agents within a single layer versus the entire horizontal slice	▪ Adding the LLM to every aspect of the system may be delayed until the end of the project     ▪ Less granular milestones, which could complicate agile methodology for achieving project objectives

- Separation of concerns, for navigation, data processing, and action.

After analyzing the table, the team decided to adopt vertical partitioning for LLM integration. This decision was made primarily to simplify debugging and testing, which are critical to the project's success. If the robot with LLM integration cannot be reliably tested, there is no way to ensure that the desired functionality is achieved. Another factor influencing this decision was the limited resources of the Raspberry Pi; the LLM is very memory-intensive, and only a small number of agents can run simultaneously. Vertical partitioning reduces the number of active LLM agents at any given time which makes it more feasible for the hardware.

## 9.0 Project Timeline

By Jonas Andaya, Tommy Phang, Owen McKibbon, Umniyah Mohammed and Connor Yelle

Table 15 shows the proposed timeline for the project. *Figure 8* shows a Gantt chart of the proposed timeline for the project

### 9.1 Proposed Timeline

By Jonas Andaya, Tommy Phang, Owen McKibbon, Umniyah Mohammed and Connor Yelle

*Table 15: Table of Milestones and Target Date of Completion*

Milestone	Target Completion Date
Complete installation of last year's system into the robot	September 18 <sup>th</sup> , 2025
Insertion of the LLM agent into the system	October 1 <sup>st</sup> , 2025
Complete setup of the Gazebo simulation	October 17 <sup>th</sup> , 2025
Project Proposal	October 17 <sup>th</sup> , 2025
Base dashboard implementation	October 31 <sup>st</sup> , 2025
Complete setup of the desktop-based Gazebo simulation	November 14 <sup>th</sup> , 2025
Inject LLM into navigation functionality	November 21 <sup>st</sup> , 2025
Complete dashboard with all metrics	November 28 <sup>th</sup> , 2025

Progress Report	December 5 <sup>th</sup> , 2025
Inject LLM into processing functionality	December 19 <sup>th</sup> , 2025
Oral Presentations	January 19 <sup>th</sup> -23 <sup>rd</sup> , 2026
Inject LLM into action/execution functionality	January 30 <sup>th</sup> , 2026
Comparison of LLM to last year's robot	February 27 <sup>th</sup> , 2026
Final Report Draft	March 20 <sup>th</sup> , 2026
Poster Fair	March 27 <sup>th</sup> , 2026
Final Report and Video	April 8 <sup>th</sup> , 2026

## 9.2 Gantt Chart

By Jonas Andaya

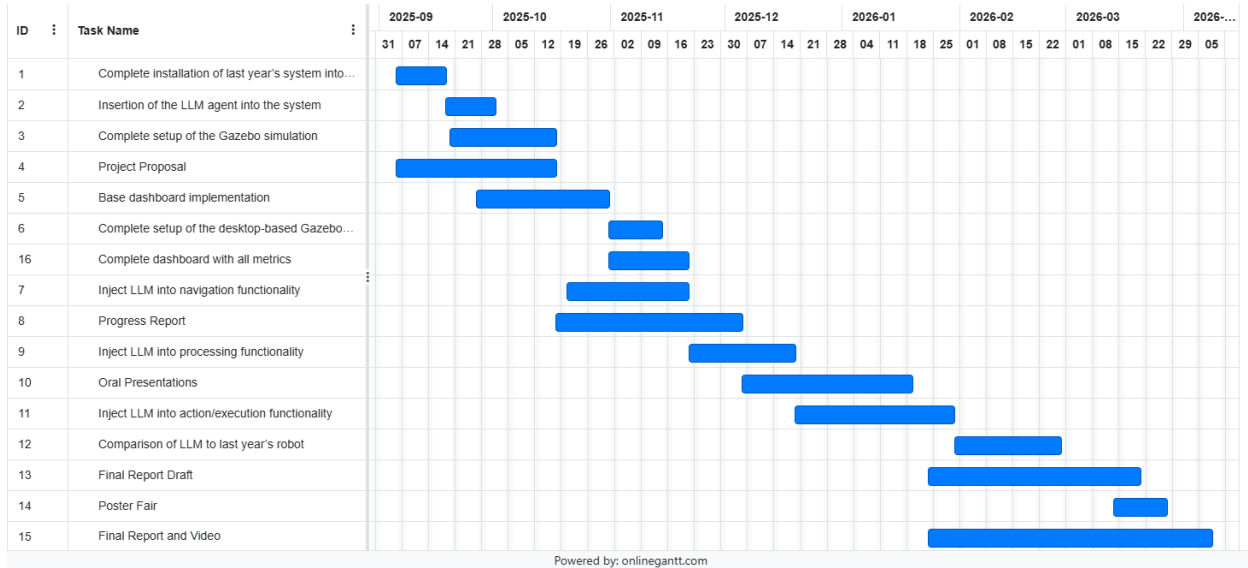


Figure 8: Gantt Chart of the Milestones

## 10.0 Risks and Mitigation Strategies

By Connor Yelle

Table 16: Project Risks and Mitigation Strategies

### Project Risk

The LLM agent injected into the Pi might be too slow to ensure true real-time decision making

Code could be pushed into the main branch of the codebase, which breaks the entire system, with no way to return

### Mitigation Strategy

The team will analyze alternate locations that the LLM could fit inside the system to lessen the dependency on the AI for real-time robot decision making

The team will utilize a CI/CD pipeline using GitHub and GitHub actions to ensure all code is tested before being accepted into the codebase. At least one team member will review the pull request to ensure the code conforms to the defined requirements.

The iCreate3 and the Raspberry Pi 4b could be damaged by external or internal forces during operation

The team will ensure that the robot is always in safe operating conditions. This means only testing the robot during non-busy times in the tunnels. The team will also ensure that they do not use software, libraries, or frameworks without prior research on if the Raspberry Pi can run it to ensure the Pi is not damaged during operation.

Electrostatic shock while handling the hardware components in the robot

The team will ensure to handle all robot hardware related material with utmost care and precaution to ensure nobody gets hurt. The Pi will be always held by the edges to ensure no internal circuitry becomes damaged or emits electricity that can shock someone.

## 11.0 Required Components and Facilities

By Connor Yelle

*Table 17: List of required Components/Facilities, Purposes, and Estimated Costs*

<b>Required Component/Facility</b>	<b>Purpose</b>	<b>Cost</b>
iCreate3	The robot itself. It will be performing the mail delivery using the system designed by our team. Already Purchased in previous years	0\$
LiDAR Sensor	The main sensor that makes the robot detect walls. Crucial for standard robot operation. Already purchased in previous years.	0\$
Raspberry Pi 4b	The computer board that is used for the robot. Runs the system, and the AI. Already purchased in previous years	0\$
Carleton Tunnels	The robot is designed to deliver mail inside of the tunnel system	0\$

under Carleton. Tunnel access is  
completely free

## 12.0 References

- [1] S. Macenski et al., “Robot Operating System 2: Design, architecture, and uses in the wild,” science.org, 2022 <https://www.science.org/doi/epdf/10.1126/scirobotics.abm6074> (accessed Oct. 1, 2025)
- [2] F. S. Marcondes et al., “Using Ollama,” SpringerBriefs in Computer Science, pp. 23–35, 2025, doi: [https://doi.org/10.1007/978-3-031-76631-2\\_3](https://doi.org/10.1007/978-3-031-76631-2_3). (accessed Oct. 8, 2025)
- [3] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566), Sendai, Japan, 2004, pp. 2149-2154 vol.3, (accessed Oct. 8, 2025)
- [4] T. S. Taylor, Introduction to Laser Science and Engineering. CRC Press, 2019, pp. 239-240, (accessed Oct. 9, 2025).

### 13.0 Appendix A: Test Data

[GIN]	2025/10/01 - 20:17:25	200	33.90205981s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:18:26	500	1m0s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:18:57	200	31.494387153s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:19:15	500	17.772581109s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:20:25	500	1m0s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:21:24	200	59.00035088s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:22:16	200	52.672553414s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:22:50	200	33.981436072s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:23:34	200	43.79380651s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:24:20	200	45.548441586s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:25:20	500	1m0s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:26:20	500	1m0s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:26:57	200	36.976111605s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:27:27	200	30.267523216s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:28:20	200	52.564717554s	127.0.0.1	POST	"/api/generate"

Figure A.9: Qwen 3.0 response times

[GIN]	2025/10/01 - 20:56:52	200	4.193984198s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:56:53	200	763.101204ms	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:02	200	8.444220096s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:03	200	605.446691ms	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:04	200	1.040949417s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:07	200	1.371202894s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:10	200	811.418978ms	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:13	200	1.030716211s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:16	200	583.893471ms	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:24	200	6.481548895s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:29	200	4.133978523s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:29	200	582.675645ms	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:57:59	200	29.068850954s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:58:01	200	2.080365011s	127.0.0.1	POST	"/api/generate"
[GIN]	2025/10/01 - 20:58:02	200	1.225444077s	127.0.0.1	POST	"/api/generate"

Figure A.10: Qwen 2.5:0.5b